CONTRIBUTIONS TO THE APPLICATION OF SNAPSHOT-BASED DATA-DRIVEN METHODS IN COMPUTATIONAL SCIENCE AND ENGINEERING

Gabriel Freguglia Barros

Tese de Doutorado apresentada ao Programa de Pós-graduação em Engenharia Civil, COPPE, da Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Doutor em Engenharia Civil.

Orientador: Alvaro Luiz Gayoso de Azeredo Coutinho

Rio de Janeiro
Julho de 2023

CONTRIBUTIONS TO THE APPLICATION OF SNAPSHOT-BASED DATA-DRIVEN METHODS IN COMPUTATIONAL SCIENCE AND ENGINEERING

Gabriel Freguglia Barros

TESE SUBMETIDA AO CORPO DOCENTE DO INSTITUTO ALBERTO LUIZ COIMBRA DE PÓS-GRADUAÇÃO E PESQUISA DE ENGENHARIA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO GRAU DE DOUTOR EM CIÊNCIAS EM ENGENHARIA CIVIL.

Orientador: Alvaro Luiz Gayoso de Azeredo Coutinho

Aprovada por: Prof. Alvaro Luiz Gayoso de Azeredo Coutinho
Prof. Adriano Maurício de Almeida Côrtes
Prof. Alessandro Reali
Prof. Alexandre Gonçalves Evsukoff
Prof. Luiz Landau

RIO DE JANEIRO, RJ – BRASIL
JULHO DE 2023

Freguglia Barros, Gabriel

Contributions to the Application of Snapshot-based Data-Driven Methods in Computational Science and Engineering/Gabriel Freguglia Barros. – Rio de Janeiro: UFRJ/COPPE, 2023.

IX, 154 p. 29, 7cm.

Orientador: Alvaro Luiz Gayoso de Azeredo Coutinho

Tese (doutorado) – UFRJ/COPPE/Programa de Engenharia Civil, 2023.

Referências Bibliográficas: p. 104 – 123.

1. Dynamic Mode Decomposition. 2. Scientific Machine Learning. 3. Snapshots Technology. I. Luiz Gayoso de Azeredo Coutinho, Alvaro. II. Universidade Federal do Rio de Janeiro, COPPE, Programa de Engenharia Civil. III. Título.

To my family.

Acknowledgements

The conception of this doctoral thesis is a compilation of the effort of different individuals on a direct or an indirect way. I would like to thank, in particular:

- Initially to God. Without Him, none of this would be possible.
- To my parents, Rogério and Rosangela, and to my sister, Nathália, for their support and unconditional love.
- To my advisor Alvaro L. G. A. Coutinho, for his trust, support, experience and patience. This work would not be possible without his spirit of leadership.
- I also thank professors Malú Grave, Alex Viguerie and Alessandro Reali, who were essential in the elaboration of this work. I appreciate our productive discussions and their collaboration in our published work. I extend my gratitude to professor José J. Camata, also fundamental to this work.
- To my colleague and friend, Rômulo Montalvão Silva, so many times indispensable in sharing knowledge, opportunities and experiences.
- To my friends, colleagues and professors from Juiz de Fora that always accompanied my journey and celebrated with me in my best moments and supported me in my worst.
- I thank, also, the research funding institutions that enabled the creation of this work. Thanks to CAPES for the funding of this research via a doctorate scholarship. I extend my thanks to ACM for the opportunity of participating, with partial funding, of the 2019 ACM Europe Summer School in HPC Computer Architectures for AI and Dedicated Applications. I also thank the SC19 organizing committee for the total funding of my participations as a SC19 student volunteer.

Resumo da Tese apresentada à COPPE/UFRJ como parte dos requisitos necessários para a obtenção do grau de Doutor em Ciências (D.Sc.)

CONTRIBUIÇÕES PARA A APLICAÇÃO DE MÉTODOS GUIADOS POR DADOS BASEADOS EM *SNAPSHOTS* EM CIÊNCIA COMPUTACIONAL E ENGENHARIA

Gabriel Freguglia Barros

Julho/2023

Orientador: Alvaro Luiz Gayoso de Azeredo Coutinho

Programa: Engenharia Civil

O uso de métodos guiados por dados no contexto de ciência computacional e engenharia vem ganhando destaque nas mais diversas áreas do conhecimento nos últimos anos. A grande disponibilidade e variabilidade de dados junto com o aperfeiçoamento de algoritmos de aprendizado de máquina e o desenvolvimento de *hardwares* com maior eficiência em aplicações em Inteligência Artificial justificam o aumento no uso desses métodos. Na presente tese de doutorado, apresentamos contribuições em métodos guiados por dados não-intrusivos baseados em *snapshots*. Nosso enfoque é na Decomposição em Modos Dinâmicos (DMD), método utilizado para caracterização de sistemas dinâmicos a partir de observações no tempo feitas por sensores espaciais. No caso de simulações numéricas, as observações são as soluções obtidas por cada passo de tempo durante toda a simulação. No entanto, a disponibilização dos dados nem sempre é feita de forma que a utilização do método seja imediata, demandando uma etapa de pré-processamento de dados. Neste trabalho, apresentamos o método e nossos resultados para experimentos numéricos com diferentes aplicações. Os dados para a aproximação via DMD são provenientes de simulações numéricas de correntes gravitacionais, deslocamentos de bolhas gasosas e modelos epidemiológicos. Consideramos que os dados demandam pré-processamento como descompressão, projeções de malha e/ou unificação de *snapshots* paralelos. Nesta tese, discutimos os resultados de reconstruções e predições temporais.

Abstract of Thesis presented to COPPE/UFRJ as a partial fulfillment of the requirements for the degree of Doctor of Science (D.Sc.)

CONTRIBUTIONS TO THE APPLICATION OF SNAPSHOT-BASED DATA-DRIVEN METHODS IN COMPUTATIONAL SCIENCE AND ENGINEERING

Gabriel Freguglia Barros

July/2023

Advisor: Alvaro Luiz Gayoso de Azeredo Coutinho

Department: Civil Engineering

The use of data-driven methods in the context of computational science and engineering has been gaining attention in many diverse research areas in the past years. The huge data availability and variability coupled with the improvement of machine learning algorithms and the development of efficient AI-guided hardware justify the increasing interest in these methods. In the current doctorate thesis, we present contributions in data-driven non-intrusive snapshots-based methods. We focus on the Dynamic Mode Decomposition (DMD), a method used to characterize dynamical systems from measurements in time made by spatial sensors. In the case of numerical simulations, the measurements are the solutions obtained at each time step. However, data is not always promptly available for ingestion, requiring an intermediate step of data preprocessing. In this study, we present the method and our results for numerical experiments in different applications. Data for DMD approximation is generated in numerical simulations of density-driven gravity currents, bubble and epidemiological problems. We consider that data requires preprocessing procedures such as decompression, mesh projection and unification of parallel snapshots. In this thesis, we discuss the results of reconstructions and short-time predictions.

Contents

Chapter 1

Introduction

Recent advances in hardware, algorithms, and data acquisition and storage brought attention to data-driven discovery in many scientific fields. The combination of the recent developments in Computational Science and Engineering (CSE), High Performance Computing (HPC), and Machine Learning (ML) led to the generation of a new subject entitled Scientific Machine Learning (SciML) which aims to use data-driven approaches to discover patterns, solve complex problems and extract insights from data using efficient and accurate strategies [1].

This thesis focuses on Dynamic Mode Decomposition (DMD), a SciML algorithm that extracts spatio-temporal coherent structures from snapshots. Even though DMD was developed for dynamical systems, it shares several concepts aligned with the current scientific machine learning state-of-the-art [2, 3]. Snapshots can be seen as simultaneous photography of measurements from a given number of spatial sensors. They can be obtained from numerical data (such as values computed for computational grids in numerical simulations) or experimental data (such as the temperature sensor measures of the ocean [4]). Both types of data require treatment: experimental data might be infused with noise or require additional treatment (i.e., acceleration data obtained from an accelerometer could be converted to displacement if required), while numerical data might be submitted to compression, Adaptive Mesh Refinement and Coarsening (AMR/C) or parallel computing. Here we focus on strategies to deal with data obtained from numerical data. Having DMD to be fueled with numerical data requires a scientific workflow that deals with the numerical simulations setup, data generation, data transformation, processing, and postprocessing. In order to approach DMD with numerical data, we need to go through concepts regarding CSE, HPC, Machine Learning, and SciML, which is the scope of the upcoming sessions.

1.1 Computational Science and Engineering

CSE is a multidisciplinary field at the intersection of mathematics and statistics, computer science, and core disciplines of science and engineering [5]. While CSE builds on these disciplinary areas, its focus is on integrating knowledge and methodologies from all of them and developing new ideas at their interfaces. As such, CSE is a field in its own right, distinct from any of the core disciplines. CSE is devoted to the development and use of computational methods for scientific discovery in all branches of the sciences, for the advancement of innovation in engineering and technology, and the support of decision-making across a spectrum of societally important application areas. CSE is a broad and vitally important field encompassing methods of HPC and playing a central role in the data revolution. While the development of HPC started many decades ago and has provided powerful computing capabilities, it has recently been recognized that integrating HPC into mathematical modeling, numerical algorithms, and large-scale databases of observations leads to a new paradigm in science and engineering. In conjunction with theory and experimentation, CSE is now widely recognized as an essential cornerstone that drives scientific and technological progress [5].

Although CSE is a large field with multiple applications and methodologies, this thesis focuses on the computational modeling of physical applications. Figure 1.1 shows the flow for creating a CSE model. We start by considering a conceptual model from a physical system, which is usually a problem of interest to industry and/or society. This physical system is usually governed by one or more partial differential equations (PDEs), yielding the appropriate mathematical model for our problem. In most cases, PDEs cannot be directly solved using analytical approaches, requiring approximation methods that allow the computation of approximated solutions for these equations given some hypothesis. That is described by the *discretization* step, where a numerical method is chosen such that it suits the mathematical problem. For larger problems, the *implementation* step requires attention such that the HPC strategies should be efficient enough to deal with the complexity of the problem. The simulation is initialized, usually consisting of solving multiple nonlinear systems of equations. The next step is *data analysis* which comprises data visualization, post-processing, and many other output data transformations. For many-query applications [6], when the system should be optimized, designed, or controlled, the cycle gets reiterated using the discoveries from the previous cycle.

This thesis focuses on solving engineering problems related to fluid dynamics and epidemiological applications. After defining the problem and the governing PDEs, we use numerical methods for discretization by applying a computational grid to the problem's spatial domain. The PDEs approximated in this thesis are also tran-

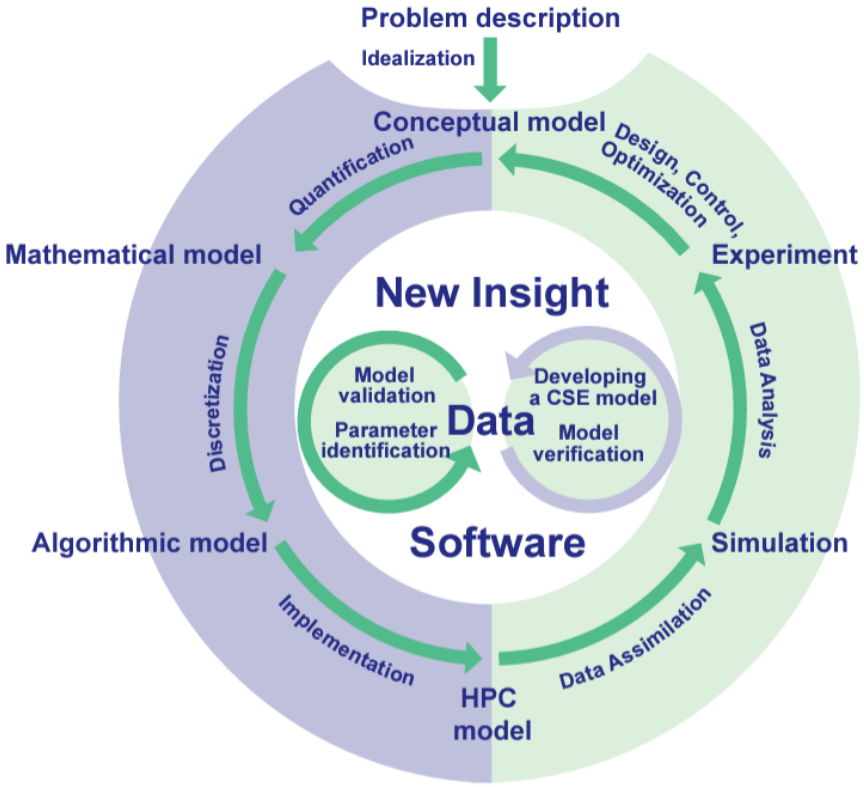


Figure 1.1: CSE cycle, from physical problem to model and algorithms to efficient implementation in simulation software with verification and validation driven by data. From [5].

sient, so the temporal evolution of the field is also approximated using numerical methods. More precisely, we use the finite element method (FEM) [7, 8] for spatial discretization and the finite difference method (FDM) [9] for time integration. Details on the spatial discretization and time integration can be seen in more detail in Appendix B. After complete discretization, the numerical simulation code is responsible for solving nonlinear systems, and the solution for these systems is the state vector for the problem at that specific instant. The solution and information about the mesh used to compute that solution are written on output files that will be imported into data visualization applications such as ParaView [10] and VisIt [11]. These tools are responsible for interpolating color maps using the solution and mesh information for proper result interpretation and analysis. This thesis refers to the solutions generated at every instant for the numerical simulation as *snapshots*. From the SciML perspective, snapshots are not strictly used for data visualization and decision-making, but they also contain underlying patterns that can be extracted to generate further analysis and to make predictions. This is covered in more detail in Chapter 2.

1.1.1 *In situ* visualization

One important aspect of the CSE cycle is when the problems are of larger complexity. In this situation, aside from the HPC strategies that deal with efficient system solving and computational speedup, it might be required to analyze the problem solution during simulation runtime. Several important features are obtained from using *in situ* visualization tools. First, it allows the visualization of very large computational problems. Simulations at the petascale/exascale level are too large to store and study directly using conventional postprocessing visualization tools, requiring special visualization strategies [12]. It is becoming customary to allow data analysis during simulation runtime instead of storing data for later analysis [13]. Also, *In situ* visualization tools allow simulation steering [14, 15], where the parameters for the simulation are manipulated during simulation runtime and data analysis during simulation runtime.

This thesis considers the Paraview Catalyst [16] as an *in situ* visualization tool that generates data during simulation runtime. Aside from data analysis and visualization, the information in the image files obtained through this strategy can allow faster predictions and analysis using DMD. That is, we consider data generated by Paraview Catalyst to fuel DMD, enabling the construction of the DMD approximation while the simulations are still running.

1.2 Data Science and Scientific Machine Learning

Data science itself is not a new concept. Humanity paved the way to modern statistics across the centuries [17] in many different scientific fields, even those not strictly related to mathematics. For instance, Florence Nightingale, a notorious nurse known for saving lives in the Crimean War during the XIX century, was a pioneer in using storytelling and data visualization to change sanitary conditions in England drastically, revolutionizing modern sanitary and medical conditions [18, 19]. That is, data science-related knowledge is long-lasting, although the term "data analysis" was coined in 1962 by John Tukey [20, 21], one of the inventors of the FFT algorithm [22], who envisioned the existence of a scientific field focused on learning from data. Since then, with the significant value obtained from data collection and analysis and advances in hardware and techniques, data science has evolved significantly with different branches, such as Artificial Intelligence (AI), which comprises Machine Learning (ML) and Deep Learning (DL), storytelling, and data engineering. One of these branches arises when data science methodologies are applied to CSE. Scientific Machine Learning (SciML) is a core component of artificial intelligence and computational technology that can be trained, with scientific data, to augment or automate human skills [1]. This emerging research area focuses on the opportunities and challenges in the context of complex applications across science and engineering and other interdisciplinary fields. That is, when we combine the possibility of exploring complex physical phenomena using CSE with the techniques that unravel information hidden in data in the ML background, we achieve more efficient and robust approaches that solve the challenges in modern engineering problems.

This combination can be translated, on the usual SciML framework - especially in the Reduced Order Modeling context - into the Offline and Online phases [23]. The Offline phase corresponds to the part of the process where data is being generated as the numerical simulations are being computed. During the offline phase, the CSE cycle described in Figure 1.1 is invoked. Modern applications usually demand larger computational resources and time, so High Performance Computing (HPC) strategies and hardware are usually employed in this phase. Conversely, the Online phase manipulates the generated data, extracts the underlying patterns in the data, and makes predictions and/or extrapolations. Although this division is simple for some cases, there are many situations, however, where the disparity between supercomputer computation speeds and I/O rates on the Offline phase is significant, meaning that data-generating applications must run concurrently with data reduction and/or analysis operations (including *in situ* visualization techniques), with which they exchange information via high-speed methods such as interprocess

communications. In other words, data can be generated faster than written to a file system, leading to a bottleneck in data storage and transfer [13]. This bottleneck affects the intersection between the Offline and Online phases, so data does not need to be completely available for the Online phase to be computed.

In terms of applications, a diverse range of computing theories and algorithms is currently used and developed under the SciML scope, such as large-scale optimization, inverse theory, reduced order modeling, uncertainty quantification, Bayesian inference, optimal experimental design, data assimilation, physics-informed deep learning, interpretable machine learning and reinforcement learning. These data-driven strategies, together with High Performance Computing (HPC), are transforming how we model, foresee, and govern intricate systems in complex applications such as turbulence, the brain, climate, epidemiology, finance, robotics, and logistics. [2]. These systems are usually nonlinear, dynamic, multi-scale in space and time, and high-dimensional, with significant underlying patterns that must be identified and modeled to achieve sensing, prediction, estimation, and control goals. For instance, Figure 1.2 illustrates the visual similarity between a picture of a real-world turbulent 3D complex flow and a figure of a numerical simulation of a laminar 2D flow over a circular cylinder. The first figure is a photograph from space of the von Kármán vortex street generated by a mountain in Rishiri Island, Japan, whose wake is visualized by the clouds, while the second figure is the flow over a circular cylinder, one of the most fundamental flows in fluids mechanics for its relevance in engineering settings. Although there is a significant gap in terms of physical complexity between the two pictures, we can observe the similar structures between them. Large and complex systems dictated by PDEs often present dominant underlying patterns [24]. Low-dimensional coherent structures express these patterns in high-dimensional data and can be extracted with scientific machine learning techniques, which would consider this valuable information for the eventual goal of sensing, prediction, estimation, and control. Some of these new techniques include robust image reconstruction from sparse and noisy random pixel measurements, turbulence control using machine learning, optimal sensor, and actuator placement, discovering interpretable nonlinear dynamical systems purely from data, and reduced-order models to accelerate the optimization and control of systems with complex multi-scale physics. The availability of vast amounts of data, enabled by remarkable innovations in low-cost sensors, orders-of-magnitude increases in computational power, and virtually unlimited data storage and transfer capabilities, is driving modern data science [2].

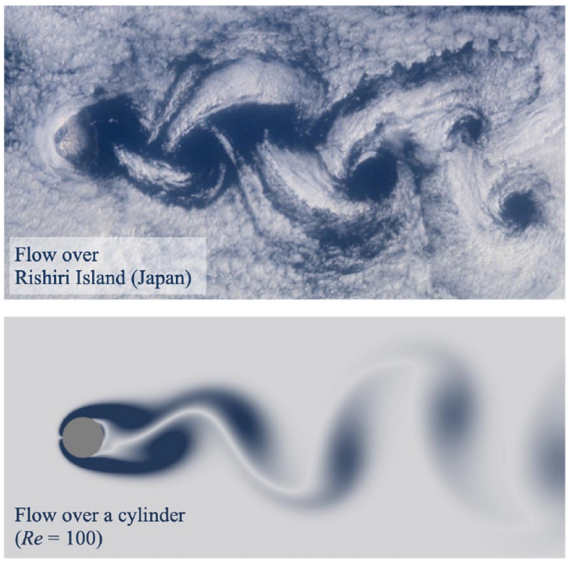


Figure 1.2: Visual comparison between the Von Kármán vortex street generated by the flow over Rishiri Island in Hokkaido, Japan, and the cylinder wake at low Reynolds number. From [25].

1.3 Motivation

The relationship between numerical simulations and data science lead to the development of several techniques that improve the efficiency and accuracy of decision-making in engineering. Data from numerical simulations, namely snapshots, usually contain underlying patterns that can be used to infer or analyze information regarding the physics of the problem being modeled. Snapshots-based SciML models are helpful for improving simulation efficiency, generating surrogate models, improving accuracy, and enabling data-driven modal analysis. Although there are several snapshots-based SciML models, our method of choice in this thesis is the Dynamic Mode Decomposition (DMD) model, an equation-free method that extracts the coherent spatio-temporal structures existent in data.

Given the nature of snapshots, it is challenging to generate a unique DMD code for all possible problems. First, each simulation output file is written in its own way and depends mainly on the library or source code employed, the file format used, the mesh considered, and the problem solved. Also, large numerical simulations are often solved using multiple parallel processors to improve performance, leading to multiple output files for the same computed time step. Although all described situations require some automation, they can be solved with proper data pre-processing. However, even after data is submitted to the preparation workflow, there are situations where data cannot be properly stored for being processed on DMD models. For instance, there are cases where numerical simulations compute solutions on different meshes during the simulation. These different meshes lead to

spatial tracking divergence, where a point in space having its field computed may not exist on a future time step given the mesh topology variation. Also, the number of nodes on a mesh may vary, prohibiting the usage of that data.

Another issue arises when data generated from numerical simulations is significantly large. Large data leads to more complex workflow architectures for DMD applications. One possible case is that, given that modern simulations may reach many millions and even billions of degrees of freedom, the output files may require a large storage size and lead to large I/O operations. Reducing disk pressure would enable the usage of the generated data in the Online Phase, where computational resources are more scarce. This occurs when the gap between CPU processing and I/O bandwidth becomes larger, which may be usual for simulations on extreme scale using HPC strategies and hardware. In this case, data generation is faster than data storage, requiring the calling of the DMD routine during simulation runtime [13].

These and many other situations are bottlenecks in the development of DMD methods. In this thesis, we address the described data issues for numerical simulations enabling the usage of their outputs as DMD models input. We consider complex nonlinear numerical simulations as input for DMD models to guarantee the robustness of the data workflow. Although the contributions are made in the DMD context, they can be properly adapted to any snapshot-based SciML model (i.e., the Proper Orthogonal Decomposition).

1.4 Contributions and Published Materials

During the development of this study, the following materials were produced. They were published after peer review or in preparation with joint authorship:

- Published by Peer-reviewed Journals and Conference Papers:
  - Directly related to this thesis:
    1. DMD application to 2D density-driven gravity currents: Barros, G. F., Côrtes, A. M. A., Coutinho, A. L. G. A. Dynamic mode decomposition for density-driven gravity current simulations, *CILAMCE 2020 - Proceedings of the XLI Ibero-Latin-American Congress on Computational Methods in Engineering*, 2020.
    2. Mesh projection strategy to circumvent AMR/C snapshots with applications to fluid dynamics and epidemiological models: Barros, G.F., Grave, M., Viguerie, A., Reali, A., Coutinho, A. L. G. A. Dynamic mode decomposition in adaptive mesh refinement and coarsening simulations. *Engineering with Computers* 38, 4241–4268, 2022. `https://doi.org/10.1007/s00366-021-01485-6`.

3. DMD application for snapshots of multiple compartments: Viguerie, A., Barros, G. F., Grave, M., Reali, A., Coutinho, A. L. G. A. Coupled and uncoupled dynamic mode decomposition in multi-compartmental systems with applications to epidemiological and additive manufacturing problems. *Computer Methods in Applied Mechanics and Engineering*, 391, 2022. https://doi.org/10.1016/j.cma.2022.114600.
4. DMD application on computational oncology: Viguerie, A., Grave, M., Barros, G. F., Lorenzo, G., Reali, A., and Coutinho, A. L. G. A. Data-Driven Simulation of Fisher–Kolmogorov Tumor Growth Models Using Dynamic Mode Decomposition. *ASME. Journal of Biomechanical Engineering.* 144(12): 121001, 2022. https://doi.org/10.1115/1.4054925.
5. How data compression affects DMD: Barros, G. F., Grave, M., Camata, J. J., Coutinho, A. L. G. A. Enhancing dynamic mode decomposition workflow with in situ visualization and data compression. *Engineering with Computers*, 2023. https://doi.org/10.1007/s00366-023-01805-y.

– Other contributions:

1. Barros, G. F., Côrtes, A. M. A., Coutinho, A. L. G. A. Parallel Solution of 3D Ohta-Kawasaki Nonlocal Phase Field Model in FEniCS. *CILAMCE - PANACM 2021 - Proceedings of the Ibero-Latin-American Congress on Computational Methods in Engineering*, 2021.
2. Barros, G. F., Côrtes, A. M. A., Coutinho, A. L. G. A. Finite element solution of nonlocal Cahn–Hilliard equations with feedback control time step size adaptivity. *International Journal for Numerical Methods in Engineering*, 122(18) 5028–5052, 2021. https://doi.org/10.1002/nme.6755.
3. Grave, M., Viguerie, A., Barros, G. F., Reali, A., Coutinho, A. L. G. A. (2021). Assessing the Spatio-temporal Spread of COVID-19 via Compartmental Models with Diffusion in Italy, USA, and Brazil. *Archives of Computational Methods in Engineering*, 28(6) 4205-4223, 2021. https://doi.org/10.1007/s11831-021-09627-1.
4. Grave, M., Viguerie, A., Barros, G. F., Reali, A., Andrade, R. F. S., Coutinho, A. L. G. A. . Modeling nonlocal behavior in epidemics via a reaction–diffusion system incorporating population movement along a network. *Computer Methods in Applied Mechanics and Engineering*, 401, 2022. https://doi.org/10.1016/j.cma.2022.

115541.

- Presentations by the author:
  - Directly related to this thesis:
    1. Barros, G. F., Côrtes, A. M. A., Coutinho, A. L. G. A. Extracting Spatio-Temporal Coherent Structures of Turbidity Current Simulations, *14th WCCM & ECCOMAS Congress*, 2021.
    2. Barros, G. F., Grave, M. Camata, J. J., Coutinho A. L. G. A. Snapshots Construction Using In Situ Visualization Tools for Data-driven Reduced Order Modeling. *USNCCM16*. (2021).
    3. Barros, G. F., Grave, M., Viguerie, A., Camata, J. J., Reali, A., Coutinho, A. L. G. A. Data Pipeline for Dynamic Mode Decomposition. *RAMSES: Reduced order models; Approximation theory; Machine learning; Surrogates, Emulators and Simulators.* 2021.
  - Presentations with contributions from the author:
    1. Silva, R. M., Jagtap, A., Shukla, K., Barros, G. F., Coutinho A. L. G. A., Karniadakis, G. E. PINN-based Reconstruction of Particle/Density-driven Gravity Flows. *8th European Congress on Computational Methods in Applied Sciences and Engineering, ECCOMAS Congress*, 2022
    2. Guerra Bernadá, G. M., Grave, M., Barros, G. F., Silva, R. M., Santos, T. L., Gesenhues, L., Camata, J. J., Côrtes, A. M. A., Elias, R. N., Rochinha, F., Coutinho, A.L.G.A. Advances in the simulation of gravity currents. *5th International Conference on Multi-Scale Computational Methods for Solids and Fluids*, 2021.
- Awards
  - Special mention at the Arts & Science Contest - World Congress on Computational Mechanics 2020[1].

It is important to highlight that all the works listed before were developed as contributions to different projects related to Finite Element Simulation of Turbidity Currents, conducted and/or hosted at the High Performance Computing Center, COPPE/Federal University of Rio de Janeiro[2].

[1] https://www.coppe.ufrj.br/pt-br/planeta-coppe-noticias/noticias/arte-e-ciencia-aluno-da-coppe-recebe-mencao-honrosa-em-congresso

[2] http://www.nacad.ufrj.br

1.5 Structure

This thesis is presented as follows: A brief introduction of SciML methods on modern problems is described in Chapter 1, as well as our objectives and the structure of this manuscript. In Chapter 2, we focus on Snapshots-based Scientific Machine Learning models. We focus on the literature review of Dynamic Mode Decomposition (DMD), our SciML method of choice. We present the mathematical background, an algorithmic view of the method, and the current state-of-the-art of DMD. We also describe in detail the DMD workflow. We highlight the main components of the workflow, including the data transformation step, the core of this thesis. In this phase, we investigate DMD, where complex snapshot architectures are provided. That is, we solve problems where:

- Snapshots are available with varying dimensionality;
- Snapshots written in parallel output files;
- Snapshots are submitted to data compression;
- Snapshots are generated during simulation runtime (i.e., data streaming);

Chapter 3 presents the physical applications considered in this work. The data obtained used to fuel DMD is generated on numerical simulations regarding fluid dynamics and epidemiological models. We describe the models and the numerical simulation parameters used for each case. In Chapter 4, we present our results for the applications considered in this thesis. We conclude our study on Chapter 5. Also, we present four appendices in this thesis. Appendix A shows some preliminary studies on Snapshots-Based SciML. We describe Proper Orthogonal Decomposition (POD), another SciML algorithm with similar purposes, and compare the results to DMD for a simple problem where both methods could be applied. In Appendix B, we briefly describe the finite element method and the simulation codes used for generating the input data of our models. In this appendix, we also describe the Adaptive Mesh Refinement and Coarsening (AMR/C) procedure and how it can improve the performance of numerical simulations. Appendix C describes the Python library developed in this thesis. The complete source code that produced the results for each contribution of this thesis and the data generated from the simulations are available. Finally, in Appendix D, we describe the data compression techniques used in some of the applications solved in this thesis.

Chapter 2

Snapshots-based Scientific Machine Learning Models

One of the most important developments in CSE is to approximate PDEs systematically so that complex problems can be solved quantitatively using algebraic operations. Aside from the already mentioned FEM and FDM, several methods exist for this purpose, such as the finite volume method (FVM) and boundary element method (BEM). The list can increase substantially if we consider meshless methods and other approaches that approximate the differential operator existent in PDEs. In this thesis, we compute numerical solutions of complex problems from dynamical systems obtained through the spatial approximation of the PDEs via FEM and temporal approximation via FDM.

Given the spatio-temporal nature of PDEs, computational simulations often produce numerical vectors where a given quantity is computed after time and space discretization. For instance, Figure 2.1 show the vortex shedding of a flow over a cylinder in 2D. Each figure describes a state in time, and each vector $\mathbf{y}_i$ for $i \in 1, 2, ..., m$ contains the spatial description of each spatial node for m time instants. The figures that show the vortex shedding are practically arrangements of the vector where the computed value for a given row on $\mathbf{y}$ is mapped accordingly with the respective nodal coordinates and colored given its magnitude. The vector $\mathbf{y}$, namely a snapshot, is the output solution from the FEM simulation at each simulation instant.

Snapshots are a rich source of patterns to be extracted from scientific machine learning models. Although there are several snapshots-based SciML models, our method of choice in this thesis is Dynamic Mode Decomposition (DMD). DMD was first proposed as a dimensionality reduction technique that extracts dominant spatio-temporal coherent structures from high-dimensional time-series data [24, 26]. Given that the data considered in this thesis is obtained by the solution of the discretization of transient PDEs, our applications can be translated as the following

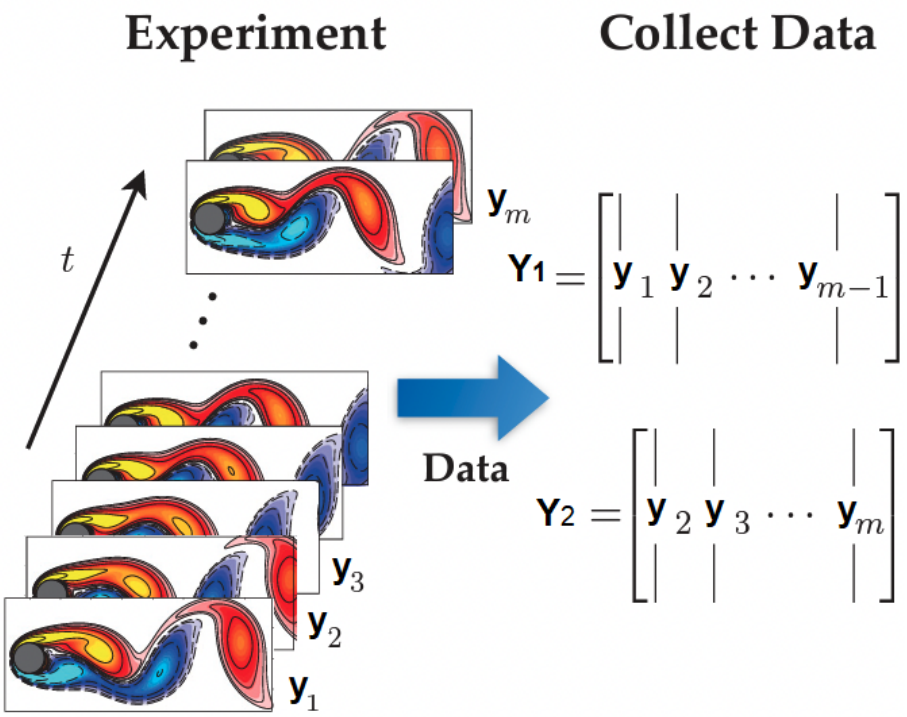


Figure 2.1: Vortex shedding snapshots. Adapted from [24].

discrete dynamical system

$$\mathbf{y}_{i+1} = \mathbf{F}(\mathbf{y}_i), \tag{2.1}$$

where $\mathbf{F}$ is an unknown function that dictates the evolution of the dynamical system. From the solution $\mathbf{y}_i$, the next state vector $\mathbf{y}_{i+1}$ can be computed. Given a collection of m measurements $\{\mathbf{y}_0, \mathbf{y}_1, \ldots, \mathbf{y}_{m-1}\}$, the snapshots matrix $\mathbf{Y}$ can be assembled by stacking the state vectors in columns such that

$$\mathbf{Y} = \begin{bmatrix} | & | & & | \\ \mathbf{y}_0 & \mathbf{y}_1 & \cdots & \mathbf{y}_{m-1} \\ | & | & & | \end{bmatrix} \tag{2.2}$$

where the matrix $\mathbf{Y} \in \mathbb{R}^{n \times m}$ is called snapshots matrix or snapshots dataset.

2.1 Dynamic Mode Decomposition

Dynamic Mode Decomposition first appeared in the fluid dynamics context [27, 28] and quickly gained popularity in the fluids community, primarily because it provides information about the dynamics of a flow, and is applicable even when those dynamics are nonlinear [29, 30]. DMD now appears on several different contexts such as biomechanical models [31], urban mobility [32], epidemiological systems [33], aeroelasticity [34], neuroscience [35] and financial trading [36]. A typical application

consists on stacking snapshots (i.e. simulated velocity/vorticity fields) to compute the DMD modes and eigenvalues, that, when taken together, describe the dynamics observed in the time series in terms of oscillatory components [29]. The method consists, basically, on a regression of data (represented as snapshots in time of a dynamical system) onto locally linear dynamics. The main idea is that, for all m snapshots existent in the snapshots matrix, DMD seeks a low-rank matrix $\mathbf{A}$ such that

$$\mathbf{y}_{i+1} = \mathbf{A}\mathbf{y}_i, \tag{2.3}$$

where $\mathbf{A}$ is chosen to minimize $||\mathbf{y}_{k+1} - \mathbf{A}\mathbf{y}_k||_2$ over all the $i = 0, 1, \ldots, m-1$ snapshots. Considering the snapshots matrix described on Equation 2.2, we can split the matrix into two matrices, such that

$$\mathbf{Y}_1 = \begin{bmatrix} | & | & & | \\ \mathbf{y}_0 & \mathbf{y}_1 & \cdots & \mathbf{y}_{m-2} \\ | & | & & | \end{bmatrix} \text{ and} \tag{2.4}$$

$$\mathbf{Y}_2 = \begin{bmatrix} | & | & & | \\ \mathbf{y}_1 & \mathbf{y}_2 & \cdots & \mathbf{y}_{m-1} \\ | & | & & | \end{bmatrix}, \tag{2.5}$$

where, for a given column number k, the state vector of the k-th column on $\mathbf{Y}_2$ is the successive solution from the k-th column on $\mathbf{Y}_1$. Finally, DMD can be posed formally as an optimization problem [37] as

$$\arg\min_{\mathbf{A}} ||\mathbf{Y}_2 - \mathbf{A}\mathbf{Y}_1||_F \tag{2.6}$$

where $||\cdot||_F$ denotes the Frobenius norm. The optimal solution for this problem is when $\mathbf{A}$ is such that

$$\mathbf{Y}_2 = \mathbf{A}\mathbf{Y}_1 \text{ and, consequently,} \tag{2.7}$$

$$\mathbf{A} = \mathbf{Y}_2\mathbf{Y}_1^{\dagger}, \tag{2.8}$$

being $\mathbf{Y}_1^{\dagger}$ the Moore-Penrose pseudoinverse [38] of $\mathbf{Y}_1$ such that $\mathbf{Y}_1^{\dagger} = \mathbf{Y}_1^T(\mathbf{Y}_1\mathbf{Y}_1^T)^{-1}$.

That is, using only observable data, it is possible to solve the linear system on Equation 2.7 and to construct a linear operator able to approximate the nonlinear dynamics of a complex system. The method is useful considering systems where no *a priori* information about the dynamics is known. In numerical simulations, for instance, this reflects on a non-intrusive characteristic of the algorithm, which can reconstruct the solutions (and, consequently, predict information that is not available in the dataset) without having access to the codes used for numerical simulations.

DMD gained more popularity when the relationship between DMD and the Koopman operator was proved [30, 39]. The Koopman operator is a linear, infinite-dimensional operator that describes the dynamics of any dynamical system - including nonlinear systems - on the Hilbert space of measurement vectors of the state [29, 40]. It is shown that DMD is a numerical approximation of the Koopman spectral analysis [30], which explains why DMD is applicable to nonlinear systems. DMD aims to construct the matrix $\mathbf{A}$ using measurements of the dynamical systems, being $\mathbf{A}$ a finite-dimensional linear model that approximates the infinite-dimension Koopman operator restricted to a measurement subspace spanned by direct measurements of the state $\mathbf{y}$ [37]. The operators are linear since the mapping in time is linear even though the dynamics that generated $\mathbf{y}_k$ are nonlinear.

Although DMD was proposed in the field of dynamical systems, we can look at the method's properties through the lens of SciML [3, 26, 41, 42]. In some works, the snapshots matrix is called training set [26] and machine learning concepts like overfitting [43] and variations in the loss function [44, 45] are discussed. The link to machine learning is natural, given the regression nature of DMD. For instance, we can think of DMD as a regression that can be trained by minimizing the loss function on Equation 2.6 using the training data on $\mathbf{Y}$ and, given the convex nature of the optimization problem, the least-squares approach considered on the construction of the pseudoinverse can be used to find the optimal parameters (matrix $\mathbf{A}$). The loss function can be altered depending on the DMD premises. For instance, on [26, 46], the authors add a rank-r restriction to the optimization problem to account for the assumed modal structure of the system and on [45] the authors modify the loss function to account for reducing the errors in older snapshots as new data arrives. Some works discuss, for instance, some preprocessing methods on DMD that are usual to ML regression problems, such as mean subtraction [47], which bring further implications to DMD, especially regarding the temporal frequencies obtained in the SVD algorithm. It is also common to see contributions in the literature where the authors mixed DMD with other well-established ML/SciML concepts [26, 48].

The simplicity, robustness, agnostic properties, strictly data-driven behavior, and mathematical foundations existent in the method reflected a growing interest in multiple research areas in the past years. Figure 2.2 shows the number of publications on DMD over the past years. The data was obtained using all databases existent in the Web of Science website [49], and present the contributions of the DMD community in chemistry, physics, engineering, business economics, telecommunications, neurosciences, acoustics, and many others. The number of publications observed comprises articles ($82,17\%$), proceeding papers ($17,83\%$), and other documents such as book chapters and review articles.

Dynamic Mode Decomposition is usually considered for three different reasons

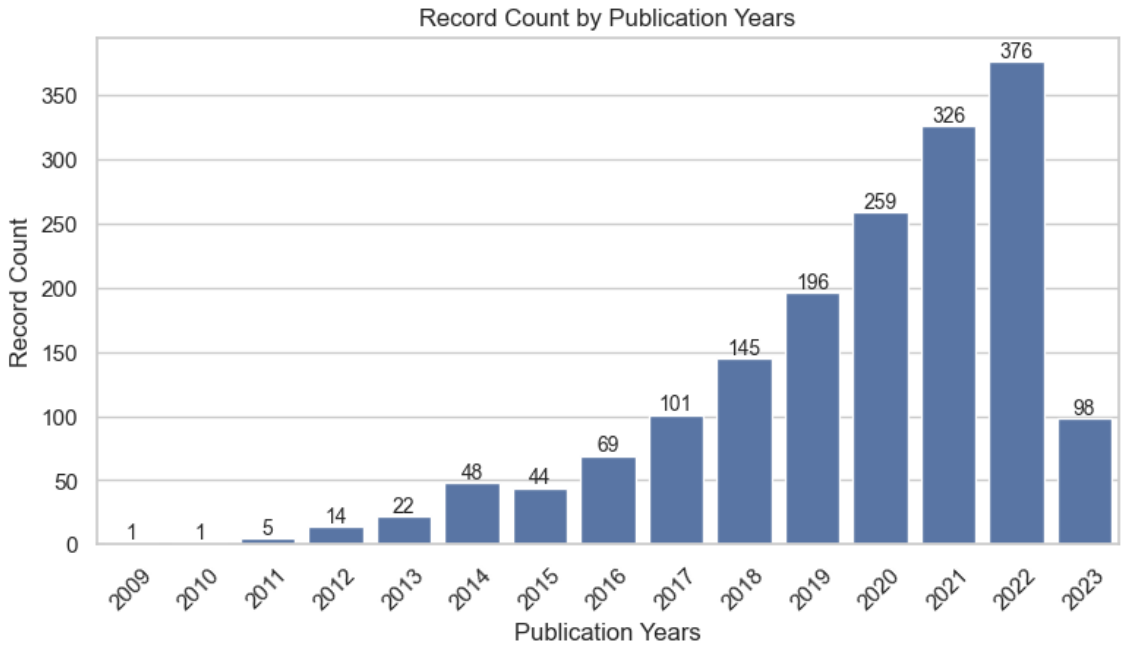


Figure 2.2: Number of publications related to DMD over the years. Data obtained from Clarivate Web of Science [49] on 04/05/2023.

[24]:

- Diagnostics: DMD naturally extracts key low-rank spatio-temporal features of high-dimensional systems, allowing for physically interpretable results regarding spatial structures and their associated temporal responses. These low-rank features can be used to characterize complex dynamical systems as a diagnostic tool. For instance, in [50], the authors used DMD for analyzing the flow dynamics for multi-phase flow in porous media, claiming that the information provided by DMD is more interpretable and practical to assess than 3D animations of the flow.

- State estimation and future-state prediction: DMD aims to construct linear maps of the dynamics of a complex system and is optimal within the data range collected. However, depending on the dynamics and some key strategies, such as intelligent data sampling or updating the regression, DMD can be used to estimate the subsequent observations of the state vectors using the information obtained from the original dataset. For example, in [51], the authors used streaming DMD - a variation of standard DMD where the basis is updated in real-time - for short-time predictions of energy generation in a wind farm with good results.

- Control: Due to the short-time future estimates property presented previously, DMD can be applied to control techniques. The short-time predictions can

be used to enable a control decision capable of influencing the system's future state. The work of [52] shows the development of DMDc - a variation of DMD that produces accurate approximations from complex systems with exogenous forcing - being suitable for control applications.

We now highlight several important contributions in the literature for using DMD. When DMD was proposed on [27], it was formulated in terms of a companion matrix, suggesting its connections with the Arnoldi algorithm [29]. It was further elaborated on [28], where the Singular Values Decomposition factorization was added as a first step on $\mathbf{Y}_1$ to allow the computation of the first r more relevant modes instead of the complete pseudoinverse matrix, increase the efficiency and numerical stability of Equation 2.8. Later, the exact DMD was proposed [29], generalizing the DMD algorithm for a larger class of datasets, especially non-sequential datasets, and proposing a more consistent calculation of the DMD modes.

DMD has some well-studied pitfalls. In terms of measurements, since DMD is based on linear measurements, the resulting models may not accurately capture nonlinear transients. To circumvent this issue, several works were proposed to identify nonlinear measurements that evolve linearly in time, establishing a coordinate system where the nonlinear dynamics appear linear. The extended DMD [53, 54], the kernel DMD [55], LANDO [43] and variational approach of conformation dynamics (VAC) [56, 57] enrich the model with nonlinear measurements. These special nonlinear measurements are generally challenging to represent, and deep learning architectures are being employed to identify nonlinear Koopman coordinate systems where the dynamics appear linear [58]. Another disadvantage of DMD is when data is noisy. It is known that computed eigenvalues are easily biased by noise in data [59]. This bias is a result of the fact that the standard DMD algorithm treats the data pairwise (matrix $\mathbf{A}$ maps the relation between $\mathbf{y}_i$ and $\mathbf{y}_{i+1}$). Some noise-robust variants are proposed on [59–61].

In terms of analysis, DMD was improved in several ways. By coupling Multiresolution Analysis (MRA) with DMD, the Multiresolution Dynamic Mode Decomposition (mrDMD) [4] is capable of naturally separating multiscale spatio-temporal features, providing effective means to uncover multiscale structures existent in the data. For instance, on [4], the authors showed that, even with a large volume of data containing the measured temperature in the oceans, the *El Niño* phenomenon, a rare event that would not be a dynamically relevant structure considering the dataset size, was successfully captured by DMD. Another important contribution in terms of analysis is the Dynamic Mode Decomposition with Control (DMDc) [52]. The DMDc method was originally developed to analyze measurement data collected from complex systems where exogenous forcing has been applied during the observation period. That is, DMD now takes the form $\mathbf{y}_{k+1} = \mathbf{A}\mathbf{y}_k + \mathbf{B}\mathbf{u}_k$ where $\mathbf{u}_k$ is the current

control and $\mathbf{B}$ measures the impact of the forcing $\mathbf{u}_k$ on the observations $\mathbf{y}_{k+1}$. The physics-informed DMD (piDMD) was recently proposed to add physical principles - such as symmetries, invariances, and conservation laws - to DMD, increasing the method's ability to generalize outside the training data and deal with noise. The significant increase in the DMD alphabet coupled with the observations made on Figure 2.2 confirms that DMD is an active research topic with several contributions in the past years.

Now we extend the numerical description for DMD on signal reconstruction and/or short-time predictions. By splitting the snapshots matrix on Equations 2.4 and 2.5 and defining the problem on Equation 2.8, we can compute the DMD modes and eigenvalues needed for signal reconstrution. For numerical purposes, computing the matrix $\mathbf{A}$ using pseudoinverses is not advisable. The product $\mathbf{Y}_2\mathbf{Y}_1^\dagger$ leads to a $A \in \mathbb{R}^{n\times n}$ matrix and the pseudoinverse $\mathbf{Y}_1^\dagger$ is generally ill-conditioned [62]. What characterizes DMD as a dimensionality reduction technique is its ability to circumvent the eigendecomposition of $\mathbf{A}$ by considering a rank-reduced representation $\tilde{\mathbf{A}}$. That is, we consider the SVD of $\mathbf{Y}_1$ such that

$$\mathbf{Y}_1 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T, \tag{2.9}$$

where $\mathbf{U} \in \mathbb{R}^{n\times(m-1)}$ and $\mathbf{V} \in \mathbb{R}^{(m-1)\times(m-1)}$ are matrices containing the left and right singular vectors and $\mathbf{\Sigma} \in \mathbb{R}^{(m-1)\times(m-1)}$ is a diagonal matrix with real, nonnegative and decrescent entries named singular values. The singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{m-1}$ are hierarchical and can be interpreted on how much the singular vectors influence on the original matrix $\mathbf{Y}_1$. According to the Eckart-Young theorem [63], when subjected to a truncation rank r, the matrix $\mathbf{Y}_1$ can be optimally approximated as

$$\mathbf{Y}_1 \approx \tilde{\mathbf{Y}}_1 = \mathbf{U}_r\mathbf{\Sigma}_r\mathbf{V}_r^T, \tag{2.10}$$

where $\mathbf{U}_r \in \mathbb{R}^{n\times r}$ is a matrix containing the first r columns of $\mathbf{U}$, $\mathbf{V}_r \in \mathbb{R}^{(m-1)\times r}$ contains the first r columns of $\mathbf{V}$ and $\mathbf{\Sigma}_r \in \mathbb{R}^{r\times r}$ is the matrix containing the first r singular values, where $1 \leq r < m-1$. The case where $r = m-1$ leads to Eq. 2.9. Considering the SVD decomposition, we can compute the pseudoinverse as

$$\mathbf{Y}_1^\dagger = \mathbf{V}\mathbf{\Sigma}^{-1}\mathbf{U}^T, \tag{2.11}$$

and, instead of computing the full rank pseudoinverse, a low-rank approximation can be computed as,

$$\tilde{\mathbf{Y}}_1^\dagger = \mathbf{V}_r\mathbf{\Sigma}_r^{-1}\mathbf{U}_r^T. \tag{2.12}$$

That said, a low-rank approximation of $\mathbf{A}$, named $\tilde{\mathbf{A}}$, can be computed as

$$\tilde{\mathbf{A}} = \mathbf{U}_r^T \mathbf{Y}_2 \mathbf{V}_r \mathbf{\Sigma}_r^{-1}. \tag{2.13}$$

Now we can compute the discrete eigenvalues λ_j and eigenvectors ϕ_j of $\tilde{\mathbf{A}}$. The eigenvectors are used to compute the DMD modes which, coupled to the eigenvalues, are sufficient to reconstruct the dynamics of the system measured [29]. For illustration, we compare the eigenvalues of $\tilde{\mathbf{A}}$ and $\mathbf{A}$ on Appendix A. A summary of the standard DMD algorithm is described in Algorithm 1. It is important to mention that step 6 of the algorithm computes the exact DMD modes (following the so-called exact DMD algorithm). The original DMD had a different strategy to compute the modes - now entitled projected DMD modes - where $\mathbf{\Psi}^{DMD} = \mathbf{U}_r \mathbf{W}$. It is also common to see the computation of the DMD modes being scaled depending on the application. The most common scaling factor is dividing the DMD mode by the corresponding eigenvalue in order to penalize spurious modes with large norms but quickly decaying contributions to the dynamics. However, this mode scaling factor is usually arbitrary [29].

Algorithm 1 Reconstruction of dynamical systems using the standard DMD method

INPUT: Snapshots $\mathbf{Y} = \{\mathbf{y}_0, \ldots, \mathbf{y}_{m-1}\}$
OUTPUT: Signal reconstruction $\tilde{\mathbf{y}}(t) \approx \mathbf{y}(t)$
1: Set $\mathbf{Y}_1 = \{\mathbf{y}_0, \ldots, \mathbf{y}_{m-2}\}$ and $\mathbf{Y}_2 = \{\mathbf{y}_1, \ldots, \mathbf{y}_{m-1}\}$.
2: Compute the SVD of $\mathbf{Y}_1$, $\mathbf{Y}_1 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.
3: Define the truncation rank r and compute $\tilde{\mathbf{A}} \coloneqq \mathbf{U}_r^T \mathbf{Y}_2 \mathbf{V}_r \mathbf{\Sigma}_r^{-1}$.
4: Compute eigenvalues and eigenvectors of $\tilde{\mathbf{A}}\mathbf{W} = \mathbf{W}\mathbf{\Lambda}$.
5: Compute continuous eigenvalues $\mathbf{\Omega}$ from discrete eigenvalues $\mathbf{\Lambda}$ obtained, $\omega_i = \ln(\lambda_i)/\Delta t_i$, where $\Delta t_i = t_{i+1} - t_i$.
6: Compute DMD modes $\mathbf{\Psi}^{DMD} = \mathbf{Y}_2 \mathbf{V}_r \mathbf{\Sigma}_r^{-1} \mathbf{W}$.
7: Compute vector $\mathbf{b}$, $\mathbf{b} = (\mathbf{\Psi}^{DMD})^{\dagger} \mathbf{y}_0$.
8: Reconstruct approximation $\tilde{\mathbf{y}}(t)$, $\tilde{\mathbf{y}}(t) = \sum_{i=1}^{k} b_i \psi_i \exp(\omega_i t)$

Remark: One can be confused with the DMD nomenclature. The original DMD, proposed on [27, 28], was the fundamental work that described the algorithm as a data-driven approach to extract global modes. The algorithm proposed by the authors of [27, 28] is different from Algorithm 1 in several aspects: i.e., the snapshots are mapped as a Krylov sequence, and the approximation of the matrix $\mathbf{A}$ is computed via a companion matrix $\mathbf{S}$. Later, on [29], after the mathematical foundations proven on [30, 39], a more generalized approach called exact DMD was proposed. This generalization allowed the analysis of a larger group of datasets and more robust and accurate modal calculations. The exact DMD is the current algorithm used in most works regarding DMD and has become the standard approach

considering the many DMD variants that are being proposed. From now on, the standard DMD refers to the exact DMD, which is the algorithm proposed on Algorithm 1 with its steps detailed in this section. This thesis considers two DMD algorithms: the standard DMD, described to this point, and the streaming DMD, which is yet to be discussed in this section.

Deciding the optimal truncation rank r for DMD is a crucial and not trivial choice. If r does not embrace the necessary dynamics to be inherited, DMD might not consider all the scales existent in the observable data. Also, if r is too large, errors regarding the pseudoinverse's ill-conditioning behavior may arise, especially if the difference in orders of magnitude between the first and the r-th singular values is too large. On Appendix A we illustrate how increasing r affects the DMD approximation. Also, the low-rank structures extracted from DMD are ranked in terms of dynamical importance [64]. This ranking is one crucial difference between DMD and other truncated SVD methods, such as Proper Orthogonal Decomposition (POD), where the energy of each (spatial) mode extracted is directly related to its corresponding singular value. This thesis uses the "relative energy" metric for a directed starting guess for choosing r. More details can be seen in Section 2.2.3.

The Singular Value Decomposition is the core procedure of DMD and is directly responsible for the dimensionality reduction of the problem. SVD can represent a significant part of the computational effort for high dimensional snapshots, meaning that improving SVD performance leads to significant CPU time gains. There are various algorithms for this purpose that depend on the application considered. One important contribution in this direction is seen on [65], where the method of snapshots was proposed. This method circumvented the difficulties in computation SVD on datasets where $n \gg m$. Another significant contribution is the randomized SVD (rSVD for short) algorithm [66], a non-deterministic algorithm able to compute the near-optimal low-rank approximation of a given large dataset with good efficiency. In this study, we employ the latter. Considering that $n \gg m$, the rSVD is described on Algorithm 2. The rSVD also presents two features:

- Oversampling: adding p columns to the random projection matrix decreases the variance of the singular value spectrum of the matrix obtained after the projection onto $\mathbf{P}$. Oversampling leads to an accuracy gain of the algorithm (as a rule of thumb, 5 to 10 extra columns suffice). In this study, we consider $p = 5$ columns in oversampling.

- Power iterations: projection of the snapshots matrix onto the $\mathbf{Q}$ subspace can lead to a matrix with slow decaying singular values, affecting the low-rank truncation. For this case, preprocessing $\mathbf{Y}$ by applying successive subspace iterations such as $\mathbf{X}^{(q)} = (\mathbf{X}\mathbf{X}^T)^q\mathbf{X}$ where q is the number of power iterations.

Since this can result in excessive computational cost, it is often a small number $q < 5$. This study considers $q = 1$ for all numerical results.

Algorithm 2 SVD factorization through the rSVD algorithm.

INPUT: $\mathbf{Y}_1$ matrix

OUTPUT: Truncated singular values $\mathbf{\Sigma}_r$ and truncated matrices containing the singular vectors $\mathbf{U}_r$ and $\mathbf{V}_r^T$.

1: Define a target rank $r < (m-1)$.

2: Create a random matrix $\mathbf{P} \in \mathbb{R}^{m-1 \times r+p_{over}}$. Here oversampling can be considered such that more random p_{over} vectors can be used to increase factorization accuracy.

3: Project $\mathbf{Y}_1$ onto the random matrix space such that $\mathbf{Z} = \mathbf{Y}_1\mathbf{P}$ and $\mathbf{Z} \in \mathbb{R}^{n \times r+p_{over}}$.

4: If desired, q power iterations can be processed at this point. This is, $\mathbf{Z} := [\mathbf{Y}_1(\mathbf{Y}_1^T\mathbf{Z})]^q$

5: Find an orthonormal basis $\mathbf{Q} \in \mathbb{R}^{n \times r+p_{over}}$ by applying the QR decomposition on $\mathbf{Z}$

6: Apply the regular SVD factorization to $\mathbf{Q}^T\mathbf{Y}_1$. This returns $\mathbf{\Sigma}_r$ and $\mathbf{V}_r^T$. Spatial modes $\hat{\mathbf{U}}_r \in \mathbb{R}^{r+p_{over} \times r+p_{over}}$.

7: Reconstructing high dimensional modes $\mathbf{U}_r$ by projecting the SVD modes on the dual space of $\mathbf{Q}$, that is, $\mathbf{U}_r = \mathbf{Q}\hat{\mathbf{U}}_\mathbf{r}$.

The current state-of-the-art of most DMD applications is the standard DMD equipped with the rSVD algorithm. However, depending on how data is provided and the DMD modes' desired use, DMD algorithms come in different flavors. For instance, the foundation of the standard DMD algorithm is the snapshots matrix. There are situations, however, where data is provided one snapshot at a time, and DMD is required for predictions, control, or analysis with the available data. The streaming DMD algorithm updates the DMD modes and eigenvalues online with every relevant snapshot added [67–69] without requiring the storage of the snapshots matrix for the computation of the DMD modes and eigenvalues. In this situation, a different algorithm for the SVD factorization is employed, namely the iSVD algorithm [70–72]. The remaining of the algorithm, however, is strictly the same. An alternative to the streaming DMD algorithm is the online DMD [45], where the authors slightly changed the loss function on Equation 2.6 to accommodate online learning. If one considers "forgetting" the contributions of older snapshots, the windowed DMD [45] is also an option. Table 2.1 shows some aspects of three different DMD algorithms.

In this thesis, we consider applications where data is provided on a batch of snapshots, enabling the construction of one single snapshots matrix for the problem and using standard DMD. In this case, the rSVD algorithm is invoked. However, we also consider some applications where data is provided during simulation runtime. For this purpose, the streaming DMD is preferred, and the iSVD for streaming data

Aspect	Standard	Streaming	Online
Store past snapshots	Yes	No	No
Track time variations	No	Yes	Yes
Real-time DMD Matrix	Yes	Yes	Yes

Table 2.1: Common DMD types and their aspects. Adapted from [45]

is considered [70, 72]. The iSVD is especially useful in real-time applications where DMD is applied for control or when simulations are excessively time-demanding, making data scarcely available in the simulation's early stages and progressively available. This data flow can enable insights from different stages of the dynamics instead of the usual standard DMD. In this study, we have implemented a slightly different version [71] of the usual iSVD algorithm, initially proposed for Proper Orthogonal Decomposition (POD) applications. In this algorithm, for a given collection of snapshots $\mathbf{Y}$, a new snapshot $\mathbf{y}$ updates the singular values and vectors such that

$$\begin{bmatrix} \mathbf{Y} & \mathbf{y} \end{bmatrix} = \begin{bmatrix} \mathbf{U}_Y & \mathbf{h} \end{bmatrix} \begin{bmatrix} \mathbf{\Sigma}_Y & \mathbf{l} \\ \mathbf{0} & g \end{bmatrix} \begin{bmatrix} \mathbf{V}_Y & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix}^T \tag{2.14}$$

where $\mathbf{U}_Y$, $\mathbf{\Sigma}_Y$, $\mathbf{V}_Y$ are the singular values and vectors of $\mathbf{Y}$, being $\mathbf{Y}$ the matrix containing the already existent snapshots during the increment of the new snapshot $\mathbf{y}$. If no snapshots are available, the matrices can be initialized as $\mathbf{\Sigma}_Y = [||\mathbf{y}||_2]$, $\mathbf{U}_Y = [\mathbf{y}/||\mathbf{y}||_2]$, $\mathbf{V}_Y = [1]$, being $\mathbf{y}$ the first snapshot available and $||\cdot||_2$ the $\mathcal{L}^2$ norm. Vectors $\mathbf{l} = \mathbf{U}_Y^T\mathbf{y}$ and $\mathbf{h} = (\mathbf{y} - \mathbf{U}\mathbf{l})/g$ and scalar $g = \sqrt{\mathbf{y}^T\mathbf{y} - \mathbf{l}^T\mathbf{l}}$ are responsible for updating the snapshots basis. It is important to mention that not all snapshots are needed to improve the basis. If $g < \epsilon_{SVD}$, where ϵ_{SVD} is a predefined threshold, the snapshot does not influence the new basis by any means. In this study, we consider $\epsilon_{SVD} = 10^{-15}$. It is also important to truncate $\mathbf{U}_Y$, $\mathbf{\Sigma}_Y$, $\mathbf{V}_Y$ to preserve the rank r. Also, an orthogonalization check is advisable on $\mathbf{U}_Y$ at each increment, invoking a QR decomposition on $\mathbf{U}_Y$ in case of loss of orthogonality of the basis. At the end of the acquisition of the snapshots, the generated SVD matrices will preserve the first k more relevant vectors of the included snapshots on the system. Both standard and streaming DMD algorithms are presented respectively in Algorithms 1 and 3.

2.2 DMD Workflow

In this section we describe the workflow considered for this study. Scientific workflows have become increasingly popular as a way of specifying and executing data-intensive analyses. In such systems, a workflow can be graphically designed by chaining together data transformations (e.g., snapshot creation, matrix factorization, etc.), where each transformation may take input data from previous

Algorithm 3 Reconstruction of dynamical systems using the Streaming DMD method

INPUT: Independent snapshots $\{\mathbf{y}_i\}$, number of DMD modes r, ϵ_{SVD}, SVD matrices $\mathbf{U}, \mathbf{V}, \mathbf{\Sigma}$ if already initialized.

OUTPUT: Signal reconstruction $\tilde{\mathbf{y}}(t) \approx \mathbf{y}(t)$

if $i = 0$ **then**

1: $\{\mathbf{y}_i\} = \{\mathbf{y}_0\}$. Matrices $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{\Sigma}$ must be initialized.

2: Initialize $\mathbf{U} = \mathbf{U}_Y \leftarrow [\mathbf{y}_0/||\mathbf{y}_0||]$

3: Initialize $\mathbf{\Sigma} = \mathbf{\Sigma}_Y \leftarrow [||\mathbf{y}_0||]$

4: Initialize $\mathbf{V} = \mathbf{V}_Y \leftarrow [1]$

5: Initialize $\mathbf{Y} = [\mathbf{y}_i]$

else

6: Compute vector $\mathbf{l} = \mathbf{U}_Y^T \mathbf{y}_i$

7: Compute scalar $g = \sqrt{\mathbf{y}_i^T \mathbf{y}_i - \mathbf{l}^T \mathbf{l}}$

8: Compute vector $\mathbf{h} = (\mathbf{y}_i - \mathbf{U}\mathbf{l})/g$

if $g > \epsilon_{SVD}$ **then**

9: Update $\mathbf{Y} \leftarrow [\mathbf{Y}\ \mathbf{y}_i]$ (optional)

10: Apply SVD on $\mathbf{Q} = \begin{bmatrix} \mathbf{\Sigma}_Y & \mathbf{l} \\ \mathbf{0} & g \end{bmatrix} = \mathbf{U}'\mathbf{\Sigma}'\mathbf{V}'^T$.

if $r_{count} \leq r$ **then**

11: Update $\mathbf{U} \leftarrow \begin{bmatrix} \mathbf{U}_Y & \mathbf{h} \end{bmatrix} \mathbf{U}'$

12: Update $\mathbf{V} \leftarrow \begin{bmatrix} \mathbf{V}_Y & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} \mathbf{V}'^T$

13: Update $\mathbf{\Sigma} \leftarrow \mathbf{\Sigma}'$

else

14: Update $\mathbf{U} \leftarrow \mathbf{U}_Y \mathbf{U}'[:, :r]$

15: Update $\mathbf{V} \leftarrow \mathbf{V}_Y \mathbf{V}'^T[:, :r]$

16: Update $\mathbf{\Sigma} \leftarrow \mathbf{\Sigma}'[:, :r]$

end if

end if

end if

17: Returns $\mathbf{U}$, $\mathbf{V}$, $\mathbf{\Sigma}$ and $\mathbf{Y}$ (optional). Computation of DMD modes and eigenvalues proceeds identically as seen in lines 3-8 of Algorithm 1.

transformations, parameter settings, and data coming from external data sources. In general, a workflow specification can be thought of as a directed graph (often acyclic), where nodes represent modules of an analysis and edges capture the flow of data between these modules [73], that allow researchers to organize their computations. Large-scale distributed computing infrastructures are typically used for the execution of scientific workflows, specially in machine learning applications [74].

In snapshots-based SciML models, part of the modules are located in the Offline phase and the remainder of the workflow nodes are located in the Online phase. Figure 2.3 illustrated the standard Offline-Online phases for snapshots-based SciML models. Often the Offline phase is conducted on HPC environments such as supercomputers or high performance cloud instances while the Online phase is performed on personal workspaces or smaller clusters. Following the standard workflow for snapshots-based SciML models, data generation and basis extraction are conducted on the Offline phase and then the Online phase can be initialized only once the data is properly stored and ready to be assessed.

For the numerical cases described in this thesis, the separation between Offline and Online phases may differ for each application depending on the data type provided and the necessary preprocessing, the DMD workflow is the same for all problems and is illustrated on Figure 2.4. That is, data is generated from numerical simulations, pre-processed, processed and eventually post-processed. Notice that the DMD block of graph nodes is entitled PADMe. This is due to our library[1] that is used for all applications in this thesis. The main necessity for creating PADMe was to simplify the creation of the snapshots matrix to be ingested on DMD. That is, PADMe would be responsible for tracking the simulations output files, open each file regardless of their extension, read the data and stack the snapshots into the snapshots matrix irrespective on how much pre-processing is required. For instance, in case of parallel output files for the same time step, PADMe would be responsible for tracking the communication nodes of the mesh to avoid multiple entries for the same set of spatial coordinates and stacking the snapshots correctly mapped. With the snapshots matrix created, one could use any libraries for snapshots-based SciML models such as PyDMD[75] or libROM[76]. That is, PADMe was conceived with the idea of assisting in data manipulation before DMD computation rather than an alternative option of the previous mentioned libraries. However, if desired, PADMe is also equipped with a modularized and robust DMD code. Each one of the steps of DMD can be replaced by other technologies (e.g. the SVD factorization can be replaced by autoencoders) without any change to the flow of the codes. We also created a post-processing and visualization module for properly analyze DMD output data such as eigenvalues, relative errors and other metrics. Appendix C describes

[1] `https://github.com/gf-barros/padme`

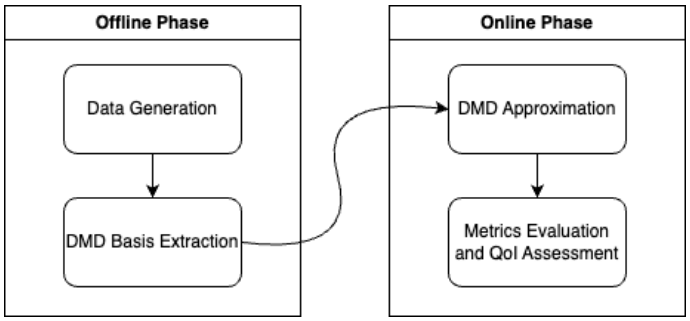


Figure 2.3: Simplified Offline and Online phases for snapshots-based SciML models. Adapted from [77].

the library in more detail.

Regarding the graph nodes of the DMD workflow, the standard DMD approach takes into consideration the data generated after the simulation is completed, thus a batch of snapshots is created on the `Snapshots Generation` phase, to be later ingested and processed in the DMD code. In situations where the standard DMD is employed, the Offline and Online phases are well separated and follow the simplified approach seen on Figure 2.3. There are some situations, however, where the barrier between Offline and Online phases is more diffuse. For instance, with the increasing gap between CPU processing and I/O bandwidth for complex HPC architectures - a situation that occurs mostly for simulations in extreme scale - data might take more time to be available for analysis. We can take advantage of that by providing data generated on-the-fly and executing the DMD models simultaneously with data generation. A workflow developed with this philosophy is the Online Data Analysis and Reduction (ODAR) motif [13]. In this case, the authors describe the modules - `Simulate`, `Reduce`, `Analyze` and `Store` - for data-driven approaches with different possible configurations, depending on the application. In this thesis, apart from the standard DMD, we also consider the DMD workflow for data provided through the execution of *in situ* visualization tools during simulation runtime, generating image files interpolated by color maps of the current state vector. This strategy takes the streaming DMD rather than the standard DMD, that is, the snapshots are now image vectors containing pixel values, the iSVD algorithm is invoked instead of the rSVD algorithm and image metrics are considered to measure the reconstruction of the solutions. This approach resembles the ODAR workflow in the sense that data analysis proceeds during simulation runtime and the workflow described in Figure 2.4 is preserved for this case. The difference lies in the repeated execution for each PNG file generation: a new image snapshot becomes available, the basis is updated and data is analyzed concurrently with the `Numerical Simulations` phase.

Remark: Workflows - term used mostly in the computer science community - and pipelines - term often used in the data science community - are slightly differ-

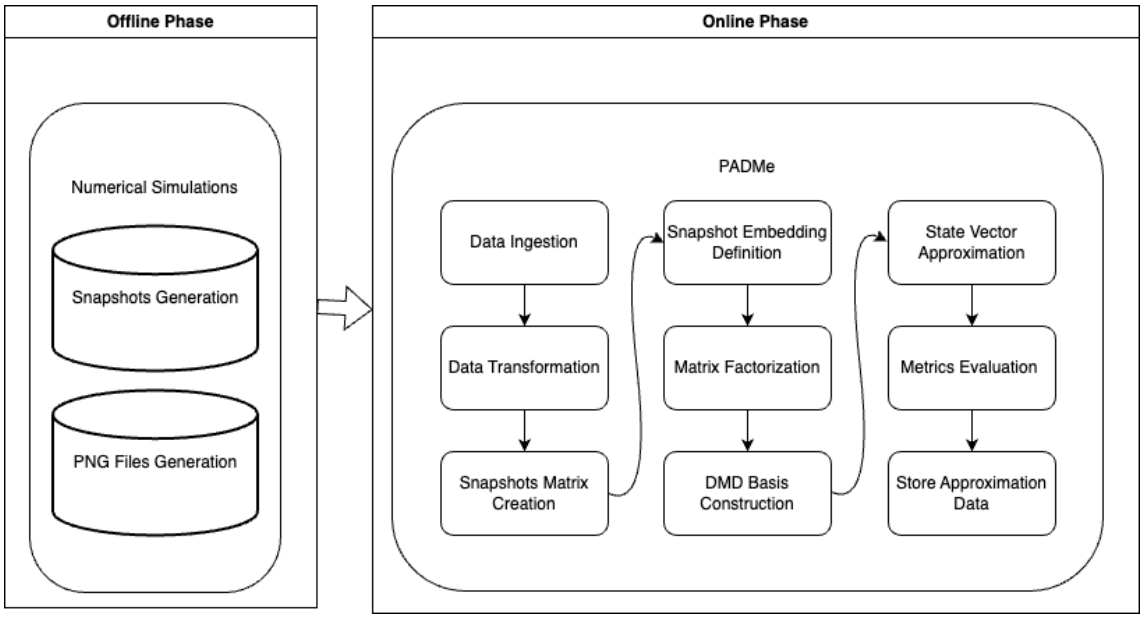


Figure 2.4: DMD Workflow used in this thesis. Most of our contributions are situated on the `Data Transformation` phases.

ent concepts. For instance, on [78] the author describes how a scientific workflow improved scientific analysis from raw data, including its generation. On [79], the authors describe the importance of having a systematic data pipeline that describes the data flow from raw ingestion until model validation and how to automate its creation. In both cases, raw data is converted into processed information for a given application. From a global perspective, we can address that a workflow composition deals with workflow components and data selection as well as the logical sequencing of tasks and resource provisioning [74], while pipelines can be seen as a description of the data flow that consists on several data transformations, modeling and validations where data gets transformed at each step until the generation of information itself. Figure 2.4 describes the workflow that ranges from the `Numerical Simulations` - and everything it comprises such as mesh creation, element integration, system solver and `Data Generation` - until storing the DMD results. There are transitions among different data flow that follows the usual machine learning pipelines format. In this thesis, given that the Offline and Online phases involve different types of algorithms and are executed in different types of computers - the Offline phase is often handled by numerical simulation codes and invoked in supercomputers, while the Online phase invoked on workstations -, the term workflow will be used to describe the complete data flow from numerical simulations until DMD post-processing.

In the upcoming sections, we describe briefly each one of the steps needed for the DMD computation from numerical data. For every DMD application tested on Chapter 4, this workflow will be revisited and a list of details for each graph node is highlighted. Some of the graph nodes are standard in the DMD computation

while others might have subtle differences (i.e. the `Snapshots Matrix Creation` varies for the standard DMD and the streaming DMD [45]) In this thesis, our main contributions are towards the `Data Transformation` phase. We describe those in more detail. The graph nodes existent in the Offline phase are also briefly described as well as the metrics used to measure the quality of the DMD approximation. The remaining graph nodes are grouped and described on a separate section.

2.2.1 The Offline Phase

This step regards the generation of numerical data used as input for DMD. The `Numerical Simulations` step is responsible for solving the nonlinear systems and generating the output files containing the nodal values for the solutions and mesh information. Different finite element libraries and output file formats are considered for each one of the applications described in Chapter 3.

The `Numerical Simulations` phase comprises the generation of data used in the DMD algorithm as well. Our DMD library is equipped with two DMD algorithms - the standard DMD and the streaming DMD - with different computation strategies. For the standard DMD, we consider a batch of output files generated from the simulations. This step is represented by the `Snapshots Generation` graph node in the DMD workflow. That is, nodal values as well as mesh information are stored in HDF5 [80]. The second DMD algorithm in our library, the streaming DMD, is employed when DMD is invoked during simulation runtime for each piece of data generation. For this purpose, we integrate our `Numerical Simulations` phase with ParaView Catalyst [16, 81], an *in situ* data processing visualization developed upon ParaView [10], to generate PNG snapshots during simulation runtime for some applications. In this situation, a predefined plane is considered and a color map is plotted for the solution interpolation and a figure file is created to store the pixels from the color map. Further information about this coupling can be seen on [68]. The PNG format is chosen since it requires significantly less storage than standard output files and allows faster transfer speed for data streaming. That is, instead of waiting for the simulations to be completely executed for data to be served to the DMD model, data can be provided during simulation runtime. This is illustrated by the `PNG Files Generation` graph node. Note that the two graph nodes regarding data generation are independent, such that they can be invoked independently.

Details for the numerical simulations considered in this thesis can be seen on Table 2.2, where each application is associated with the number of spatial dimensions simulated (n_{sd}), the usage of AMR/C schemes, the libraries used, the output file type and if the output files are submitted to data compression, as well as their location in this manuscript. We also show which section of this chapter belong to

Application	n_{sd}	Mesh	Library used	Output	Data Compression	Section
Lock-exchange	2/3	Fixed/Adaptive	`FEniCS`	HDF5	Yes	4.1
Bubble rising	3	Adaptive	`libMesh`	HDF5	Yes	4.2
SEIRD model	2	Adaptive	`libMesh`	HDF5	No	4.3

Table 2.2: Table showing the different data used to generate the results in this thesis.

the discussion of the results of each case. We considered `FEniCS`[82] and `libMesh`[83] as the finite elements libraries for this thesis.

2.2.2 Data Transformation

In this section, we explore snapshot technology. In most situations, snapshots not always are provided ready for ingestion in the DMD model. From simulation output files, data usually comes compressed, split into several parallel partitions or not adequate for fitting in the snapshots matrix due to mesh adaptivity. In this thesis, we focus on these three problems: parallel snapshots, adaptive solutions and compressed snapshots.

Parallel snapshots:

Numerical simulations of large problems usually demand high performance computing techniques and hardware. One common strategy for dealing with complex simulations is parallel computing. In this case, the mesh is divided into multiple partitions where each subdivision is computed on a processor core. All partitions are computed simultaneously and, after processing, the results are stored separately on different output files. When post-processing parallel snapshots, for visualization or DMD processing, the results should be assembled back to the original mesh. There are degrees of freedom solved exclusively on an individual core, while there are degrees of freedom computed in multiple partitions. To avoid miscalculation of the latter, proper handling of parallel snapshots must be taken into account. Figure 2.5 shows a finite element mesh being split into four partitions. If we count the geometrical nodes in both figures, we note that Figure 2.5(a) contains 49 nodes obtained from the original discretization. That is, the snapshots matrix for the finite element solutions computed in this mesh, considering that each node holds one degree of freedom, should contain 49 rows. By looking at Figure 2.5(b), we observe that the total number of nodes is $18 + 16 + 16 + 18 = 68$, even though it is the same mesh. This occurs because communication nodes appear in multiple partitions and are computed more than once.

Given this issue, for parallel snapshots, proper treatment to assemble snapshots is required, a grid search is needed to map the local degrees of freedom in the partitions

(a) Example mesh before partitioning. (b) Example mesh after partitioning.

Figure 2.5: Mesh submitted to partitioning for parallel computations. Results are computed in each partition and stored separately.

to the global indexation values. The algorithm that performs this manipulation is described in Algorithm 4 for a single snapshot.

Algorithm 4 Assembling a single parallel snapshot.

INPUT: Parallel snapshot $\{\mathbf{y}_i^j\}$ for all partitions $j = 1, 2, \ldots, n_{parts}$ at time instant i.
OUTPUT: Assembled snapshot $\{\mathbf{y}_i\}$ at instant i.
1: Load all partition data into memory $\forall j = 1, 2, \ldots, n_{parts}$ including degrees of freedom coordinates and mesh connectivity.
2: Store all coordinates $\mathbf{x} = \{x, y, z\}$ into an array. The coordinates are sorted to improve the efficiency of the computation of item 3.
3: Check for coordinates duplicates. In case of precision issues, compute the distance between each node and a subset of neighbor nodes of the mesh. If the distance is lesser than the element size (or the smallest element edge size), it can be considered as duplicated.
4: Compute the average of duplicate nodes.
5: Map the elements accordingly and position the average of the communication nodes in the correct row.
6: Store the assembled snapshot.

From this point, the parallel snapshot is now assembled and ready to be ingested on the DMD algorithm. It is important to guarantee that the mesh nodes are equally numbered for all snapshots. For most cases, it is only required to apply Algorithm 4 once, given that the map obtained in step 5 is valid and can be easily applied to all snapshots.

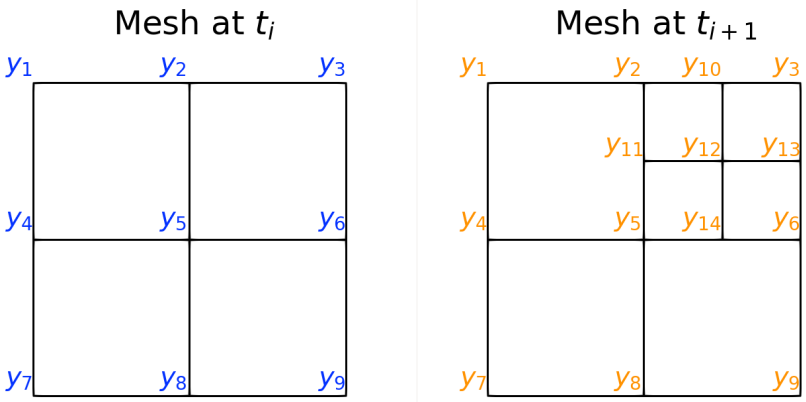
Solution projection:

Another issue with snapshots is when simulations consider AMR/C strategies. AMR/C methods manipulate the grid on numerical simulations such that for a given time step a new mesh is generated. Meshes can be refined - that is, more geometrical or interpolation nodes are added, enriching the description of the desired region -

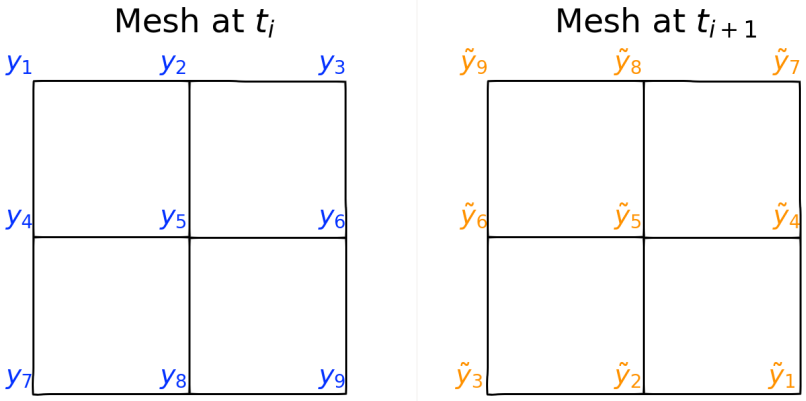
or coarsened, where the nodes are removed from the region, reducing the size of the resulting system. However, finite element simulations equipped with AMR/C strategies provide solutions in different function spaces, depending on the mesh used to compute the solution on a given time step. Spatial adaptivity on transient finite element simulations leads to meshes with a different number of nodes, numbering, and nodal coordinates. It can also lead to different mesh topologies and structures. For that reasons, snapshots obtained by AMR/C simulations cannot be stacked in columns to construct the snapshot matrix. Even if one considers an AMR/C strategy that restricts the adaptive meshes to preserve the dimensionality of the snapshots, the difference between the nodal coordinates of the various meshes will lead to misleading dynamics captured by DMD. Figure 2.6 shows two different situations where data generated from the simulations would create problems for its ingestion on DMD or any other snapshots-based SciML method. Figure 2.6(a) shows the same system being solved in two different meshes for different time steps. In this case, some nodes such as y_{14} do not exist on the mesh used to solve the system at t_i. Also the dimensions of the vectors are different, meaning that a snapshots matrix cannot be assembled. That is, vectors $\mathbf{y}_i \in \mathbb{R}^9$ and $\mathbf{y}_{i+1} \in \mathbb{R}^{14}$ would affect the assembly of a snapshots matrix. Another situation is described on Figure 2.6(b), where two meshes share the same topology but with different node ordering. This affects snapshots-based SciML methods as well given that for the same row we have measurements for different points in space. The snapshots matrix would be created, but nodal value y_1 would track a different spatial coordinate than $\tilde{y}_1$, affecting the approximation provided by DMD.

To circumvent this issue we consider applying the $\mathcal{L}^2-$projection [84] of the numerical simulation results for different meshes at each time-step into a reference target mesh. After analyzing all adaptive snapshots, we create a reference target mesh such that all spatial scales can be properly inherited. That is, even that all snapshots have different dimensionalities given the varying topology of the meshes, a reference mesh is created to support all snapshots such that the fixed mesh contains elements matching the smallest element size in the adaptive mesh to avoid coarse approximations. Details on mesh projection formulation and strategies are seen on Section B.3.3 in Appendix B. This tailored reference mesh is described in this work as target reference mesh and should not be confused with the target mesh generated during the AMR/C procedure.

We considered `Gmsh`[85], an open-source robust mesh generator, as our software of choice for defining and creating the target meshes for this study. The output for each time step is the snapshot with constant dimensionality n such that all nodes in space are correctly mapped and capturing the dynamics existent in the system. This strategy is relatively simple since the $\mathcal{L}^2-$projection consists of solving



(a) Different number of mesh nodes after AMR/C routine is invoked.



(b) Different nodal numbering for the same mesh topology.

Figure 2.6: Situations where different mesh topology might lead to problems when assembling the snapshots matrix.

a linear system where the generated matrix is a mass matrix, that, in the finite element context, is defined by a self-adjoint operator, enabling more efficient solvers. In terms of versatility, the $\mathcal{L}^2-$projection method is flexible because a solution obtained for a given mesh can be naturally projected onto reference target meshes with different topologies and dimensionalities. Also, since the mesh projection is a vital part of AMR/C algorithms, finite element algorithms frequently present efficient implementations of interpolation or projection techniques.

Depending on the context, this step can be done during simulation runtime. On [86], the authors evaluated the performance of mesh projection inside the numerical simulation code and observed that the computation time required for this step demanded 5% of the total simulation code time in the worst case in the tested applications. If one has access to the simulation code, it could be interesting to project the mesh using the resources of the Offline phase. Also, given that most situations involve solving a more complex problem (such as nonlinear equations), adding the cost of solving a linear problem with an self-adjoint operator is negligible. Also, given that the snapshots should be transformed for the sake of storing the snapshots before applying DMD in the workflow, the mesh projection during simulation runtime does not affect the following solutions. For this case, the mesh projection step in the `Data Transformation` phase would be skipped since the data would be already transformed during the Offline phase.

Considering the situation where data is provided in adaptive meshes, the `Data Transformation` phase should be invoked to make the snapshots available for processing in the DMD workflow. That is, the snapshots should be transformed regardless of the simulation, specially if one does not have access to the finite element codes used. Output files of various formats contain information regarding the mesh used (such as nodal coordinates and connectivities) for visualization purposes and, by properly reading these files, the solutions can be reconstructed under a finite element framework (such as `FEniCS` [82] or `libMesh` [83]) or on a code developed by the user. However, this totally non-intrusive approach can significantly increase the computational cost due to several I/O operations that are often less efficient when compared to computational intensive operations - such as the mesh projection operation itself. Since the mesh constantly varies in an AMR/C finite element simulation, each solution obtained in the simulation has to be imported, reconstructed under its original mesh, and projected. After the projection, the user can choose to dump the solution in the disk or stack the snapshots on the snapshot matrix. For the first case, another I/O operation would be invoked. This non-intrusive strategy is especially suitable (and restricted) to be used with DMD, which is also a non-intrusive algorithm. For other intrusive snapshot-based methods such as Proper Orthogonal Decomposition (POD), one needs to project finite element matrices onto the com-

puted basis and, therefore, requires access to the code [87]. The mesh projection algorithm for adaptive snapshots used in this thesis is then described on Algorithm 5. Note that we use the $\mathcal{L}^2$ projection procedure built-in in the `FEniCS` framework.

Algorithm 5 Projecting a single adaptive mesh snapshot into a fixed reference mesh.

INPUT: Adaptive snapshot $\{\mathbf{y}_{adap}\}$
OUTPUT: Assembled snapshot $\{\mathbf{y}\}$
1: Create the reference mesh using `Gmsh` aware of the scale of the problem and the minimum element size in the adaptive mesh.
2: Convert the generated `.msh` file into the `DOLFIN .xml` format file, readable by the `FEniCS` framework.
3: Load the snapshots data into memory including mesh information such as degrees of freedom coordinates and mesh connectivity.
4: Create the `Mesh()` object using the `FEniCS` library and the subsequent definitions of finite elements, function spaces and functions.
5: Call `MeshEditor` to insert global nodes and connectivity and reproduce the AMR/C mesh of that snapshot. The solution will be completely reconstructed with the `FEniCS` objects.
6: Instantiate `Mesh()` object for the reference mesh and its subsequent objects.
7: Apply the $\mathcal{L}^2-$projection of the adaptive solution onto the function space of the reference target mesh.
8: Stack the resulting snapshot into the snapshot matrix $\mathbf{Y}$ or output the projected solutions to disk.

Data compression:

The introduction of compression into a scientific workflow can benefit storage limitations, I/O bottlenecks, and bandwidth. The use of data compression is widely considered to improve computational performance and model accuracy in computational science and engineering [13, 88–94], and in machine learning [95–98] applications. In numerical simulations, it is common to compress the solution data to reduce disk pressure.

In this thesis, we consider some applications where the output files are submitted to data compression. We describe ZFP, the data compression strategy used in this thesis, on Appendix D. In this step, the Python package `hdf5plugin` is considered to decompress the compressed data into snapshots. The `hdf5plugin` provides HDF5 compression filters for a wide range of compression algorithms, having ZFP included. Differently from the previous sections, this is a straightforward task that requires no additional algorithm to the code. Importing the `h5py` and `hdf5plugin` packages suffices to decompress data for DMD processing.

2.2.3 Metrics Evaluation

Metrics are useful Machine Learning concepts for analyzing the performance of a given model. In this section, we describe the metrics used in all stages of the DMD workflow.

Relative energy:

Despite the choice of r for DMD demands some trial and error, some techniques can be used to find a good starting point. There are two methods for choosing r for numerical applications, described as hard threshold techniques [24]: the elbow method and the "relative energy" [6] method. The elbow method consists on plotting the singular values on a logarithmic scale and finding the region where the decay of the singular values σ_i is smaller in comparison with the previous values of i, resembling an elbow. The "relative energy" method consists on choosing r such that

$$\kappa = \frac{\sum_{i=1}^{r} \sigma_i^2}{\sum_{i=1}^{m-1} \sigma_i^2}, \tag{2.15}$$

$$\tau \geq 1 - \kappa, \tag{2.16}$$

where m is the rank of the factorized matrix, r is the number of vector basis for truncation, σ_i is the ith singular value in decrescent order and τ is a tolerance threshold, i.e. 10^{-6}. This method implies that more than $100(1-\tau)\%$ of the variance in the data is retained by the approximation. For the case where DMD is used on experimental (or numerical but noisy) data, more sophisticated solutions are presented in the literature [99, 100]. It is important mention that, although the relative energy is often presented as a form to primarily measure the approximation for DMD, this metric is not directly related to the quality of the DMD approximation [64]. The first r modes yield good approximation for the $\mathbf{Y}_1$ matrix but no information can be inferred from this metric about the accuracy of the DMD modes or eigenvalues.

Relative Frobenius norm:

Given that the result of a DMD approximation is a matrix that approaches the snapshots matrix, the Frobenius norm of the difference can be a good metric to evaluate how accurate the solution is. The formula is given by

$$\eta_F = \frac{||\mathbf{Y} - \mathbf{Y}^{DMD}||_F}{||\mathbf{Y}||_F}, \tag{2.17}$$

where $\mathbf{Y}$ is the snapshots matrix, $\mathbf{Y}^{DMD}$ is a matrix containing the DMD approximation solutions for the same instants sampled on $\mathbf{Y}$ and $||\cdot||_F$ represents the Frobenius norm. In this metric, we take the ratio between the Frobenius norm of the difference between ground truth and approximation and the Frobenius norm of the ground truth. The result, when multiplied by 100, reflects the percentage of how accurate the approximation is.

Relative $\mathcal{L}^2$ norm in time:

Another useful metric in engineering applications is the $\mathcal{L}^2$ norm between two vectors. In this thesis, we compute the $\mathcal{L}^2$ norm between each snapshot and its approximation. Differently from the relative Frobenius norm between two matrices (translated into a scalar), the relative $\mathcal{L}^2$ norm in time yields a temporal curve, given that the $\mathcal{L}^2$ norm is computed for each simulation instant. This allows further analysis of the dynamics of the problem. The expression for the relative $\mathcal{L}^2$ norm in time is

$$\eta_i = \frac{||\mathbf{y_i} - \mathbf{y}_i^{DMD}||_2}{||\mathbf{y}||_2}, \tag{2.18}$$

where $\mathbf{y}$ is a single snapshot for the solution at the ith instant, $\mathbf{y}_i^{DMD}$ is the DMD approximation solution at the ith instant and $||\cdot||_2$ represents the $\mathcal{L}^2$ norm. In this metric, we take the ratio between the $\mathcal{L}^2$ norm of the difference between ground truth and approximation for a given instant and the $\mathcal{L}^2$ norm of the ground truth. The result, when multiplied by 100, reflects the percentage of how accurate the approximation is on that instant. When performed for the all snapshots in the snapshots matrix, the result is a vector containing the relative $\mathcal{L}^2$ norm for all time instants sampled in the snapshots matrix.

Structural Similarity Index:

In this thesis, we also consider image reconstruction when using *in situ* visualization tools for generating PNG files and reconstructing the pixels snapshot matrix using DMD. Although using the previous metrics on this approximation is conceptually accurate, more dedicated metrics can be used to retrieve a straightforward and objective evaluation for quantitative assessment of the reconstructed images. In this study, we compute the Structural Similarity Index ($SSIM$)[101–103] between output and input images to evaluate the approximation quality properly. While the previous metrics compare pixel values individually, $SSIM$ acts on a window of pixels. The $SSIM$ of a given group of pixels ranges from -1 (completely uncorrelated match) to 1 (perfect match) and is a product of the computed contributions of luminance, contrast and structure. Considering two windows containing a group

of pixels w_1 and w_2 where w_1 is located on the image snapshot and w_2 is composed by the same pixels on the reconstructed image, the $SSIM$ is computed as,

$$SSIM = \frac{(2\mu_{w_1}\mu_{w_2} + c_1)(2\sigma_{w_1 w_2} + c_2)}{(\mu_{w_1}^2 + \mu_{w_2}^2 + c_1)(\sigma_{w_1}^2 + \sigma_{w_2}^2 + c_2)} \tag{2.19}$$

where the subscripts w_1 and w_2 are respective to two windows of a common size of pixels, μ, and σ^2 are, respectively, the average and variance of either w_1 and w_2, being $\sigma_{w_1 w_2}$ the covariance of w_1 and w_2. In this study, we have used the $SSIM$ implementation from scikit-image [104].

2.2.4 The remaining graph nodes

Data Ingestion

This is the first node of the PADMe library, used to read the snapshots from the simulation output files. Considering that all of our applications export data in HDF5 files, we use the `h5py`[2] library for extracting data from output files in Python.

Snapshots Matrix Creation

This step is responsible for assembling the snapshots matrix. That is, after data transformation, the snapshots are stacked vertically on a matrix. Depending on the DMD algorithm chosen - the standard DMD or the streaming DMD -, this step can be invoked or not. For the standard DMD, the snapshots matrix acts as a batch of data to be processed at once, while on the streaming DMD, the basis gets computed as the snapshots arrive.

Snapshots Embedding Definition

Although different strategies such as the extended DMD [53], the kernel DMD [55] and LANDO [43] can be used, we used the standard snapshots embedding procedure, where the state vectors are the snapshots themselves.

Matrix Factorization

Although there are more modern strategies for matrix factorization, in this thesis, we opt for SVD-like decompositions to construct the DMD basis. We focus on the rSVD for the standard DMD algorithm and on the iSVD for the streaming DMD. Both algorithms have been already described in the previous sections.

[2] https://www.h5py.org/

DMD Basis Construction

In this thesis, we use the basis truncation before computing the DMD basis. That is, after the SVD decomposition, we select the first r modes to compute the DMD basis. The procedure to compute the DMD modes is described in Algorithm 1 and is valid for both standard DMD and streaming DMD.

State Vector Approximation

Although DMD has several applications ranging from modal analysis to parameter extrapolation, in this thesis we focus on reconstructing the original dynamics using the DMD modes. For the SEIRD model, we test the DMD capabilities on short-time future estimates.

Chapter 3

Applications

In this chapter we present the numerical simulations that generate the data used to fuel the DMD algorithm. We present various complex numerical simulations that are of interest to both industry and society. First we start with two different fluid dynamics simulations: the gravity currents and the bubble rising problem. Gravity currents are complex flows generated by the different densities of fluids and are important in aircraft safety, atmospheric pollution, entomology and pest control, dense-gas technology and the oil industry [105] while the rising of a bubble within a quiescent liquid constitutes one of the most important and canonical problems of multiphase flows [106]. We model the gravity current problem using different strategies and libraries. We also describe the modeling process for a continuous SEIRD model, an epidemiological application that measures the spread for the COVID-19 disease on a given area. The following sections describe in more detail each of the numerical models and the simulation parameters.

3.1 Gravity currents

Gravity currents are formed when a heavier fluid moves horizontally into a lighter one, and are frequently observed in both natural and engineered systems [107, 108]. In many cases, the density difference between fluids is small, allowing for the use of the Boussinesq approximation, which is applicable in situations such as freshwater rivers entering saltwater oceans or atmospheric flows involving warm and cold air. However, in circumstances where the density differences are significant, such as industrial gas leaks, tunnel fires, powder snow avalanches, turbidity currents, and pyroclastic flows, the full variable density equations must be employed [109]. Developing simplified models for predicting these flows is desirable but requires establishing the validity of assumptions regarding the flow nature. High-resolution numerical simulations can provide valuable insights, including access to quantities that are difficult to measure experimentally, such as the spatially and temporally

resolved dissipation field. The most commonly used geometry for studying gravity currents is the lock-exchange configuration [110]. The lock-exchange layout consists on having a rectangular container divided into two compartments by a membrane. The left chamber is filled with a heavy fluid A, while the right one contains lighter fluid B. Upon release of membrane, a dense front moves rightwards along the lower boundary, while the light front propagates leftward along the upper boundary [107].

Gravity currents can be generated by differences in fluid density or differential particle loading. In this thesis, we consider two models where part of the fluid domain is filled with sediments while the remaining is pure fluid. The first model is a 2D lock-exchange with simple assumptions regarding boundary conditions and no settling while the second model is a 3D lock-exchange equipped with outflow boundary conditions [107] and sedimentation. In both cases, we consider that the Boussinesq hypothesis is valid, that is, the ratio between fluid densities is close to 1.0. The non-dimensional governing equations in this case are the conservation of mass, momentum and sediment transport

$$\nabla \cdot \mathbf{u} = 0, \tag{3.1}$$

$$\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} + \nabla p - \frac{1}{\sqrt{Gr}} \Delta \mathbf{u} - c\mathbf{e}^g = 0, \tag{3.2}$$

$$\frac{\partial c}{\partial t} + (\mathbf{u} + u_s \mathbf{e}^g) \cdot \nabla c - \frac{1}{Sc\sqrt{Gr}} \Delta c = 0. \tag{3.3}$$

where $\mathbf{u}$ is the fluid velocity, p is the pressure, c is the sediment concentration field and $\mathbf{e}^g$ is gravity direction $\mathbf{e}^g = (0, -1)$. Quantity c is normalized to map the interaction between fluids and equals to $c = 1.0$ for the part of the domain where the fluid is mixed with sediments and $c = 0.0$ for the remainder of the domain where fluid does not contain sediments. Also, dimensionless numbers such Gr is the Grashof number, that expresses the ratio between buoyancy and viscous effects, and Sc is the Schmidt number, that expresses the ratio between diffusion and viscous effects, should be given.

3.1.1 2D Lock-Exchange

For the first model we consider a 2D tank, that is, a rectangular domain with length $L = 18$, height $H = 2$. The boundary conditions for this case are $\mathbf{u} = \mathbf{g}$ on Γ_{in} and $(-p\mathbf{I} + \frac{1}{\sqrt{Gr}} \nabla \mathbf{u}) \cdot \mathbf{n} = \mathbf{h}$ on Γ_h in which $\mathbf{g}$ is a prescribed velocity value on Γ_g and $\mathbf{h}$ is the interaction traction force on Γ_h. For the transport equation, boundary conditions modeling the transport of particles are the Dirichlet boundary condition $c = c_{in}$ on Γ_c, which describes the quantity of sediment entering in the flow domain, no-flux boundary condition $(u_s \mathbf{g} c - \frac{1}{Sc\sqrt{Gr}} \nabla c) \cdot \mathbf{n} = \mathbf{0}$ on Γ_h with $\Gamma = \Gamma_{in} \cup \Gamma_h$ and

$\Gamma_{in} \cap \Gamma_h = 0$. This means that the conditions are no-slip for the momentum equation and no-flux for the transport equation. Also, no injection of fluids or sediments are considered, leading to $\mathbf{g} = 0$ and $c_{in} = 0$. As for initial conditions, we consider a divergence-free initial condition for the velocity field. For the transport equation, the initial conditions are such that the heavy fluid ($c = 1$) is represented as a column with dimensions $H \times S = 1 \times 2$ area units located at the left border of the tank and the light fluid ($c = 0$) fills the rest of the domain. Figure 3.1 illustrates the domain and the initial conditions. For this problem, we consider a turbulent flow such that $Gr = 5 \times 10^6$ and $Sc = 1.0$.

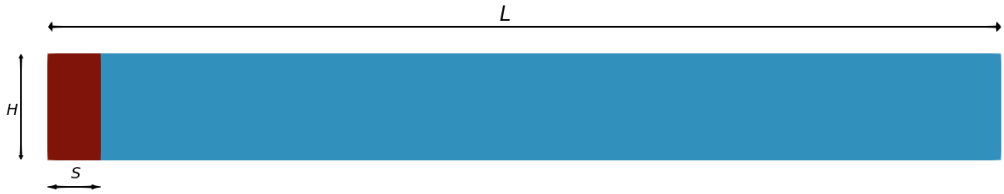


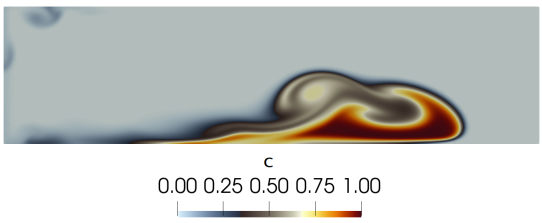
Figure 3.1: Scheme illustrating the initial conditions and boundary dimensions for the density-driven gravity flow example.

To solve the governing equations, we implement the RBVMS formulation [111–114] for Equations (3.1 - 3.3) using the `FEniCS` 2019.1 [82] framework. The backward-Euler time integration scheme is considered for its unconditional stability. The problem is solved using a monolithic fully coupled formulation for pressure, concentration and velocity and the nonlinear system returns the solution for $\mathbf{u}, p$ and c. The Newton solver is considered for the nonlinear system with relative tolerance of 10^{-7} and a relaxation parameter of 0.9. The linear solver is the GMRES solver with a relative tolerance of 10^{-5} equipped with the ILU(0) factorization preconditioner.

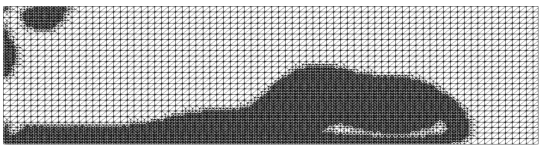
For this problem, we consider AMR/C strategies to refine the mesh on regions of interest, increasing the accuracy of the solution in this part of the domain, while coarsening the mesh on regions with less dynamics, reducing the dimensionality of the nonlinear system evaluated by the solver. AMR/C in this problem is advisable since the dynamics are predominant on the interface between the fluids. Most of the domain is not affected in the early stages of the simulation, and the use of fine meshes outside these regions may represent unnecessary computational effort. For comparison, we run the simulation considering a fixed mesh and an adaptive mesh. For the fixed mesh approach, the spatial mesh is discretized into 700×100 cells, where each cell is divided into two linear triangles. For the adaptive mesh simulation, we focus on employing an interface-tracking adaptive mesh error indicator that flags and refines the mesh where the two fluids interact for the adaptive mesh simulation. The error indicator for the mesh refinement is $|\nabla c|$ being larger than a given tolerance. For this purpose, a coarse mesh containing 175×25 cells is considered, the interfaces

are refined considering two levels of refinement, and the mesh refinement is invoked at every time step. More details on the numerical formulation of the problem and AMR/C strategies are provided on Appendix B. Time is discretized into 3000 time steps of size $\Delta t = 0.01$. For this problem, we consider that an output file is written for every computed solution. That is, the output writing interval $\Delta t_O = \Delta t$ leads to 3001 snapshots considering the initial conditions at $t = 0$. Details of the formulation of the problem can be found in [115]. For this case, the simulation code as well as the resulting data were created and provided by the author.

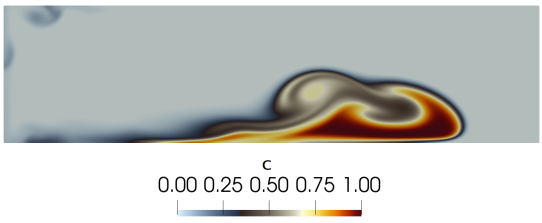
Figure 3.2 shows the solution and the mesh for $t = 10$ time units. Notice that the interface between two fluids contains more nodes and elements while other regions - with less dynamics - are less refined. Not only visually, we can assess the impact of applying AMR/C strategies on density-driven currents on Figure 3.3, where the number of nodes in the mesh during the simulation time for the adaptive mesh is compared with the fixed 701×101 nodes mesh. We observe that the number of nodes in the adaptive simulation is smaller than the fixed mesh for all the simulation time.



(a) Adaptive solution



(b) Adaptive mesh



(c) Fixed mesh solution

Figure 3.2: Results and mesh for the first 8 length units of the domain at $t = 10$ time units.

In lock-exchange configurations, it is desirable to track the position of the front of the current. Also, for this problem with the specified boundary conditions, the quantity of sediments in the domain is fixed during the simulation, leading to a mass

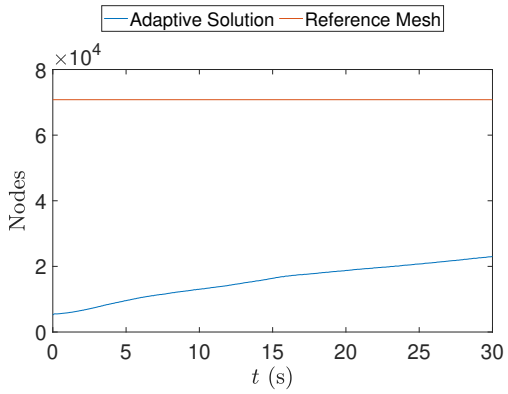


Figure 3.3: Number of mesh nodes in time for the adaptive solution and the proposed reference mesh for the density-driven gravity current example.

conservation of the sediments. Figure 3.4 shows the front position and total mass of the system in time for both fixed and adaptive meshes simulation. Front position is computed by tracking the maximum x coordinate for the concentration field where $c \geq 0.05$ while the sediments mass for the system is computed such that

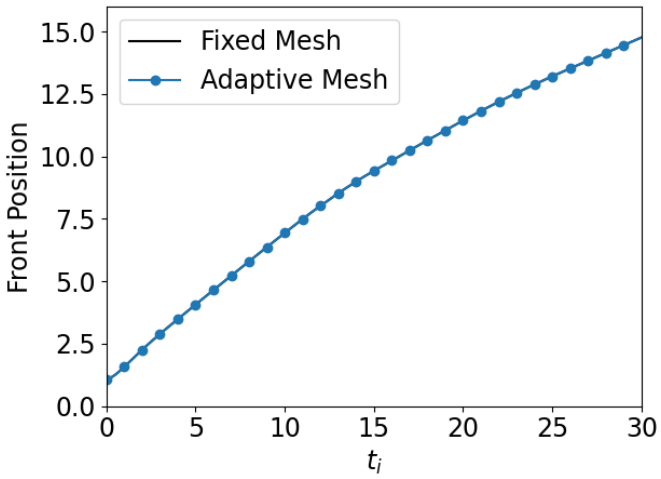
$$M_i = \int_\Omega c_i d\Omega, \tag{3.4}$$

for $i = 0, 1, \ldots, m$.

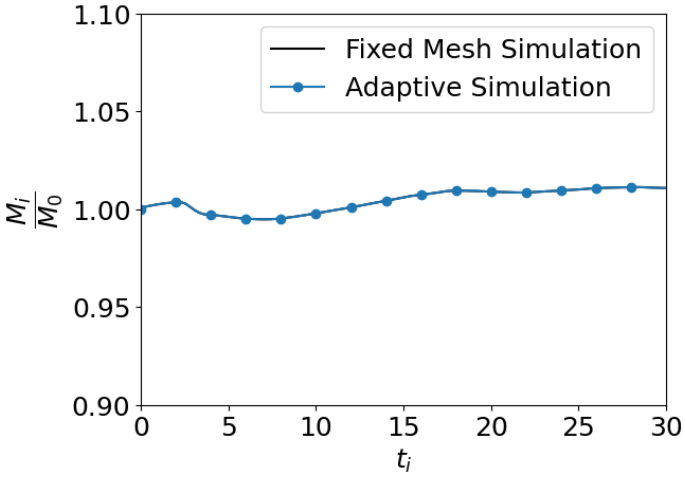
3.1.2 3D Lock-Exchange

We also consider a more complex simulation regarding the lock-exchange configuration in this thesis. In this case, the solution for Equations (3.1 - 3.3) is approximated on a 3D domain. Also, the boundary conditions are different for this model. We consider a convective flux condition $\frac{\partial c}{\partial t} - u_s \nabla c \cdot \mathbf{n} = 0$ on the bottom boundary Γ_b responsible for removing deposited sediments located on the bottom of the domain. Now the boundaries are Γ such that $\Gamma = \Gamma_{in} \cup \Gamma_h \cup \Gamma_b$ and $\Gamma_{in} \cap \Gamma_h \cap \Gamma_b = 0$. For the initial conditions, the same premises from the 2D lock-exchange simulation are preserved. Figure 3.5 shows the initial condition for the 3D domain. In this case, the domain is a tank of dimensions $H = 2$, $W = 0.1$ and $L = 20$. Again, no injection of fluids or sediments are considered. For the sediments filled fluid column, we consider $S = 0.75$.

For this problem, the RBVMS formulation is also considered for Equations (3.1 - 3.3) and the simulations are created using the `libMesh` [83] library. The backward-Euler time integration scheme is considered for its unconditional stability. The problem is solved using a staggered formulation for pressure, concentration and velocity. The nonlinear system is linearized using the Inexact Newton method with



(a) Front position



(b) Mass conservation

Figure 3.4: Front position and mass conservation for the fixed mesh and adaptive mesh simulations.

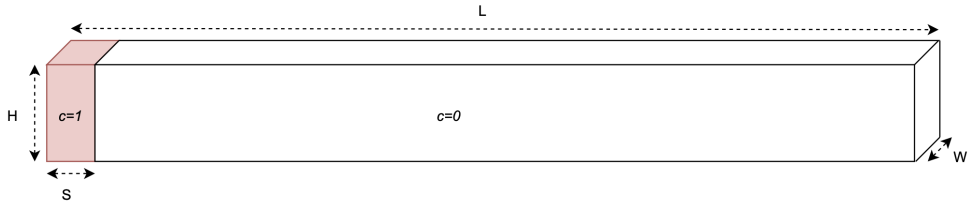


Figure 3.5: Scheme illustrating the initial conditions for the 3D density-driven gravity flow example.

a tolerance of 10^{-3}. After linearization, the equations are solved in 16 cores using Block-Jacobi + GMRES(35) with local ILU(0) preconditioning. GMRES tolerance is 10^{-6}. The time step size is 0.001, and the simulation runs until reaching the maximum time of 20 time units. Figure 3.6 shows the solution for the concentration

field for $t = 10$ time units. This simulation was conducted by the remainder authors of [68], which kindly provided the data for experimentation in this thesis.

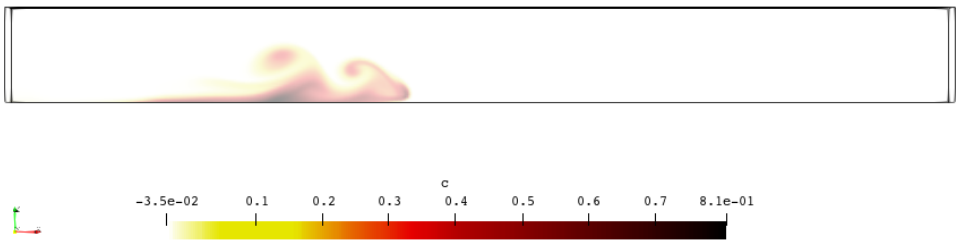


Figure 3.6: Results for the first 8 length units of the domain at $t = 10$ time units.

The output of the simulation is written in compressed HDF5 raw data files that are written with an output interval of $\Delta t_O = 0.01 = 10\Delta t$, leading to 2001 snapshots. That is, we generate one snapshot at every 10 computed solutions. Mesh data is stored in a single HDF5 file. Each output file stores the variables u, v, w, and p from Navier-Stokes equations, the variable c, and the deposition, d, that is, the integral over time of the deposition rate, $r = u_s c_b$, where c_b is the sediment concentration at the lower boundary. In this thesis, we aim on applying DMD to four datasets containing the identical solutions obtained for this simulation. The difference lies on the compression strategy for the solution data to the output files.

For this contribution, the numerical simulation generates four datasets detailed in Table 3.1. The first dataset (hereinafter described as Case A) contains the snapshots in their original configuration, that is, the nodal values for the finite element simulations without any data transformation for each time instant. The second case (Case B) contains the snapshots submitted to lossless compression, that is, when data can be compressed and decompressed without any loss of information. Cases C and D comprise the situations where data is compressed and a portion of the information is lost. The difference between cases C and D are mostly related to how aggressive is the compression (and, consequently, the loss of information). Case C preserves data on a precision of 10^{-6} for the floating point numbers while Case D preserves the information up to 10^{-3}. The proposed simulations share the same parameters such as mesh, time step and physical properties and were evaluated under the `libMesh` library with the coupling of the ZFP algorithm for data compression [68]. Details on the data compression technique used in this thesis can be seen in Appendix D.

Table 3.2 shows the storage requirements for a single snapshot of each physical

Table 3.1: Different cases for data compression

Cases	Compression Strategy	Parameters
A	No compression	-
B	Lossless compression	-
C	Lossy compression	10^{-6} accuracy
D	Lossy compression	10^{-3} accuracy

quantity solved on the nonlinear equation as well as the compression rate (CR) for each case. The compression rate is computed as the ratio between the storage required for the raw data and the compressed data. The table shows that lossless compression yields lower CR when compared to lossy schemes. We can define the desired accuracy threshold for floating point numbers for lossy compression, and higher ratios are observed for larger thresholds. We notice that for a threshold of 10^{-6}, the storage required for sediment concentration (c) is 14 times smaller than the original data size, while for a threshold of 10^{-3}, the compressed output is up to 30 times smaller than the original data. Moreover, the highest compression ratios seen in both lossless and lossy schemes are on variables where the number of non-zeros is low. We also observe that the CR might reach two orders of magnitude on sparse data such as c and d. That is, in other words, we can store up to 116 compressed d snapshots in exchange of one single raw d snapshot. In this study, we analyze if this phenomenon affects the performance of DMD approximations for the same data with different levels of compression.

Another important metric regarding mesh compression is the Peak Signal-to-Noise Ratio (PSNR). PSNR is one of the the most critical indicators used to assess the distortion of reconstructed data versus original data in lossy compression [116, 117]. Figure 3.7 shows the PSNR for the original scalar sediment concentration data, c, in lossy compression. The final PSNR value is calculated as the mean PSNR values obtained in each np parallel partition through the following formula:

$$\text{PSNR} = \frac{1}{np}\sum_{p=1}^{np} 20log10(\max c - \min c)_p - 10log10(MSE_p) \tag{3.5}$$

where $MSE = \frac{1}{m}\sum_{i=1}^{m}(c_i - \bar{c}_i)^2$, c is the original concentration data and $\bar{\phi}$ is the reconstructed data. The higher PSNR values indicate better reconstruction data quality. In image processing, PSNR values in lossy images and video compression are in the range of 30-50 dB, while for 16-bit data, PSNR typically varies between 60 and 80 dB [118]. Three-dimensional scientific data in [119] also presents PSNR values around 60 dB. As expected, the comparison presented in Figure 3.7 indicates better PSNR values for lossless compression, PSNR intermediate values for fixed-accuracy mode with tolerance 10^{-6} and smaller values for fixed-accuracy with tolerance 10^{-3}.

Table 3.2: Storage requirements (in MBytes) for each quantity of interest and the compression rates for the lock-exchange simulation

	u		v		w		p		c		d	
	Storage	CR	Storage	CR	Storage	CR	Storage	CR	Storage	CR	Storage	CR
Raw Data	1384.9		2737.3		2737.3		2737.3		2737.3		2737.3	
ZFP Lossless	1275.5	1.1	2480.8	1.1	2534.7	1.1	2348.9	1.2	2631.0	1.04	194.3	14.1
ZFP 10E-06	295.4	4.7	142.9	19.2	556.7	4.9	654.6	4.2	189.9	14.41	35.0	78.2
ZFP 10E-03	120.9	11.5	12.3	221.7	222.8	12.3	323.3	8.5	90.7	30.18	23.4	116.8

However, note that for a fixed accuracy of 10^{-3}, several outputs are within the limit of what is considered acceptable according to the PSNR metric.

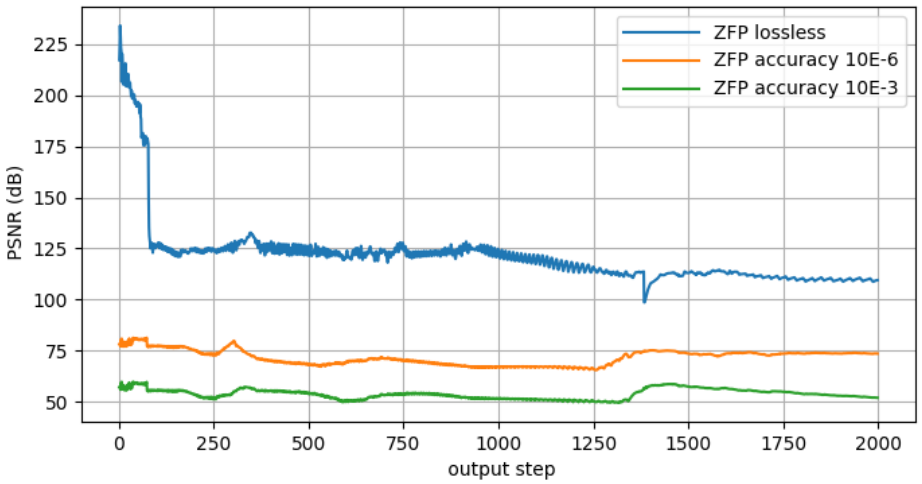


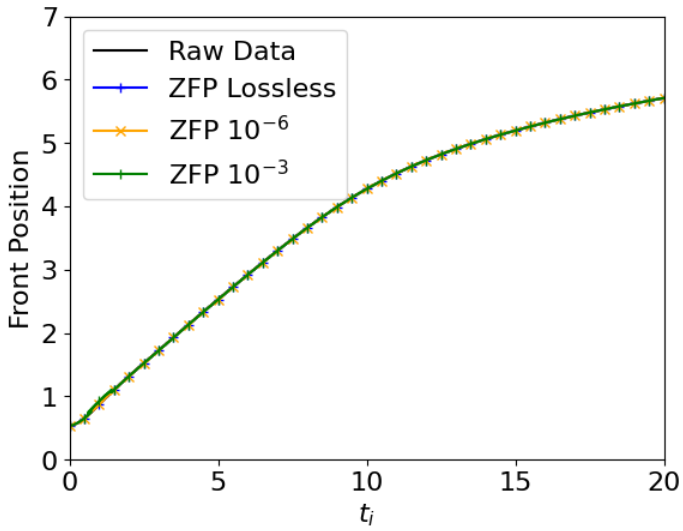
Figure 3.7: PSNR for lossless and lossy compression for all sediment concentration output files.

For this problem, we also consider generating image files during simulation runtime for DMD approximation. The motivation for using image files instead of the usual HDF5 files is the significant data reduction associated with this technique. For instance, considering a midplane image for the lock-exchange simulation, the PNG file for the concentration field has 499×51 pixels, yielding 153 MBytes needed to store all image files generated during the simulation. Compared with the values seen in Table 3.2 for storing the HDF5 files, the PNG files represent a more efficient approach. This strategy allows fast data transfer for quick updates on the streaming DMD algorithm. Figure 3.8 shows a PNG image generated during simulation runtime via Paraview Catalyst for the instant $t = 12$. For this problem, both datasets are being applied to DMD: the compressed HDF5 files that will eventually become a batch of snapshots for the standard DMD algorithm and the PNG image files that will be added to the dataset one at a time for the streaming DMD algorithm.

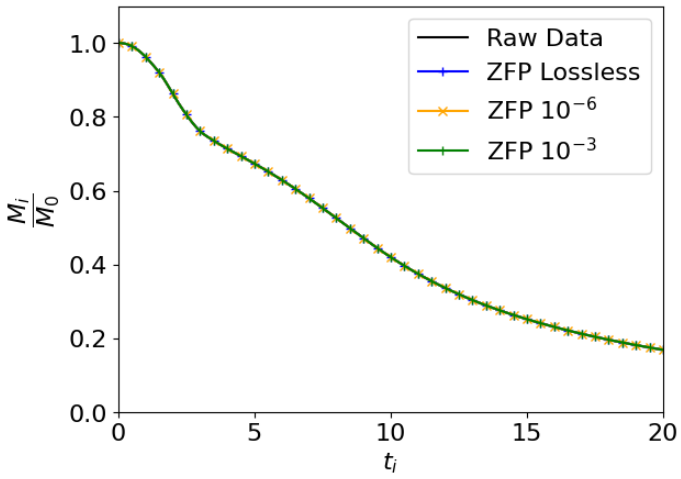
Figure 3.8: *In situ* visualization of the sediment concentration profile at the midplane image file at $t = 12$.

Similar to the 2D case, we analyze the front position and the total sediment mass in time of the problem. In this case, there is no mass conservation for the sediments given that during the evolution of the current the deposited sediments

are being removed from the domain through the bottom boundary (given the considered boundary conditions). Figure 3.9 shows the front position and total mass of sediments in time for this problem. We consider Paraview to extract the maximum x coordinate of the field containing $c \geq 0.05$ to compute the front position and Equation 3.4 to compute the sediments mass for each instant t_i.



(a) Front position.



(b) Mass of sediments over time.

Figure 3.9: Front position and mass of sediments for the 3D lock-exchange simulation.

3.2 Bubble rising

Another interesting multi-fluid system of interest in science and the industry is the bubble rising problem. A fundamental understanding of the bubble rising

physics is crucial in several practical applications, ranging from the rise of steam bubble in boiler tubes to gas bubbles in oil wells [120]. In this problem, one fluid - fluid A - is confined on a specific domain and one sphere (or circle) containing the other fluid - fluid B - is initialized on the bottom part of the domain. Fluid B should be a less dense fluid than fluid A, so that the bubble rises towards the top of the domain. In this case, the difference between densities should generate the driven force for fluid motion. To model this problem we couple the Navier-Stokes equations with an interface capturing method called convected level-set [121–123]. The convected level-set is a method used to represent the interface between two-phases and, by a convection equation, to move the interface as the flow evolves. A force that has an important role in bubble problems is the surface tension $\mathbf{F_{st}}$, which is applied using the Continuum Surface Force Model (CSF) [124]. We write the governing equations in their dimensional form as,

$$\begin{aligned} \nabla \cdot \mathbf{u} &= 0, \\ \rho\frac{\partial \mathbf{u}}{\partial t} + \rho\mathbf{u}\cdot\nabla\mathbf{u} + \nabla p - \mu\nabla^2\mathbf{u} - \rho\mathbf{g} - \mathbf{F_{st}} &= 0, \\ \frac{\partial \alpha}{\partial t} + (\mathbf{u} + \lambda\mathbf{U})\cdot\nabla\alpha - \lambda\,\mathrm{sgn}(\alpha)S &= 0, \end{aligned} \tag{3.6}$$

where ρ is the density, μ is the dynamic viscosity, $\mathbf{g}$ is the acceleration of gravity vector, α is the level-set function and λ is a penalty constant. The penalty constant λ sets the contribution of the re-initialization equation in the convection equation and should be chosen such that a small value for λ may not be enough to correct the iso-surfaces and to recover the signed distance properties adequately, while a large one may change the interface shape [123]. The quantity U is such $U = \mathrm{sgn}(\alpha)\dfrac{\nabla\alpha}{||\nabla\alpha||}$ where

$$\mathrm{sgn}(\alpha) = \begin{cases} 1 & \text{if } \alpha > 0 \\ 0 & \text{if } \alpha = 0 \\ -1 & \text{if } \alpha < 0 \end{cases} \tag{3.7}$$

and S a function related to the level-set signed distance function. In this case, we consider the function proposed in [123] which is

$$S = \left|\left|\nabla\left(\frac{1}{1+\exp(-\alpha/E)}\right)\right|\right| = \frac{1}{4E} - \frac{\alpha^2}{E}, \tag{3.8}$$

where E defines the thickness where the transition between phases is defined. For practical purposes, E is defined as a function of the interface element size h_e such that $E = 2h_e$.

To approximate the governing equations, we use a finite element RBVMS for-

mulation [111–114]. For the temporal integration, we apply the Backward-Euler method to the Navier-Stokes equations, while for the convected level-set, we use the BDF2 method. The whole model is implemented in `libMesh`. AMR/C is considered for this problem and the strategy considered is the built-in approach of `libMesh`. Similar to the 2D lock-exchange simulation, the use of AMR/C strategies requires snapshot pre-processing to enable the construction of the snapshots matrix. For this purpose, we project the snapshots during simulation runtime into a structured mesh containing equal size elements that match the smallest element obtained in the adaptive mesh. For comparison and visualization reasons, we output both adaptive and projected fields into the HDF5 files. Further details of the AMR/C strategy used in this problem can be seen on Appendix B. The initial configuration is illustrated on Figure 3.10 and it consists of a spherical gas bubble of radius $R = 0.25$ m centered at $\mathbf{x} = (0.5, 0.5, 0.5)$ m in a $[1 \times 1 \times 2]$ m domain filled with liquid.

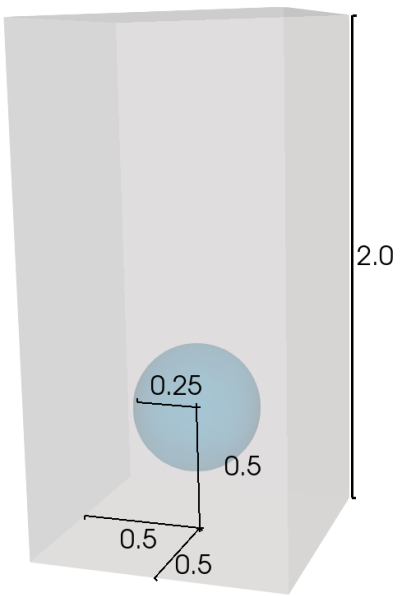


Figure 3.10: Initial configuration and dimensions for the bubble rising problem.

The no-slip boundary condition is applied to all boundaries. Regarding the simulation parameters, the problem is solved for 241 time steps (including the initial condition) for a time step size of $\Delta t = 0.0125$ s, yielding 3 s of total time simulated. The output writing interval is $\Delta t_O = \Delta t$, leading to 241 snapshots. The liquid density and viscosity are $\rho = 1000$ kg/m^3 and $\mu = 10$ kg/(m·s) while the gas density and viscosity are $\rho = 100$ kg/m^3. The surface tension is 24.5 N/m and gravity is $\mathbf{g} = (0, -9.81)$ m/s^2. For the solver, a staggered approach is considered and the nonlinear system is solved for $\mathbf{u}$, p and α. The PETSc library solves the linear system of equations coming from the linearization of the Navier-Stokes equations invoked

by `libMesh`, applying the GMRES with Block-Jacobi preconditioner together with ILU(0) within each block. The problem is parallelized into 8 cores. Results are seen on Figure 3.11. This simulation was conducted by the authors of [123], which kindly provided the data for usage in this thesis.

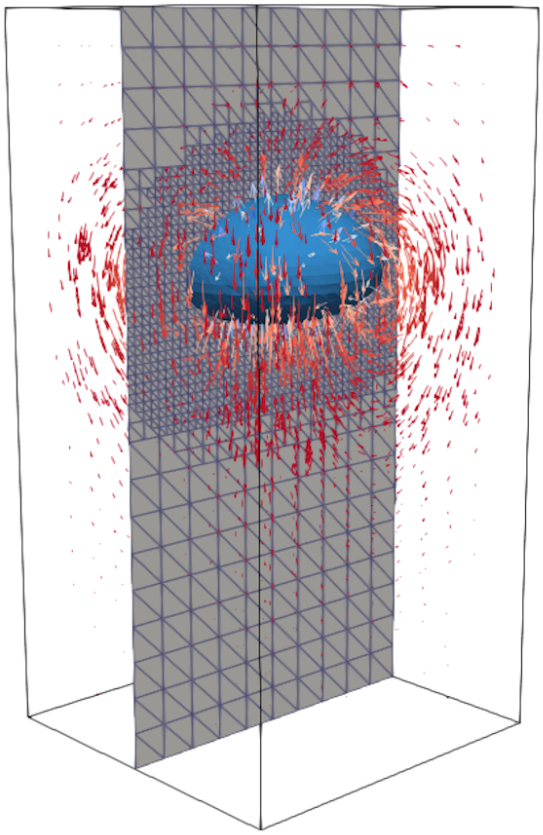


Figure 3.11: Adaptive mesh and solution containing bubble shape and velocity field at $t = 2.5$s.

For this problem we also considered data compression. The same compression strategies used for the 3D lock-exchange problem (seen on Table 3.1) are applied for the bubble rising problem. We also evaluate the output files in terms of size and compression rates in Table 3.3. For this specific problem, we compress both the level set field on the adapted mesh and the level set field on the projected mesh. The projected mesh contains more nodes and elements, and given that it is the same mesh for all snapshots, it enables the use of DMD in this case. All the other fields are stored in their original, adapted meshes. Compared with Table 3.2, we notice lower compression rates in the bubble case. There are reasons for these lower rates. First, the use of AMR/C strategies leads to nodal numbering issues that directly affect data compression. New nodes are added or removed when the mesh is refined or coarsened. Then, the values associated with the nodes added or removed may not favor compression, in the sense that the array holding the nodal values may not have smooth values even in the region where the solution is smooth due to

the node numbering. The second reason is the sharpness of the gradients for the solution. Compared with the lock-exchange simulation's concentration field, the thin interface between the two immiscible fluids leads to less efficient data compression. Considering that the level-set (α) field is ideal for tracking the dynamics on DMD, we observe, for the lossy compression with the threshold of 10^{-6}, a compression rate of 2.8 times while for the threshold of 10^{-3}, the compression rate approaches 5 times. Figure 3.12 shows the PSNR for all level-set output files. Although the differences between lossless and lossy compression rates are smaller when compared to the gravity current problem, the PSNR values of all compression methods are within the range of values between 60 and 90 dB. As noticed in the previous problem, PSNR intermediate values for fixed-accuracy mode with tolerance 10^{-3} are within the limit of what is considered acceptable.

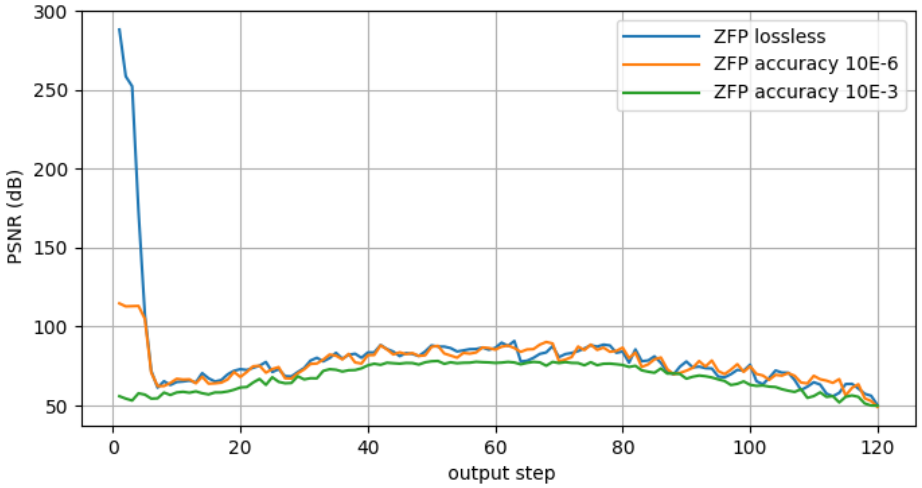


Figure 3.12: PSNR for lossless and lossy compression for the level-set function output files.

Similar to the 3D lock-exchange problem, we also consider generating image files during simulation runtime for DMD approximation for the bubble rising problem. Here we consider the α field on the vertical mid plane. The PNG files contain 148×296 pixels, leading to a total of 3 MBytes required for storage compared with the larger storage needed for storing the HDF5 files as seen in Table 3.3. Figure 3.13 shows the PNG files for the bubble rising problem at different stages of the simulation.

3.3 SEIRD model for COVID-19

The study of infectious disease spreading is a well-established field and has given rise to the area of science called *mathematical epidemiology* [125]. Mathematical

Table 3.3: Storage requirements (in MBytes) for each quantity of interest and the compression rates for the bubble simulation

	u		v		w		p		α		α (projected mesh)	
	Storage	CR	Storage	CR	Storage	CR	Storage	CR	Storage	CR	Storage	CR
Raw Data	44.9		44.9		44.9		44.9		44.9		133.5	
ZFP Lossless	41.6	1.1	41.3	1.1	41.5	1.1	41.0	1.1	40.6	1.1	111.5	1.20
ZFP 10E-06	14.4	3.1	15.3	2.9	14.4	3.1	22.9	2.0	16.0	2.8	38.9	3.43
ZFP 10E-03	7.5	6.0	8.5	5.3	7.5	6.0	15.9	2.8	9.0	5.0	22.4	5.97

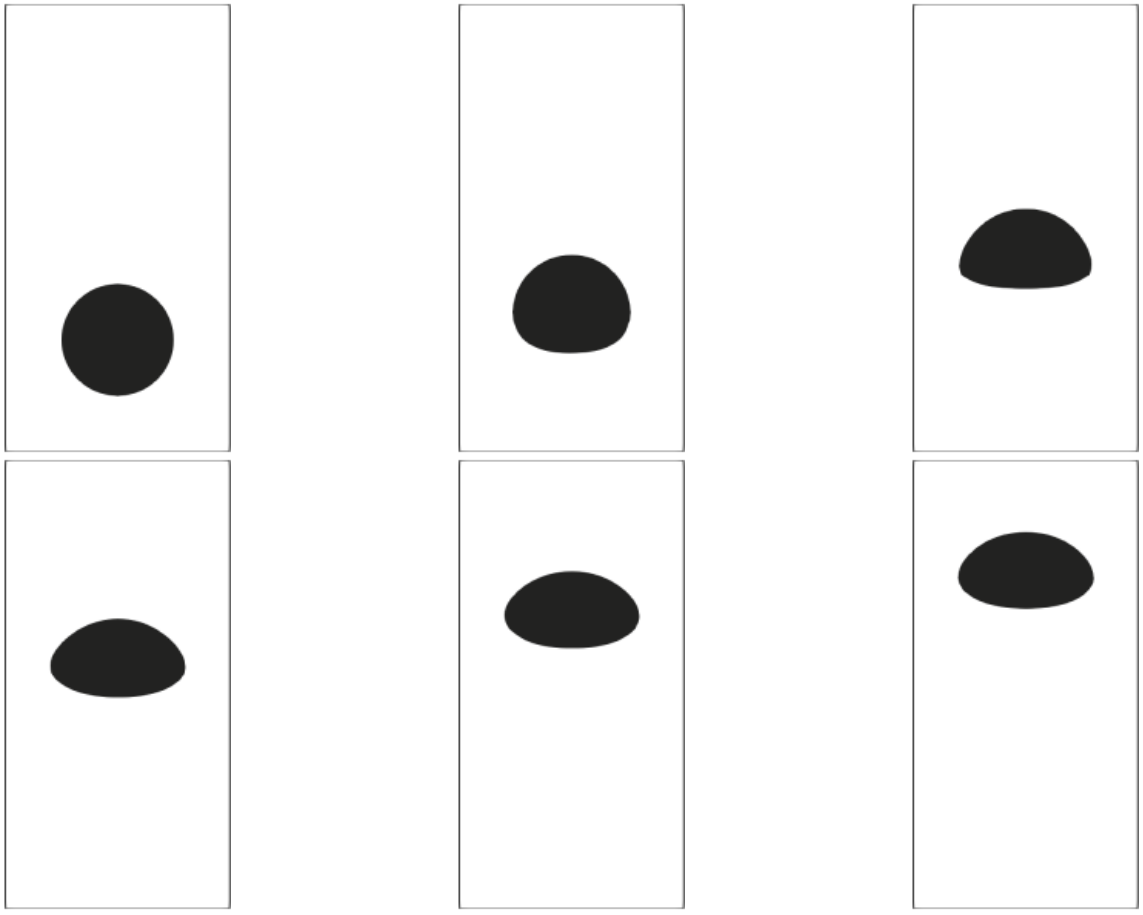


Figure 3.13: *In situ* visualization image file of the bubble shape. From top left to bottom right, $t = \{0.000, 0.625, 1.250, 1.875, 2.500, 3.000\}$s.

epidemiology proposes models that help the understanding of epidemics and to outline policies to control infectious diseases. In Brazil, studies of this type have been carried out for years for diseases such as Dengue [126] and Zika [127], and, in a global context, diseases such as HIV [128], SARS [129], Malaria [130], Ebola [131], among others. The COVID-19 pandemic brought the need for more research in this area. Several models for this pandemic outbreak have been presented in [132–137].

Disease transmission may be modeled as *compartmental models*, in which the population under study is divided into compartments and has assumptions about the nature and time rate of transfer from one compartment to another [138].

In this thesis, we focus on applying DMD on the data generated on the study of [139] and the model created in [133]. In this study, the authors used a partial differential equation (PDE) model to capture the continuous spatio-temporal dynamics of COVID-19. That is, by using PDE models, the authors are able to incorporate spatial information more naturally and allow for capture the dynamics across several scales of interest. The compartmental strategy is the SEIRD model, where the population on a given region can be divided into:

- Susceptible: individuals that can contract the SARS-COV-2 virus;
- Exposed: individuals that contracted the SARS-COV-2 virus;
- Infected: individuals that were exposed to the virus and presented symptoms;
- Recovered: individuals that recovered from the infection or never presented any symptoms after being exposed to the virus;
- Deceased: individuals that had complications after the COVID-19 infection and deceased;

This division and the possible transition between compartments can be seen on Figure 3.14. Some hypothesis for the model follow:

- No mortality beyond the COVID-19 disease is considered;
- No births are considered (the total susceptible compartment is monotonically decreasing);
- Individuals on the exposed compartment may reveal symptoms and switch to the infected compartment (symptomatic cases) or never develop symptoms and move directly to the recovered compartment (asymptomatic cases);
- Individuals on the exposed and infected compartments are able to spread the disease;
- Exposure and the development of symptoms are separated by a latent period of time;
- New cases of exposed people might appear randomly in the system (representing, for instance, people who return from a travel carrying the SARS-COV-2 virus);
- The probability of contagion is directly related to the population size (Allee effect [133]);
- Population dynamics is proportional to population size; i.e., more movement occurs within heavily populated regions;
- No diffusion parameter is considered for the deceased compartment;

Each one of these compartments represent a physical quantity (respectively, s, e, i, r and d) that will be inserted on a system of coupled reaction-diffusion equations. The resulting PDE system is:

$$\frac{\partial s}{\partial t} + \beta_i \left(1 - \frac{A}{n_{pop}}\right) si + \beta_e \left(1 - \frac{A}{n_{pop}}\right) se + f - \nabla \cdot (n_{pop} \nu_s \nabla s) = 0, \tag{3.9}$$

$$\frac{\partial e}{\partial t} - \beta_i \left(1 - \frac{A}{n_{pop}}\right) si - \beta_e \left(1 - \frac{A}{n_{pop}}\right) se + (\alpha_r + \gamma_e) e - f - \nabla \cdot (n_{pop} \nu_e \nabla e) = 0, \tag{3.10}$$

$$\frac{\partial i}{\partial t} - \alpha_r e + (\gamma_i + \delta) i - \nabla \cdot (n_{pop} \nu_i \nabla i) = 0, \tag{3.11}$$

$$\frac{\partial r}{\partial t} - \gamma_e e - \gamma_i i - \nabla \cdot (n_{pop} \nu_r \nabla r) = 0, \tag{3.12}$$

$$\frac{\partial d}{\partial t} - \delta i = 0, \tag{3.13}$$

where $n_{pop} = n_{pop}(\mathbf{x})$ is the total population of the domain - or the sum of all compartments. Quantity A characterizes the Allee effect (persons) and f is a source field that depends on space and time (persons). The quantities β_i and β_e is the transmission rate (persons^{-1}days^{-1}) between compartments, α_r represents the latent rate (days^{-1}), γ_e and γ_i describe the recovery rate and δ is the death rate (days^{-1}). Parameters β and γ are follow by subscripts i, that indicates the transmission rate between symptomatic and susceptible, and e, that points the transmission rate between asymptomatic and susceptible. Parameters ν_s, ν_e, ν_i and ν_r map the diffusion parameters for each compartment (km^{2}persons^{-1}days^{-1}). The parameters values are chosen to simulate the COVID-19 scenario as accurate as possible. For instance, parameters β vary depending on the lockdown measures imposed on February and March of 2020. For the exact parameters values used in the simulation, we refer to the original paper [140]. Another quantity that is usually of interest but is obtained by post-processing the solution from the SEIRD model is the cumulative infected compartment $\mathbf{c}(\mathbf{x}, t_k) = \sum_{j=0}^{k} i(\mathbf{x}, t_j)$. Due to abuse of notation, the cumulative infected compartment c should not be confused with the sediment concentration c described in Sections 4.1.1 and 4.1.3. The same thing is applied to the recovered compartment r, that should not be confused with scalar r that indicates the number of basis vectors used for truncation in the DMD algorithm.

The numerical simulations were conducted using the `libMesh` library. Equations (3.9 - 3.13) are spatially discretized in space using a Galerkin finite element variational formulation. The resulting systems of equations are stiff, leading us to employ implicit methods for time integration. We apply the Backward Differentiation Formula (BDF2), which offers second-order accuracy while remaining unconditionally stable. The physical domain is the Lombardy region in Italy, one of the western

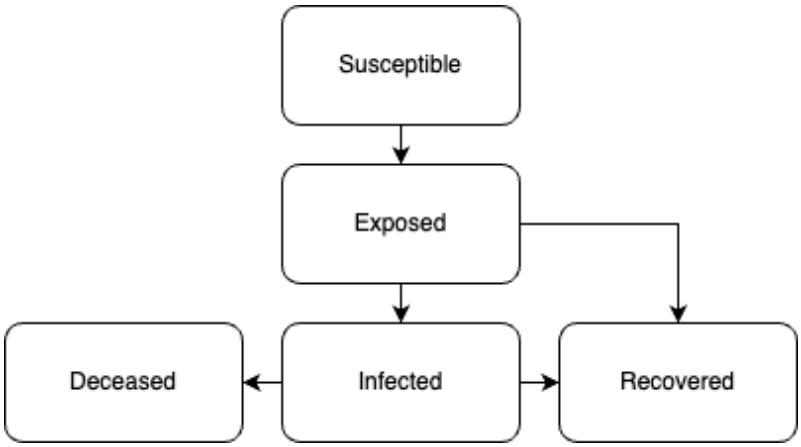


Figure 3.14: Flowchart illustrating the possible transitions between compartments on the considered SEIRD model

epicenters of the COVID-19 outbreak, that was kindly provided by the authors of [132]. An unstructured mesh is considered due to the complex geometry imposed by the domain. The mesh is generated using `Gmsh` and is uniformly refined as the simulation starts. In this example, again, an adaptive mesh refinement and coarsening (AMR/C) strategy is used to refine the spatial domain in regions of interest while coarsening the areas where dynamics are less important. After refining the whole mesh in one level, the mesh presents a minimum spatial resolution of approximately 1 kilometer. This procedure allows the solver to coarsen the regions where no significant dynamics are observed while preserving the scales of the regions of interest. For the sake of comparison, a fixed mesh presenting 13158 nodes and 25340 equal size elements is considered as well. Both adaptive mesh and fixed meshes have the same minimum element size, meaning that the adaptive mesh smallest elements are of the size of the fixed mesh element size. More details on the implementation and AMR/C strategies can be seen on Appendix B. The simulation considers a time step size of $\Delta t = 0.25$ days for the numerical integration with an output writing interval $\Delta t_O = \Delta t = 0.25$ days for the observations - that is, all computed solutions are outputted by the model. The total number of days simulated is 60, leading to a total of 241 snapshots considering the initial conditions. The file format for the simulation output is HDF5 [80]. Figure 3.15 shows the physical domain of the problem and the initial conditions for the susceptible and the exposed compartments. The remaining compartments are initialized with zero values and their dynamics are consequently dictated by the first two compartments. Figure 3.16 show the susceptibles compartment at $t = 46$ days for the simulation with a fixed unstructured mesh and the adaptive mesh, yielding similar solutions.

In this problem we consider that the population states transition exclusively through the compartments and some hypothesis for the model is that no nativity is

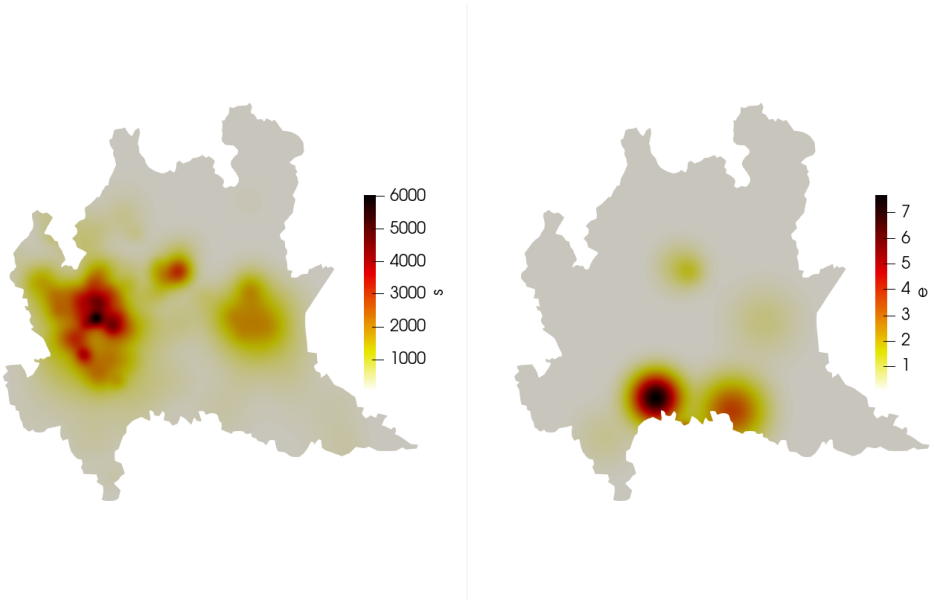
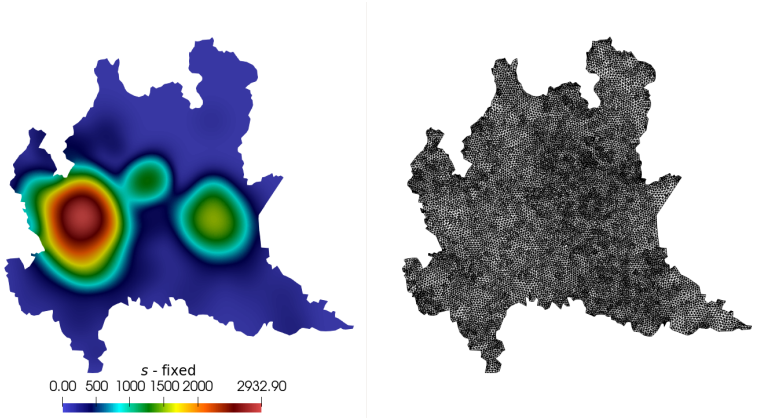


Figure 3.15: Initial conditions for the s and e compartments of the SEIRD model numerical simulations. Compartments i, r and d are initialized with zeros. Spatial domain is the Lombardy region, in Italy.

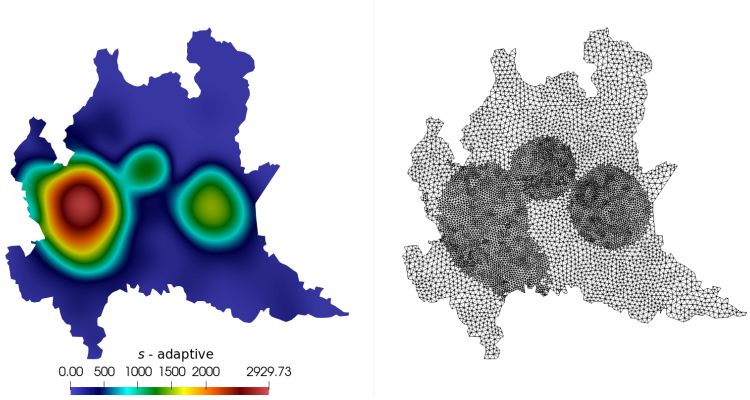
considered in this period and no other form of demise is considered except for the population in the d compartment. This leads to a conservation of population among the compartments. That is

$$\frac{dn_{pop}}{dt} = \frac{d}{dt}\int_{\Omega}(s+e+i+r+d)d\Omega = 0. \tag{3.14}$$

Figure 3.17 shows the normalized population number during the simulation for the fixed and the adaptive meshes. The population conservation calculation is done by computing $\int_{\Omega}(s+e+i+r+d)d\Omega$ for all time instants and dividing the values by the quantity computed for instant $t = 0$.



(a) Fixed mesh solution



(b) Adaptive solution

Figure 3.16: Solution for the susceptible compartment at $t = 46$ days obtained using an adaptive mesh and its respective projection onto a fixed reference mesh.

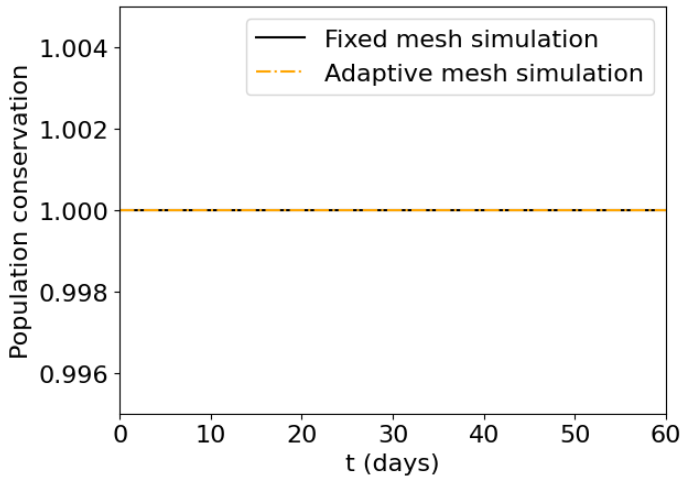


Figure 3.17: Population conservation for both adaptive and fixed mesh results.

Chapter 4

Numerical Results

In this chapter we apply DMD - through the lenses of the workflow presented on Figure 2.4 - to the numerical applications presented on Chapter 3. For each case, data is generated on a specific format and DMD is considered for a particular reason. In this chapter, we start by discussing the usage of DMD to reconstruct the dynamics of a 2D gravity current. DMD is used to analyze the dynamic modes and the quality of the reconstructions. We also analyze the application of AMR/C strategies on the 2D lock-exchange application. In this case, the `Data Transformation` phase requires projecting the solution into a tailored reference mesh such that all snapshots now belong to the same vector space, enabling the use of DMD to compute the approximation. We then extend the problem to the case where the simulation is now performed in three spatial dimensions and the data generated is compressed and parallel. For this purpose, the `Data Transformation` phase is responsible to decompress and assemble all snapshots, enabling their use in the DMD code. We also consider the case where data is generated during simulation runtime, that is, snapshots are not obtained by the usual means of the simulation output files but rather through the usage of an *in situ* visualization tool that generates image files of the state vector on-the-fly. For both cases, we evaluate the reconstructions of the solutions based on the data received. Next, we apply DMD to another fluid dynamics problem, the 3D bubble rising, also using compressed and parallel snapshots and also invoking the *in situ* visualization tool. In this case, we also need to deal with snapshots with varying dimensionalities given the AMR/C strategy considered to adjust the mesh topology as the bubble rises. That said, the `Data Transformation` phase needs to deal with all possible snapshots pre-processing techniques presented in this thesis for this example. For this problem, DMD is again invoked to reconstruct the dynamics of the problem depending on the data obtained. Finally, we extend the usage of DMD to a epidemiological model used to model the spread of COVID-19 in the Lombardy region, in Italy. In this case, data is generated without data compression and using serial simulations, even though the simulation is

equipped with an AMR/C strategy and requires snapshot pre-processing. After the `Data Transformation` phase, DMD is then considered for short-time future predictions to infer the spread of COVID-19 for 14 days for all compartments.

4.1 Gravity currents

In this section, we apply DMD to snapshots generated from the gravity current simulations described on Section 3.1. This section is divided into two sections: DMD applied to data obtained from a 2D lock-exchange problem and the DMD approximation of a 3D lock-exchange simulation.

4.1.1 Fixed mesh 2D lock-exchange

In this section we explore the results obtained on the DMD approximation from the 2D lock-exchange simulation data. This model require no additional data transformation and the pipeline for DMD approximation is the simpler possible. This example is ideal for exploring the DMD model and compare methodologies. For this example, we only consider the snapshots of c for our calculation. From the DMD workflow on Figure 2.4, we approach the problem using the tools described as:

- Data Generation: Simulation is processed using `FEniCS` 2019.1.0 and data is generated in the `FEniCS` HDF5 format.
- Data Transformation: None required.
- Data Ingestion: Python `h5py` library.
- Snapshots Matrix Creation: c snapshots.
- Snapshot Embedding Definition: state vectors.
- Matrix Factorization: SVD and rSVD algorithms.
- DMD Basis Construction: $r = \{50, 100, 150, 200, 250\}$.
- State Vector Approximation: Signal reconstruction.
- Metrics Evaluation: Relative energy, relative Frobenius error, relative error in time.

First, data is ingested and the snapshots matrix is created. Then, matrix $\mathbf{Y}_1$ is factorized into singular values and vectors. To illustrate the singular values decay and to compute relative energy κ (described in Equation 2.15), we use NumPy's

standard SVD algorithm [141]. The function invokes the high-performance LAPACK routine `_gesdd` [142], which computes all singular values from $\mathbf{Y}_1$. Figure 4.1 shows the monotonically decreasing singular values from $\mathbf{Y}_1$. We observe that as r increases, κ approaches 1.0, indicating that the approximation of $\tilde{\mathbf{Y}}_\mathbf{1}$ becomes closer to the original $\mathbf{Y}_1$ matrix.

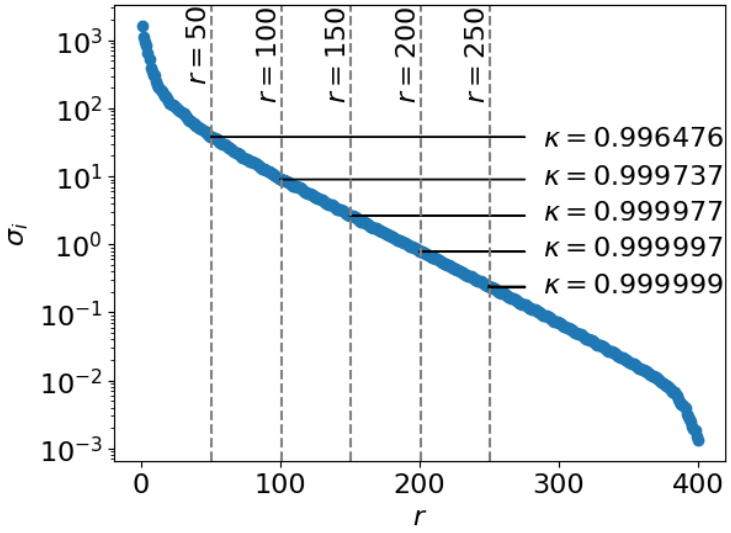


Figure 4.1: Singular values for the $\mathbf{Y}_1$ matrix containing the 2D lock-exchange snapshots and the values for κ for each $r = \{50, 100, 150, 200, 250\}$. Singular values were limited to a maximum of $r = 400$ for proper visualization.

By factorizing the $\mathbf{Y}_1$ matrix for this problem, information about the spatial modes can be extracted and analyzed. For instance, Figure 4.2 shows some important features from the lock-exchange dynamics. The first figure is a snapshot at $t = 25$ time units, showing the gravity current nose already developed and advancing through the domain. The second figure is the temporal mean of mesh node values during the simulation. The next four figures represent the most energetic spatial modes, where the dominant structures of the flow can be observed. We can notice at the left part of the domain the influence of the initial conditions on both mean field and spatial modes figures.

We now illustrate the comparison between the use of a standard SVD routine and a rSVD routine on the DMD algorithm in Figure 4.3. We compare NumPy's standard SVD algorithm with our implementation of the rSVD procedure using NumPy for the algebraic computations (such as QR factorization, for instance). Results are presented in terms of accuracy and efficiency for five different values of r. For the accuracy, we compare the Frobenius norm relative error between the snapshots matrix and the obtained reconstruction of the matrix $\tilde{\mathbf{Y}}_\mathbf{1} = \mathbf{U}_r\Sigma_r\mathbf{V}_r^t$, that is, for this comparison, η_F is computed according to Equation 2.17 but the error is computed for $\mathbf{Y}_1$ and $\tilde{\mathbf{Y}}_1$ instead. We observe that, for both algorithms, the

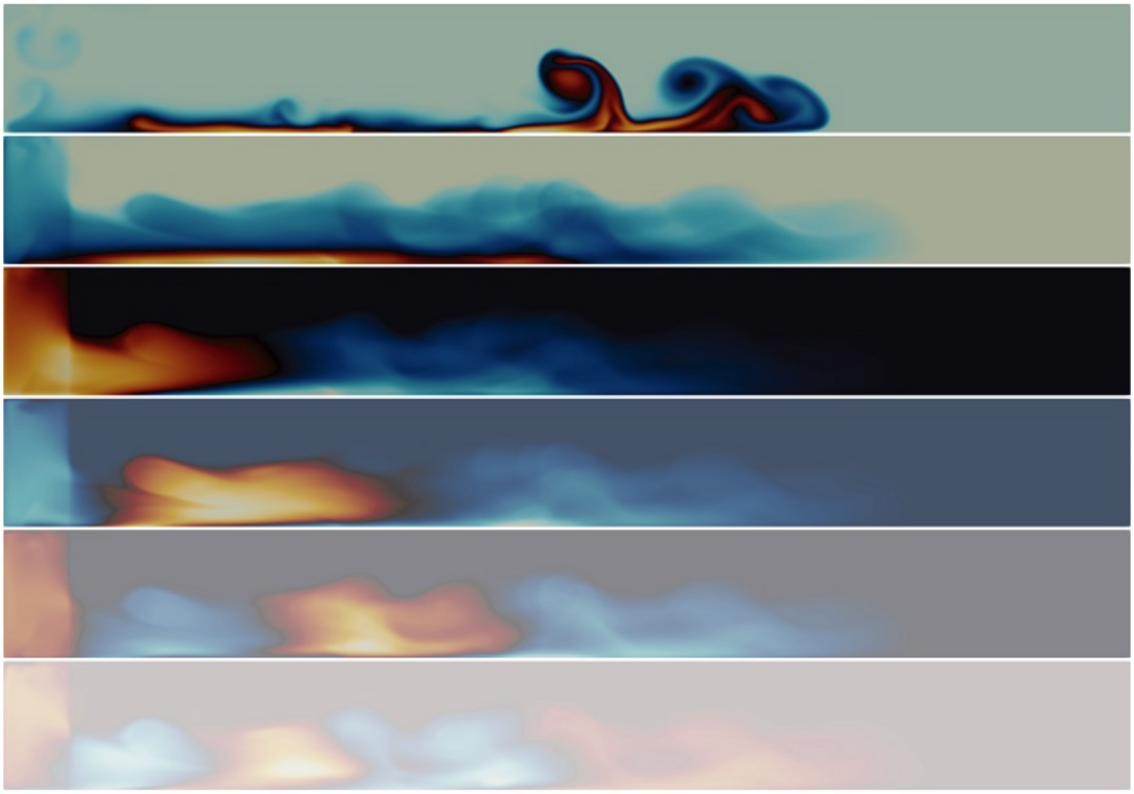


Figure 4.2: A snapshot, the mean field and the first four spatial modes for the 2D lock-exchange problem.

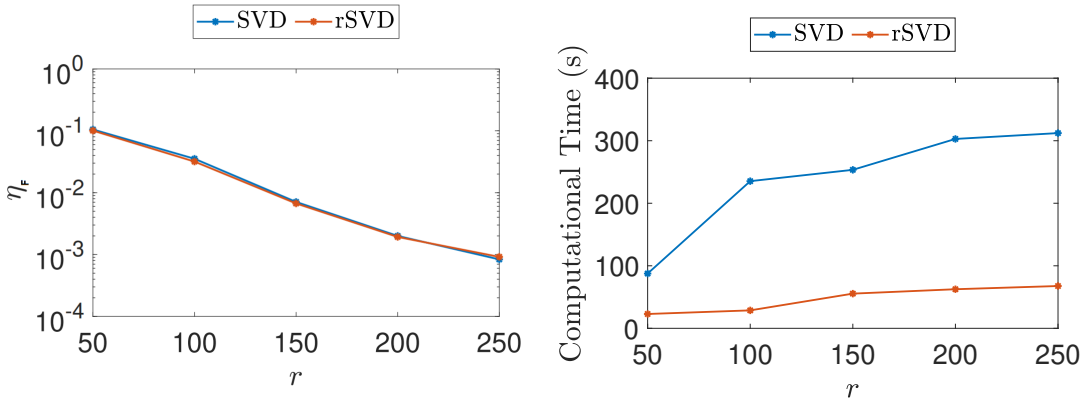


Figure 4.3: Comparison of the Frobenius norm relative error and computational time for different values of r of the DMD code using the SVD and the rSVD algorithms.

relative error decreases when the truncation rank r increases. The results for both procedures are similar. As for the efficiency, we measure the time required for both cases and for the different values of r. We observe that the results for the rSVD are faster for all the tested cases and it can also be noted that the rSVD presents similar accuracy when compared to the SVD algorithm for all values of r. That said, the rSVD is a more efficient approach with little to no cost to accuracy of the matrix

reconstruction.

With the singular values and vectors obtained, the DMD approximation can be computed. Figure 4.4 shows the relative error in time between the original snapshots and the reconstruction of the same state vector using DMD. The results are also confirmed in Table 4.1, where the relative error between the reconstructed and original snapshot matrices is shown for each case as well as the "speedup"[1] for each case. The speedup here is the ratio of the time needed to execute the DMD code and the time needed to generate the simulation data used to fuel DMD. We observe that, while increasing r, the relative error decreases for all time steps and affects the overall relative error of the matrices. In terms of computational effort, we note that the speedup decreases as we increase r. This happens because the numerical complexity of the randomized SVD algorithm considers the rank r in its implementation, differently from the standard SVD routine that completely decomposes the matrix onto all singular values and vectors and is eventually followed by the basis truncation.

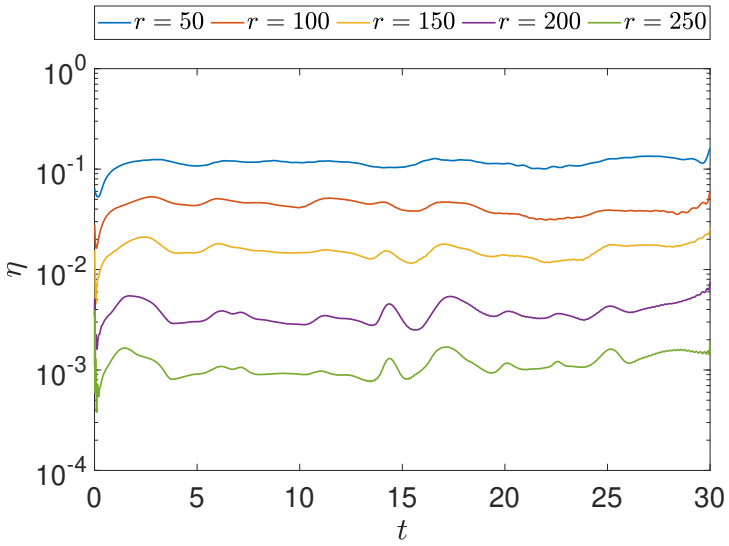


Figure 4.4: Relative error for the reconstruction for the lock-exchange example considering different values of r.

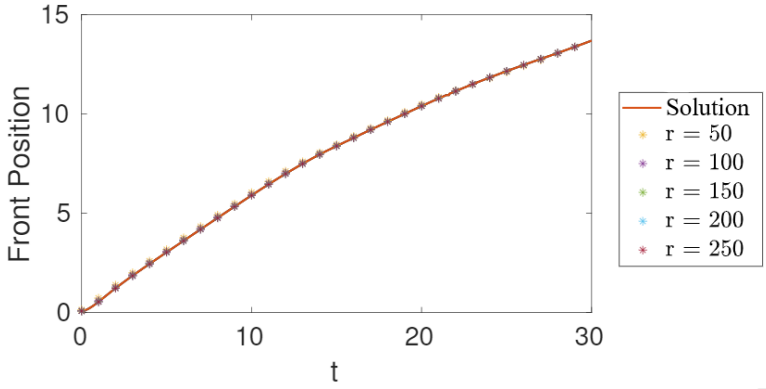
Lastly, we can evaluate the quantities of interest common to density-driven gravity currents. Figure 4.5 shows the front position and mass conservation for both simulations and the respective reconstructions. We note that, even for lower values of r that yield larger $\mathcal{L}^2$ relative errors, the quantities of interest for the DMD approximation are in good agreement with the quantities of interest computed for the

[1]Speedup is quoted because, yet it is an interesting metric to evaluate the approximation efficiency, it cannot replace the simulations used to generate the dataset that fuels DMD.

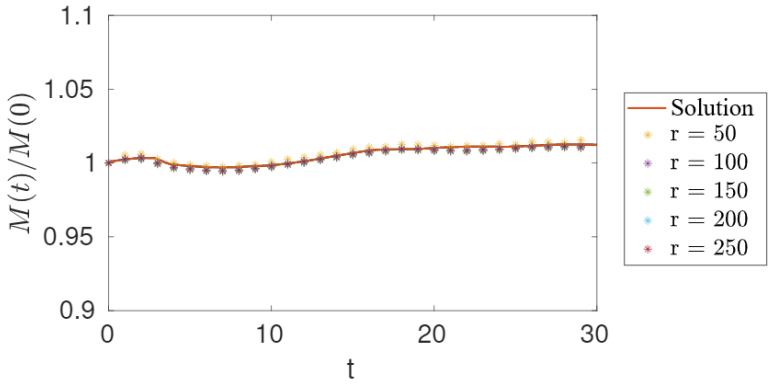
Table 4.1: Relative error between reconstructed data and the snapshots and speedup between DMD and the numerical simulation.

Rank	Relative Error (η_F)	Speedup
50	1.081×10^{-1}	558.53
100	3.472×10^{-2}	445.91
150	6.207×10^{-3}	230.67
200	2.024×10^{-3}	204.74
250	9.604×10^{-4}	189.17

finite element simulation data.



(a) Front position



(b) Mass conservation

Figure 4.5: Front position and mass conservation for the lock-exchange example. Comparison between simulation and reconstructed data.

4.1.2 Adaptive mesh 2D lock-exchange

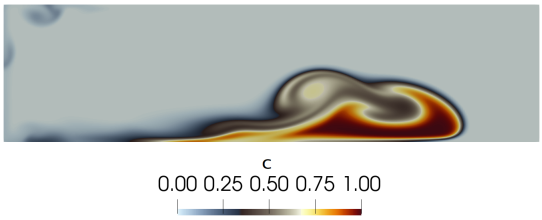
After analyzing the results for the fixed mesh 2D lock-exchange simulation, we can apply DMD to the AMR/C 2D lock-exchange simulation. According to DMD workflow presented on 2.4, the DMD code can be called with the following inputs:

- Data Generation: Simulation is processed using `FEniCS` 2019.1.0 and data is generated in the `FEniCS` HDF5 format.
- Data Transformation: Solution projection into tailored reference mesh.
- Data Ingestion: Python `h5py` library.
- Snapshots Matrix Creation: c snapshots.
- Snapshot Embedding Definition: state vectors.
- Matrix Factorization: rSVD algorithm.
- DMD Basis Construction: $r = \{50, 100, 150, 200, 250\}$.
- State Vector Approximation: Signal reconstruction.
- Metrics Evaluation: Frobenius norm, relative error in time.

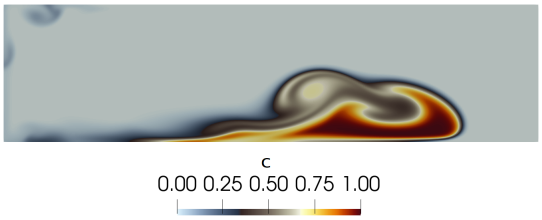
Before applying DMD to the generated data, we need to design a reference mesh able to capture all spatial scales computed in the 2D lock-exchange finite element simulation using the adaptive mesh. As a reference mesh, we consider the same 700×100 equal size cells mesh used in the fixed mesh simulation, that contains the same element size as the smallest possible element in the adaptive simulation. We compare the results from the adaptive simulation with the fixed simulation to guarantee the results are the same. That is, at this point, we have three solutions for the 2D lock-exchange: the adaptive solution obtained via AMR/C simulation, the fixed mesh simulation and the results projected from the adaptive solutions into the reference mesh. Figure 4.6 shows the solutions of all three solutions as well as the adaptive mesh at $t = 10$ time units.

Now that all snapshots belong to the same vector same, we proceed to apply DMD to the projected solution and reconstructing the solutions. Figure 4.7 shows the relative error for the reconstruction using different values of the rank r, yielding results that are very similar to the ones observed on Figure 4.4. We observe that, for increasing values of r, the relative error decreases for all steps and affects the overall relative error of the matrices, similarly to the fixed mesh case. That is, inserting more structures in the DMD basis yields better accuracy in terms of overall relative error for the tested values of r.

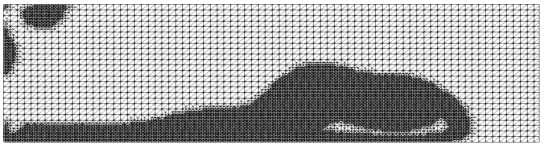
Most importantly, we can evaluate the quantities of interest common to density-driven gravity currents. Figure 4.8 shows the front position and mass conservation for both simulations and the respective reconstructions. We note that, for these quantities, all reconstructions are in good agreement with those computed with the fixed and adaptive meshes, even for smaller values of r. These results are also similar



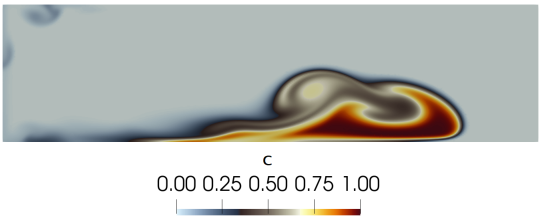
(a) Fixed mesh solution



(b) Projected adaptive solution



(c) Adaptive mesh



(d) Adaptive solution

Figure 4.6: Projected and fixed mesh solutions for the first 8 meters of the domain at $t = 10$s.

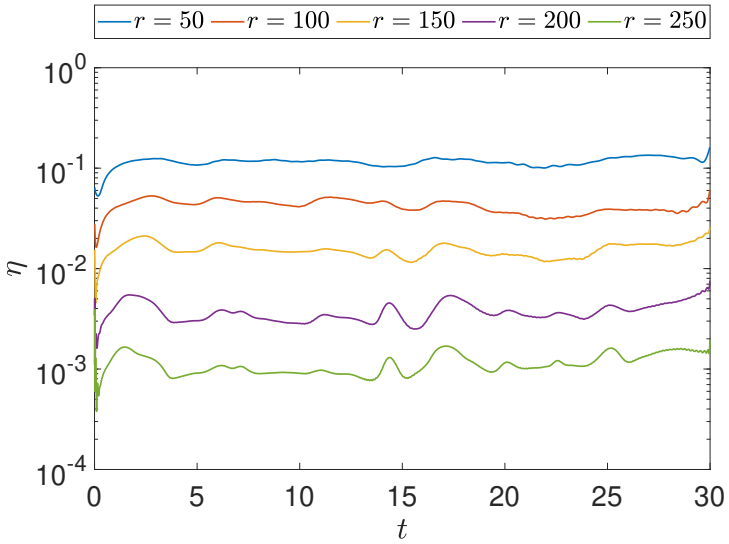
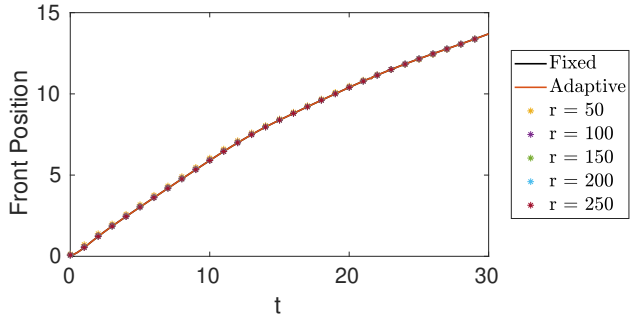


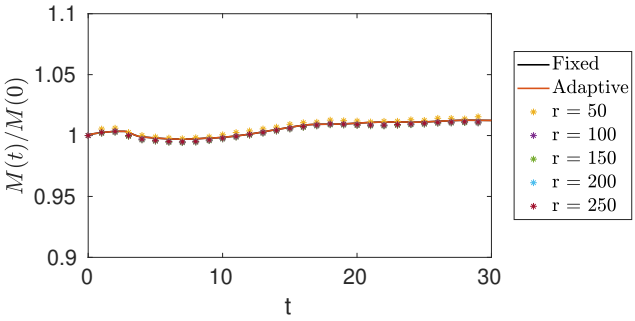
Figure 4.7: Relative error in time for the reconstruction considering different values of the rank r.

to the ones observed for the fixed mesh simulation. Again, to compute the front position we consider extracting the maximum x coordinate for the concentration

field of $c \geq 0.05$ and the mass of the system is computed using Equation 3.4. That said, we observe that the DMD approximation for AMR/C solutions projected into a tailored reference mesh able to capture all spatial scales yields very similar results when compared to fixed mesh data for the 2D lock-exchange problem.



(a) Front position



(b) Mass conservation

Figure 4.8: Front position and mass conservation for the fixed mesh and adaptive mesh simulations and reconstructions with the target mesh.

4.1.3 3D lock-exchange

In this section, we explore DMD on more complex gravity current models. For that, the 3D gravity flow model described in Section 4.1.3 is considered. This is a larger simulation, requiring HPC strategies for efficiency such as parallel computing and data compression. This section is divided into two parts: one where we analyze the standard DMD results for compressed snapshots of the concentration field with different levels of compression and another where we use streaming DMD for data being generated during simulation runtime. In the latter case, the Paraview Catalyst is responsible for generating PNG files from the current simulation state.

Compressed snapshots

In this section, we apply DMD for the 3D lock-exchange simulation compressed data. For this simulation, the blocks on the DMD workflow are the following:

- Data Generation: Simulation is processed using `libMesh` and data is generated in the `libMesh` HDF5 format.
- Data Transformation: Data decompression and assembly of parallel snapshots.
- Data Ingestion: Python `h5py` library and `hdf5plugin` module.
- Snapshots Matrix Creation: c snapshots.
- Snapshot Embedding Definition: state vectors.
- Matrix Factorization: rSVD algorithms.
- DMD Basis Construction: $r = 250$.
- State Vector Approximation: Signal reconstruction.
- Metrics Evaluation: Relative Frobenius error, relative error in time.

In this section, we compare the results from snapshots obtained from HDF5 files under different levels of compression. Information on the coupling of data compression on the `libMesh` library can be seen on [68] and more detail about ZFP, the data compression algorithm used in this thesis can be seen in Appendix D. We focus on evaluating the DMD approximation obtained from the ingestion of snapshots submitted to different data compression strategies. The results are compared in terms of relative errors and quantities of interest for each application.

From this point, our first approach is to decompress all the datasets so that data can be compared even though part of the information was lost on the lossy compression cases. For this purpose, we consider the `h5py` and `hdf5plugin`, two Python modules responsible for enabling the I/O of HDF5 files and compressing and/or decompressing HDF5 files, respectively. As soon as all data is decompressed, the parallel snapshots can be assembled. After the execution of both pre-processing steps, the snapshots matrix can be constructed, and the SVD decomposition of $\mathbf{Y_1}$ is computed. Initially, we observe the singular values obtained from the SVD decomposition of $\mathbf{Y_1}$ on Figure 4.9, where the curves show the singular values for the original data, data with lossless compression, and lossy compression with two different accuracy thresholds. After some trial and error, we choose the $r = 250$ most relevant dynamic modes of the snapshots to reconstruct the solution. It is known that, due to the reversible behavior of the lossless compression strategy, the

snapshot matrices generated from the simulations with no compression and lossless compression are identical. That is, no information is lost during the compression and decompression processes. This is also confirmed by the superposition of the curves for the singular values of the datasets for cases A and B in Figure 4.9. The curves for data generated with lossy compression reveal that the decay for the singular values reaches a plateau at a given point. For lossy compression with accuracy threshold equals to 10^{-6}, the first 350 singular values, approximately, are preserved compared to the lossless and no compression data. Given that the dashed line represents the truncation of the number of singular values for the DMD basis construction, the information lost would not affect the construction of the basis since the difference in the singular values lies ahead of the truncation threshold. For lossy compression with accuracy threshold of 10^{-3}, the truncation is more aggressive and affects the singular values before the basis truncation of $r = 250$, changing the singular values preserved for the DMD basis. According to Figure 4.9, the lossy (10^{-3}) strategy conserves the first 200 singular values, approximately.

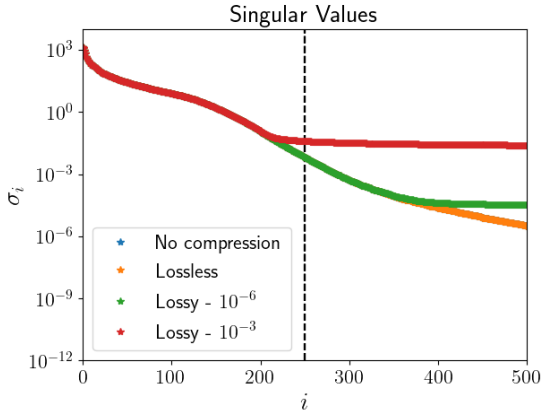
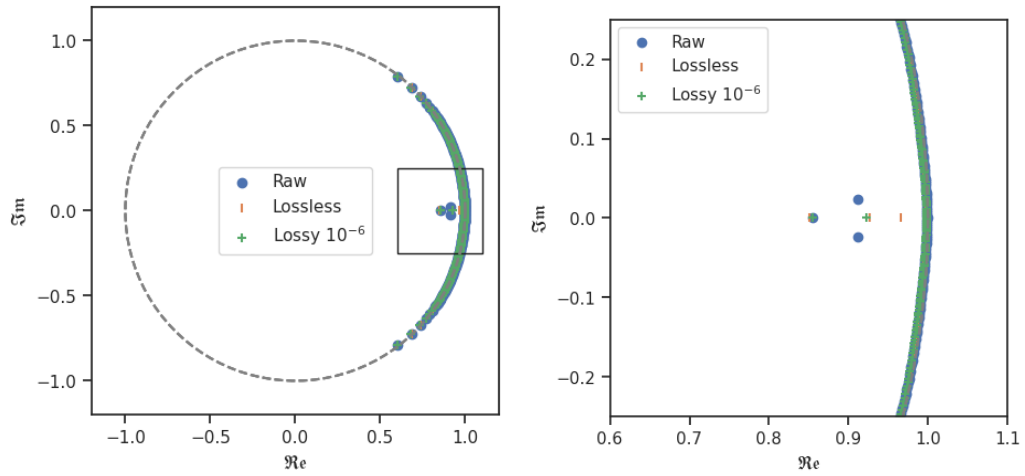


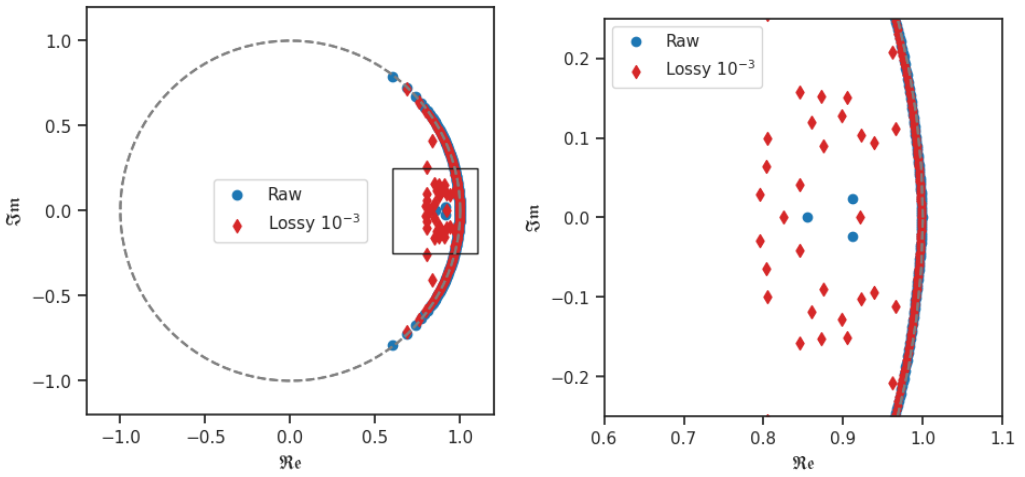
Figure 4.9: Singular values for the tested cases. The dashed line represents the number of $r = 250$ singular values and vectors selected for the DMD basis.

After analyzing the singular values, we proceed to reconstruct the solution using standard DMD to approximate the results. First, we start by investigating the effects of data compression on the eigenvalues of $\tilde{\mathbf{A}}$. Figure 4.10 shows the complex plane and the eigenvalues for this problem. Figures 4.10(a) and 4.10(b) show all eigenvalues for Cases A, B and C and Figures 4.10(c) and 4.10(d) show the comparison between Cases A and D. We observe that all eigenvalues occur in complex conjugate pairs. This is expected given that the snapshots matrix is real-valued data [143]. We notice that the eigenvalues are very similar for Cases A, B, and C, meaning that lossless and high-accuracy lossy compression strategies may not affect the dynamics significantly. When we compare Cases A and D, we observe that more aggressive lossy compression schemes yield different eigenvalues. However, the visualization of DMD eigenvalues on the complex plane *per se* may lead to redundant

components. It is important to mention that the relevance of DMD modes and eigenvalues cannot be interpreted in a straightforward manner for the exact DMD algorithm and the complex plane visualization [143]



(a) Eigenvalues of $\tilde{\mathbf{A}}$. Case D excluded. (b) Zoom at rectangular region of interest.



(c) Eigenvalues of $\tilde{\mathbf{A}}$ for cases A and D. (d) Zoom at rectangular region of interest.

Figure 4.10: Eigenvalues for the 3D lock-exchange simulation.

Although this is a first glance at the solution, we need to assess the approximation after the construction of the DMD modes. We generate the approximations and compare with the snapshots. Figure 4.11 shows the absolute error at $t = 10$ time units between the DMD reconstruction using lossy compression (threshold 10^{-3}) and finite element solution. We can see that the DMD reconstruction at that time instant yields low absolute error. Figure 4.12 shows the relative error time history for DMD reconstructions and finite element solution, while Table 4.2 shows the Frobenius relative error between the snapshots matrix and the matrix containing

all the vectors approximated by DMD. Observing Figure 4.12, we note that the

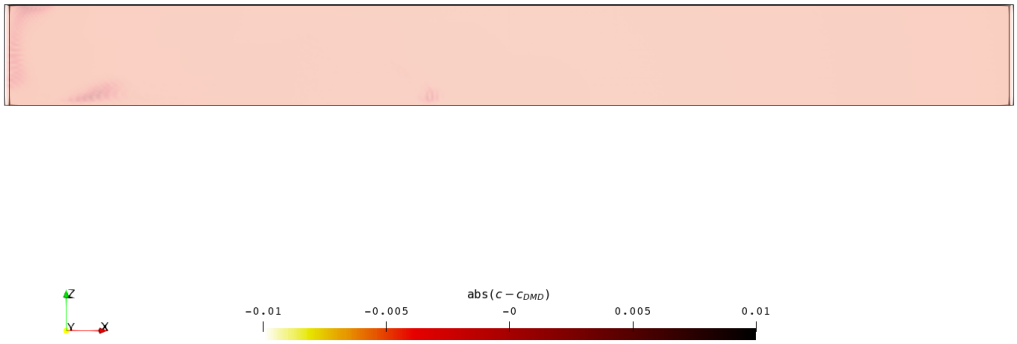


Figure 4.11: Absolute error between DMD reconstruction from lossy compressed data (threshold 10^{-3}) and original snapshot for the sediment concentration at $t = 10$ time units.

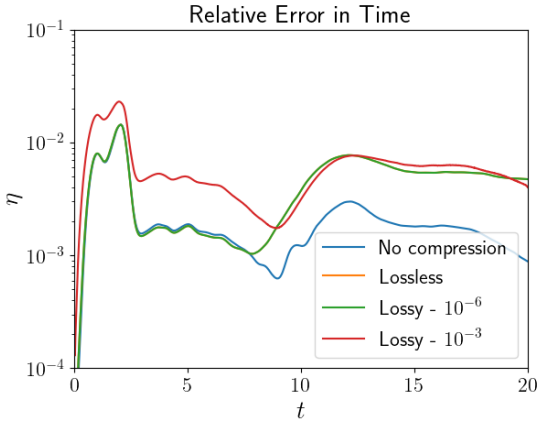


Figure 4.12: Relative error in time for the solution reconstruction.

error magnitudes are of the same order in all cases. Again, the results for lossless compression and no compression are identical. For the lossy compression cases, we observe that the relative errors are slightly higher for the latter stages of the simulation. For $t < 8$ time units, we notice that the results for the lossy compression considering an accuracy threshold of 10^{-6} are very similar to those without lossy compression. It is also essential to notice that the relative error in time tends to fit the dynamics, in the sense that the instant where more significant errors are observed (i.e., the generation of the current front around $t = 2$ time units), the dynamics observed are more rapid, leading to more significant errors in comparison with the

remainder of the simulation. We observe that the curves present similar behavior for all four cases and all relative errors computed are below 2×10^{-2}, meaning there is no significant loss of physical accuracy when using data compression techniques. Further, Table 4.2 shows the relative error in the Frobenius norm (η_F) for the original and approximated matrices. The relative Frobenius norm for all cases is of the same order of magnitude, but for the more aggressive data compression case, we note that η_F is twice as larger. When running DMD, no significant additional computation time is observed for either compressing output files during simulation runtime or decompressing snapshots for DMD computation.

Table 4.2: Frobenius relative error between approximated and original snapshots for the particle-driven gravity flow considering $r = 250$

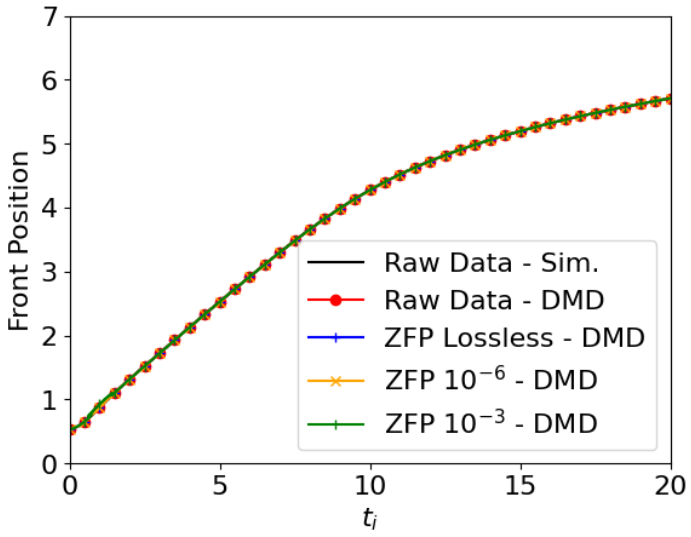
Case	η_F
A	5.28×10^{-3}
B	5.09×10^{-3}
C	5.31×10^{-3}
D	9.95×10^{-3}

Lastly, we also evaluate the quantities of interest for the lock-exchange simulation (namely the front position and the system mass). That is, we need to verify if data compression influences quantities computed with the results of DMD reconstructions. In this case, differently from the 2D lock-exchange simulation, we considered convective flux boundary conditions for the bottom of the domain, reducing the sediment concentration during the simulation. That is, mass conservation is no longer expected. Figure 4.13 shows the quantities of interest for the simulation data of Case A (uncompressed data) in comparison with the four DMD reconstructions. We show the results for front position on Figure 4.13(a) and the sediment mass for the system on Figure 4.13(b). Front position is computed by clipping the concentration field to values where $c \geq 0.05$ and extracting the maximum x coordinate. The system mass is computed using Equation 3.4. We notice that all curves are practically identical, implying that data compression did not affect significantly the quantities of interest for the problem.

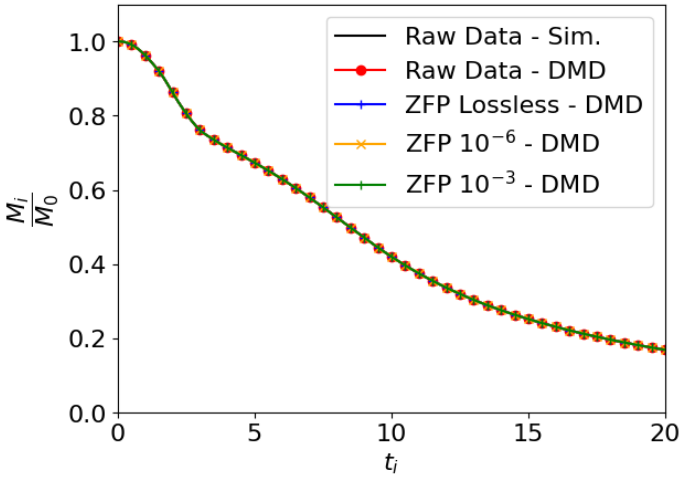
Streaming data

In this section, we focus on applying streaming DMD for the PNG files generated by ParaView Catalyst during the simulation execution. For this case, the blocks for the DMD workflow are:

- Data Generation: *In situ* visualization tool for generating image snapshots during simulation runtime.



(a) Front position.



(b) Mass of sediments over time.

Figure 4.13: Front position and mass of sediments for the 3D lock-exchange simulation.

- Data Transformation: None.
- Data Ingestion: Python `opencv` module for image snapshots.
- Snapshots Matrix Creation: Image pixels from PNG files of the c field for a given plane.
- Snapshot Embedding Definition: state vectors.
- Matrix Factorization: iSVD.
- DMD Basis Construction: $r = 250$.
- State Vector Approximation: Signal reconstruction.

- Metrics Evaluation: Relative Frobenius error in time, relative error in time, SSIM.

In this section, we evaluate the use of streaming DMD on the images obtained from Paraview Catalyst generated in runtime as the simulations evolve. Here the snapshots are pixel values of the PNG images instead of the nodal values obtained from the finite element discretization. We consider images generated at the midplane of the 3D lock-exchange domain for the concentration field. We define $r = 250$ as our threshold for a snapshots matrix containing 2001 snapshots.

For this problem, the streaming DMD is considered. In this case, snapshots arrive on-the-fly and the DMD basis is updated with every new snapshot. Although the streaming DMD does not required assembling and storing the snapshots matrix, we do so for metrics purposes. In this study, we do not consider a DMD basis with less than r vectors, although it would be possible to construct a basis with less than r vectors for fewer snapshots. For instance, for 10 relevant snapshots available, one could construct a temporary basis of $r = 10$ until reaching the threshold. However, we start constructing the basis as soon as the first $r = 250$ relevant snapshots are collected, so the DMD basis will always have their dimensions preserved while being updated with every new snapshot. Figure 4.14 shows the image produced with ParaView Catalyst for the concentration profile at the domain midplane in the lock-exchange problem and the streaming DMD reconstructed image at the same instant ($t = 12$ time units).

Figure 4.14: *In situ* visualization of the sediment concentration profile at the midplane image file at $t = 12$ time units (top) and streaming DMD reconstructed image at the same time instant (bottom).

To assess the DMD accuracy on the reconstruction of the PNG files, we consider the SSIM metric and the relative $\mathcal{L}^2$ error in time as well as the relative Frobenius norm in time. Although streaming DMD does not require storing the snapshots matrix, in this case, we stack the PNG image files into the image snapshots matrix and it varies in time with the inclusion of one image snapshot at a time. Thus, differently from the standard DMD where the computation of the Frobenius error is done once - when the approximated vectors are computed for all time instants existent in the snapshots matrix -, in this case both approximation and snapshots matrices are altered in time. So instead of returning one value for the entire simulation time,

the relative Frobenius error is assessed in time.

We start by analyzing the SSIM metric for the DMD image approximation. The PNG files have 499×51 pixels, and the window size used is 7 pixels - scikit-image's default value. We can see in Figure 4.15 that the metric stays close to 1.0 during the whole time interval of interest, indicating that the DMD reconstructed images for the sediment concentration at the midplane are of good quality. Now, comparing the

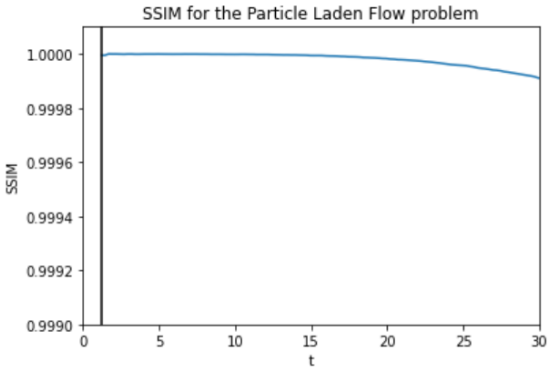


Figure 4.15: Evolution in time of the structural similarity index for the DMD reconstructed images for concentration at the midplane.

results in terms of relative Frobenius and $\mathcal{L}_2$ errors, Figure 4.16 shows the evolution in time for the Frobenius norm of the snapshots matrix and the matrix containing the approximations obtained from the streaming DMD algorithm. The $\mathcal{L}_2$ norm relative error can also be seen. The solid vertical line indicates the instant where the DMD basis has reached $r = 250$ vectors. From this point, the basis gets updated with the arrival of new snapshots relevant to the DMD modes but still preserves $r = 250$. We observe that both metrics are below 10^{-2} for all time steps, indicating that the approximation is adequate. We also notice that the largest errors in Figure 4.16 occur when the moving front is created (around $t = 3.5s$), meaning that the rapid physics during this stage are translated into more pronounced relative errors between the image files. We also notice this phenomenon in Figure 4.15, being the smallest similarity index related to this stage.

4.2 Bubble rising

Another fluid dynamics problem that arises in research is the bubble rising problem. In this section, we apply DMD to snapshots generated from the simulation described on Section 3.2. This problem was modeled using data compression, parallel computing and AMR/C strategies [68]. Following the pipeline seen for the 3D lock-exchange simulation, we analyze the results for the bubble rising problem considering the standard DMD - when snapshots are provided in a batch of output

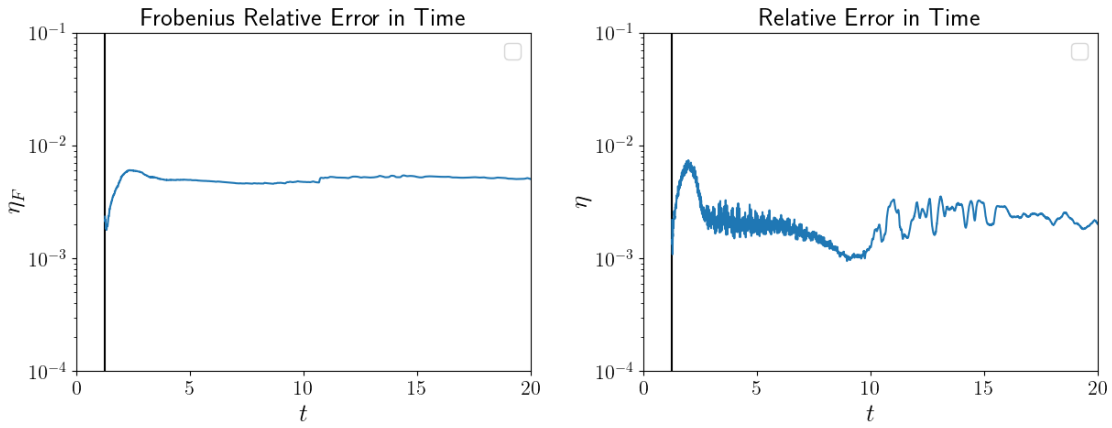


Figure 4.16: Evolution of the Frobenius relative error in time for ($r = 250$) (left) and evolution of the $\mathcal{L}_2$ relative error in time for ($r = 250$) (right) for the lock exchange problem.

data - and the streaming DMD, when an *in situ* visualization tool is responsible for creating image snapshots from a predefined midplane and DMD basis get updated with every new generated PNG file.

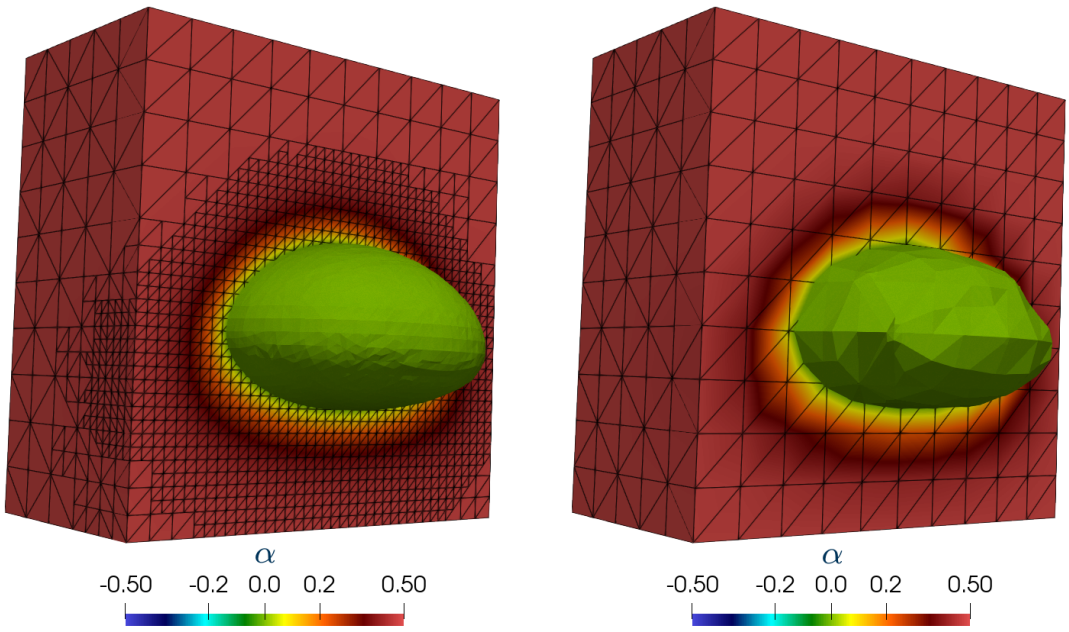
4.2.1 Adaptive mesh bubble rising with snapshot compression

For this problem, we start by applying the standard DMD to the parallel snapshots obtained through the compressed HDF5 files. The DMD workflow for this problem can be translated as:

- Data Generation: Simulation is processed using `libMesh` and data is generated in the `libMesh` HDF5 format.
- Data Transformation: Solution projection into tailored reference mesh, parallel snapshot assembly and data decompression.
- Data Ingestion: Python `h5py` library and `hdf5plugin` module.
- Snapshots Matrix Creation: α snapshots.
- Snapshot Embedding Definition: state vectors.
- Matrix Factorization: rSVD algorithm.
- DMD Basis Construction: $r = \{5, 10, 15, 30, 45, 60\}$.
- State Vector Approximation: Signal reconstruction.
- Metrics Evaluation: Relative Frobenius error, relative error in time, SSIM.

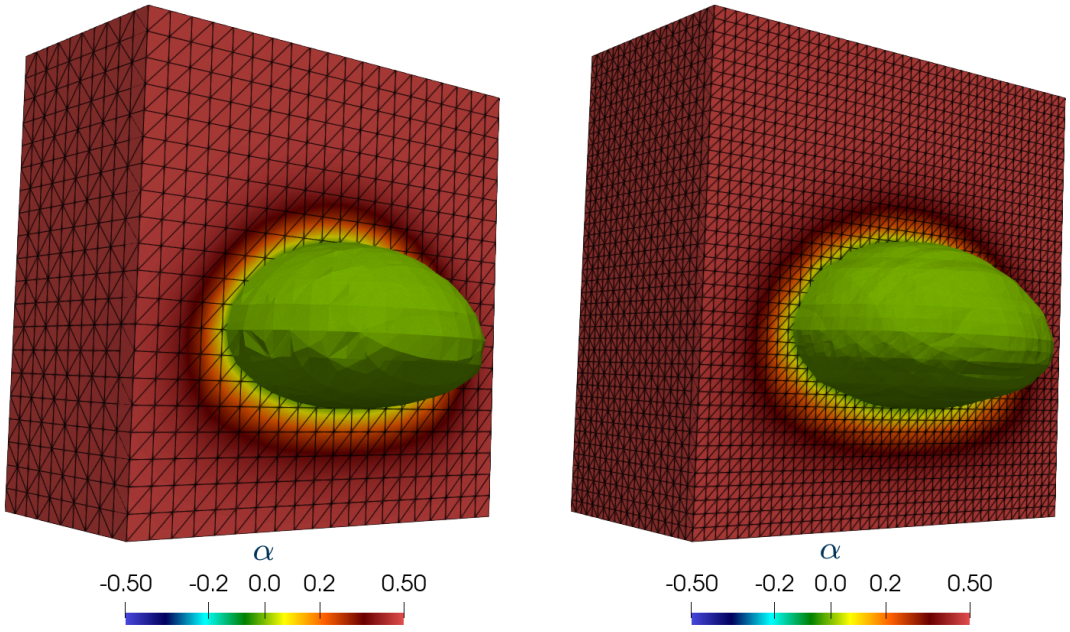
In this example, we start by decompressing all snapshots. Similarly to the 3D lock-exchange problem, the bubble rising problem also generates four datasets: the raw dataset not submitted to data compression (Case A), the compressed dataset using a lossless strategy (Case B) and two datasets submitted to lossy compressions with preserved accuracies of 10^{-6} (Case C) and 10^{-3} (Case D). The next step is to assemble the parallel snapshots by removing duplicated degrees of freedom. Now, the snapshots are decompressed and assembled and ready for projection into the reference tailored mesh. The first analysis we propose for this problem is to test three different tetrahedral meshes for our projection strategy, presented in Figure 4.17. We want to observe the behavior of DMD when the reference tailored mesh is not able to fully capture all spatial scales of the adaptive mesh. For this study, only the dataset not submitted to data compression (Case A) is considered. The three meshes named coarse, intermediate and fine, present characteristic lengths similar to the three scales existent in the refinement levels of the adaptive mesh simulation. That is, the coarse mesh represents the initial mesh on the adaptive simulation, a $10 \times 20 \times 10$ cells mesh with 12000 elements. The intermediate mesh presents smaller elements equivalent to the generated elements after the first refinement level on the AMR/C simulation, totalizing $20 \times 40 \times 20$ cells with 96000 elements, and the fine mesh contains the smallest scales presented on the adaptive numerical solution resulting on a mesh with $40 \times 80 \times 40$ cells and containing 768000 elements. The figures show the solution on half of the domain and at $t = 2.5$ s for the adaptive mesh solution and the respective projections. We can visually notice how the bubble geometry is affected by the mesh projections. In order to quantify how the solution is affected by the projection, we evaluate the quantities of interest such as the bubble volume, sphericity, center of mass, and rise velocity for the AMR/C simulation output and the projections on Figure 4.18. We observe that the geometry of the bubble is affected for the coarse mesh projection, leading to bad results relative to the bubble volume and sphericity - quantities of interest regarding the bubble structure - when compared with the AMR/C simulation, while for the intermediate and the fine meshes, the geometry of the bubble is not largely affected, and the results are compatible with the AMR/C solution. Concerning quantities of interest with respect to the bubble dynamics, we also observe that the center of mass is not affected by the projections compared with the simulation results. As for the rise velocity, we observe some minor differences in all projection cases.

We now proceed to apply DMD to the projected solutions. We analyze the results in terms of relative error between the snapshot matrix and the obtained solutions for each case. The DMD solution for the coarse mesh is compared to the projected adaptive solution onto the coarse mesh and so forth. This is done for multiple values of r, such that $r = \{5, 10, 15, 30, 45, 60\}$. Figure 4.19 shows the singular values decay



(a) Adaptive mesh solution

(b) Coarse mesh projection

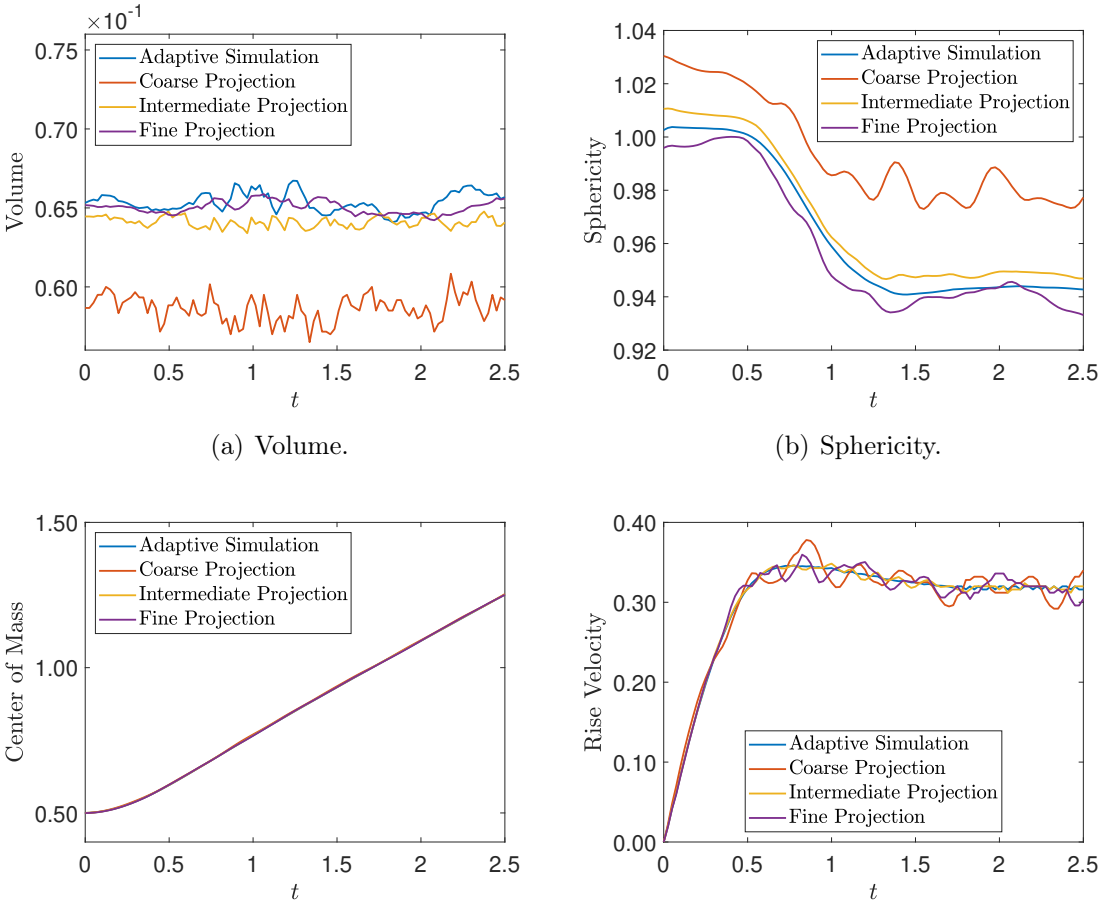


(c) Intermediate mesh projection

(d) Fine mesh projection

Figure 4.17: Adaptive solution detail at $t = 2.5$s and respective projections to the coarse, intermediate and fine meshes. Top half of the domain.

of the snapshot matrix for this example for the three reference meshes. We observe that, given a singular value σ_i, its value decreases as the projection mesh becomes coarser for all singular values visualized in Figure 4.19. We can also compare the results in terms of performance for the considered cases. We test the results for multiples values of r and present the results on Table 4.3, which shows the relative



(a) Volume.

(b) Sphericity.

(c) Center of mass.

(d) Rise velocity.

Figure 4.18: Comparison between the simulation and projection of the 3D rising bubble quantities of interest.

errors between the snapshot matrix and the DMD solution matrix, as well as the speedups between the time required for the simulation and the time required for DMD.

We observe from Table 4.3 that the relative errors decrease for larger values of r. We also note that the speedup decreases while considering finer reference meshes. The results for $r = 60$ are shown in Figure 4.20, where we observe that the errors are stable and below the value of $\eta = 3 \times 10^{-3}$, indicating good agreement between approximations and snapshots. We also illustrate the results comparing the DMD reconstructions (considering $r = 60$) for the three meshes and the adaptive mesh solutions at $t = 1.50$ s and $t = 2.50$ s in Figure 4.21. Again, we observe that the coarse mesh results do not capture the bubble geometry with the same accuracy as the intermediate and fine meshes for the reconstruction results. As for the intermediate and fine meshes, they present similar results in comparison with the projected solutions.

We now proceed to compare the results in terms of quantities of interest for the

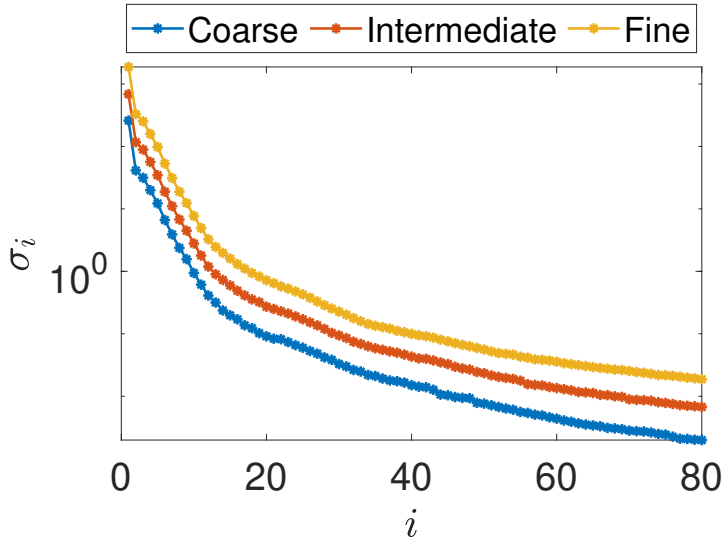


Figure 4.19: Singular values for the snapshot matrix of the rising bubble simulation.

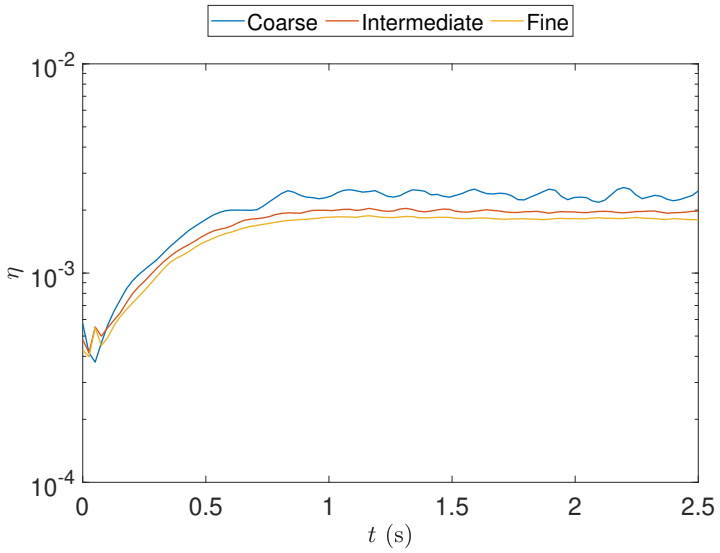
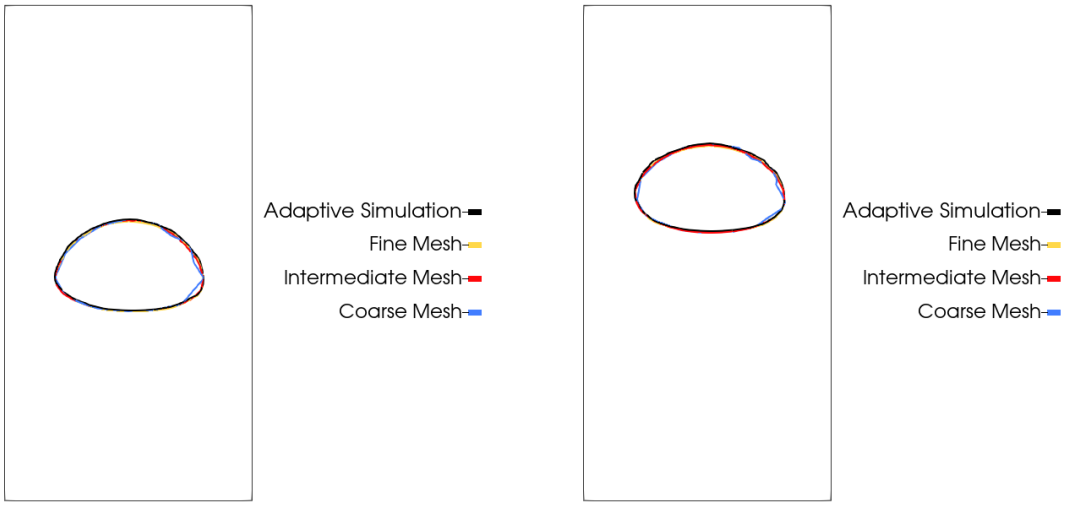


Figure 4.20: Relative error for the different mesh projection strategies of the rising bubble example.

Table 4.3: Relative error between reconstructed data and the projected snapshots and speedup between DMD and the numerical simulation. Results presented for multiple values of r

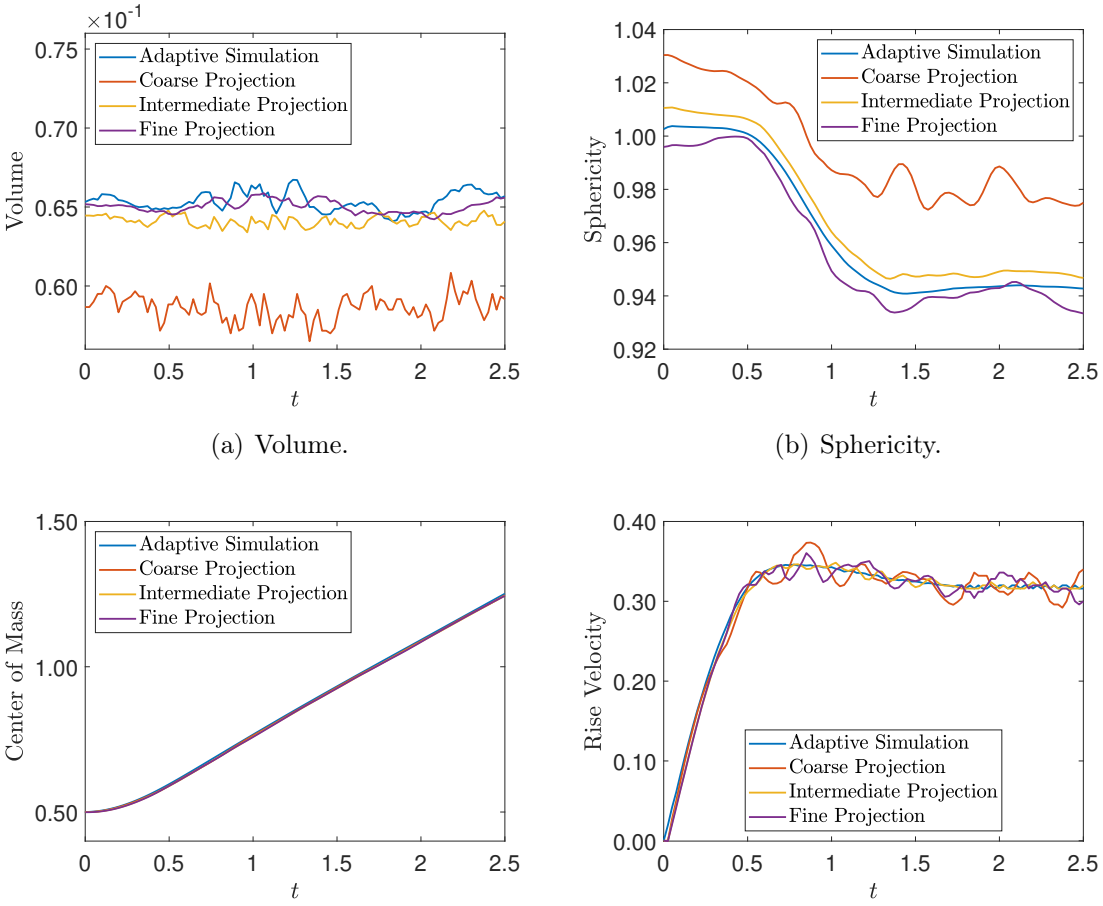
Rank	Mesh	Rel. Error (η_F)	Speedup
5	Coarse	6.222×10^{-2}	1.33×10^{5}
	Intermediate	6.655×10^{-2}	7.95×10^{4}
	Fine	6.865×10^{-2}	1.53×10^{4}
10	Coarse	2.384×10^{-2}	1.21×10^{5}
	Intermediate	2.461×10^{-2}	3.60×10^{4}
	Fine	2.490×10^{-2}	1.22×10^{4}
15	Coarse	1.415×10^{-2}	1.15×10^{5}
	Intermediate	1.402×10^{-2}	2.46×10^{4}
	Fine	1.429×10^{-2}	7.55×10^{3}
30	Coarse	8.900×10^{-3}	1.09×10^{5}
	Intermediate	8.125×10^{-3}	3.29×10^{4}
	Fine	7.687×10^{-3}	6.66×10^{3}
45	Coarse	6.382×10^{-3}	1.02×10^{5}
	Intermediate	5.820×10^{-3}	5.35×10^{4}
	Fine	5.776×10^{-3}	5.80×10^{3}
60	Coarse	5.659×10^{-3}	8.74×10^{4}
	Intermediate	5.444×10^{-3}	1.45×10^{4}
	Fine	5.518×10^{-3}	5.53×10^{3}



(a) $t = 1.50$s. (b) $t = 2.50$s.

Figure 4.21: Bubble contour at the vertical mid plane for the signal reconstruction on $t = 1.50$s and $t = 2.50$s. Results for $r = 60$.

DMD results, considering $r = 60$. Figure 4.22 shows the bubble volume, sphericity, center of mass, and rise velocity for the DMD results compared to the adaptive solution results. The same issue regarding sphericity on the coarse mesh projection is observed on the coarse mesh DMD results. However, for the intermediate and fine meshes, the values match the results observed for the projection. We observe results in conformity for the center of mass and rise velocity as well. That is, considering that Figure 4.22 is very similar to Figure 4.18, we can notice that DMD reconstruction yields good representation for the quantities of interest.



(a) Volume.

(b) Sphericity.

(c) Center of mass.

(d) Rise velocity.

Figure 4.22: Comparison between the simulation and DMD reconstruction of the 3D rising bubble quantities of interest.

Now we have observed the behavior of different reference tailored mesh candidates for the bubble rising problem, we can analyze the results of compressed data in this problem. We project Cases A, B, C and D into the fine reference mesh. This concludes the `Data Transformation` phase and the snapshots are ready for processing in the DMD code. From this point, although we have tested several values for r, we consider $r = 20$ the DMD approximation on the compressed data. Figure 4.23 shows the singular values decay of the snapshot matrix as well as the relative error

in time between the numerical results and the respective approximation. We observe

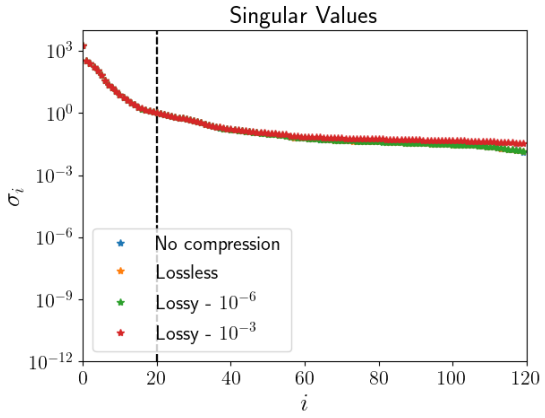


Figure 4.23: Singular values for the tested cases.

that the singular values obtained for compressed data from Figure 4.23 are similar to most of the singular values for the fine mesh on Figure 4.19. We also notice that the singular values' curves practically match, especially inside the truncation threshold for the DMD basis ($r = 20$, or $16,7\%$ of all snapshots available) represented by a vertical dashed line. Similar to the 3D lock-exchange case, the latter snapshots differ for more aggressive compression strategies. However, this does not suggests substantial differences on the DMD approximation, given that, for the first $r = 20$ singular values used for approximation in all cases, the difference is not relevant. We can verify this hypothesis by analyzing the eigenvalues of $\tilde{\mathbf{A}}$ for the bubble rising problem, illustrated on Figure 4.24. We confirme that, for this case, there is no sub-

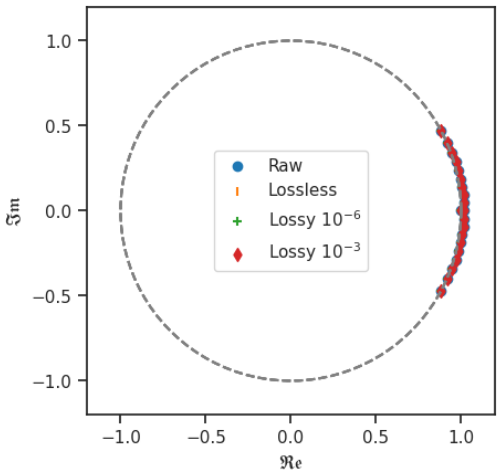


Figure 4.24: Eigenvalues of $\tilde{\mathbf{A}}$ for the bubble rising simulation.

stantial difference between the DMD eigenvalues obtained for compressed data with an accuracy of 10^{-3} in comparison with the other cases. This is different from what

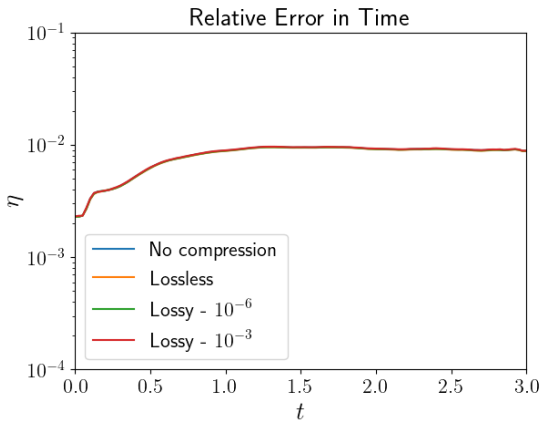


Figure 4.25: Relative error in time for the reconstruction ($r = 20$).

we observed for the 3D lock-exchange DMD eigenvalues on Figure 4.10. Given that the threshold $r = 250$ for the previous example contains different singular values for the four datasets due to data compression, the DMD eigenvalues are affected as well. We do not observe this phenomenon on the rising bubble problem, where the four datasets contain the same singular values, and, consequently, the same DMD eigenvalues. We proceed to the relative error analysis for the DMD approximation of the bubble rising problem. The relative error for the four cases confirms that there is no significant difference when using data compression in this example. The curves illustrated in Figure 4.25 show that the results are practically coincident. Figure 4.26 shows the absolute error at $t = 2.5$ s obtained from ZFP compressed data with an accuracy of 10^{-3}, revealing an adequate approximation. We also evaluate the Frobenius relative error between the snapshots matrix and the approximated solutions matrix in Table 4.4. We notice that the results are kept within an approximated Frobenius relative error of $\mathcal{O}(10^{-3})$, indicating that the approximation results have good accuracy.

Table 4.4: Frobenius relative error between approximated and original snapshots for the bubble rising problem considering $r = 20$

Test case	η_F
No compression	6.14×10^{-3}
Lossless	6.14×10^{-3}
Lossy (10^{-6})	6.16×10^{-3}
Lossy (10^{-3})	7.15×10^{-3}

We now evaluate if the data compression influences the reconstruction of the bubble mass. Figure 4.27 shows the time history of the relative error of the bubble mass computed by the finite element simulation and the DMD reconstruction using uncompressed and compressed snapshots. We note that there is little difference between the time histories. Therefore, no physical information regarding the dynamics

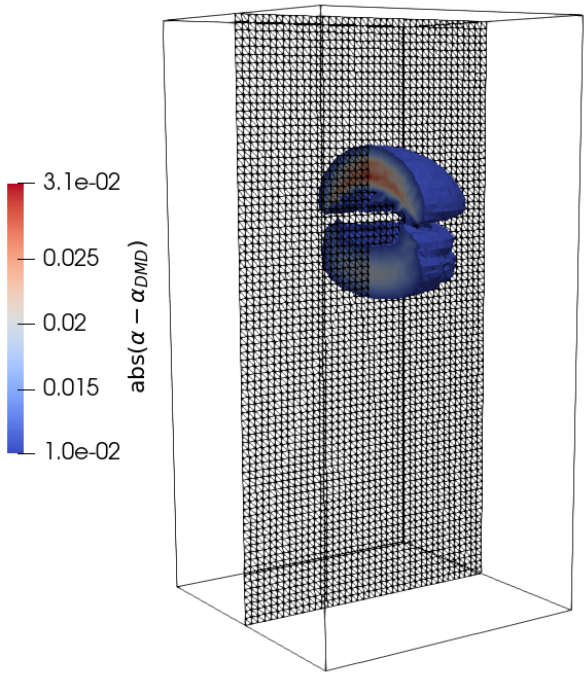


Figure 4.26: Results for the absolute error between DMD reconstruction with lossy compressed data (threshold 10^{-3}) and original snapshot at $t = 2.5s$. The clipping shows the absolute error values larger than the threshold of 10^{-2}. Note that the maximum absolute error is 3.1×10^{-2}.

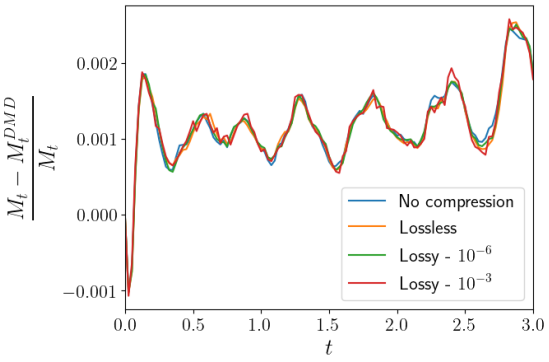


Figure 4.27: Time history of the bubble mass error for all cases. For this problem, $M_t = \int_\Omega \alpha d\Omega$ and the superscript ^{DMD} refers to the same expression but taking the DMD reconstruction of α.

is lost using data compression when reconstructing derived quantities such as the bubble mass.

4.2.2 Streaming data

In this section, we consider that data is also generated using Paraview Catalyst during simulation runtime. We consider a vertical midplane that generates a 2D image of the bubble. For this case, the blocks for the DMD workflow are:

- Data Generation: *In situ* visualization tool for generating image snapshots during simulation runtime.
- Data Transformation: None.
- Data Ingestion: Python `opencv` module for image snapshots.
- Snapshots Matrix Creation: Image pixels from PNG files of the α field for a given plane.
- Snapshot Embedding Definition: state vectors.
- Matrix Factorization: iSVD.
- DMD Basis Construction: $r = 20$.
- State Vector Approximation: Signal reconstruction.
- Metrics Evaluation: Frobenius norm in time, relative error in time, SSIM.

Figure 3.13 shows a few image snapshots generated during the simulation. The images produced by Paraview Catalyst can be seen as well as the DMD approximation at the same instants. Visually, the DMD reconstructions become blurry

compared to the original image as the bubble moves toward the top of the domain. The blurriness can be explained by using linear factorization algorithms (SVD) for advection-dominated dynamics. Although our study solely focuses on the standard DMD and streaming DMD algorithms, improving the factorization algorithm into a nonlinear approach [144–147] could improve the solution in this direction. This lack of sharpness is reflected in the evolution in time of the SSIM metric, as shown in Figure 4.29. The SSIM metric decays immediately after the bubble starts to rise, stays at the same level until around $t = 2.9$s, and is less accurate at later stages, where the bubble is near the top of the domain. However, note that the SSIM is acceptable, above 0.95 [148] in most of the time steps.

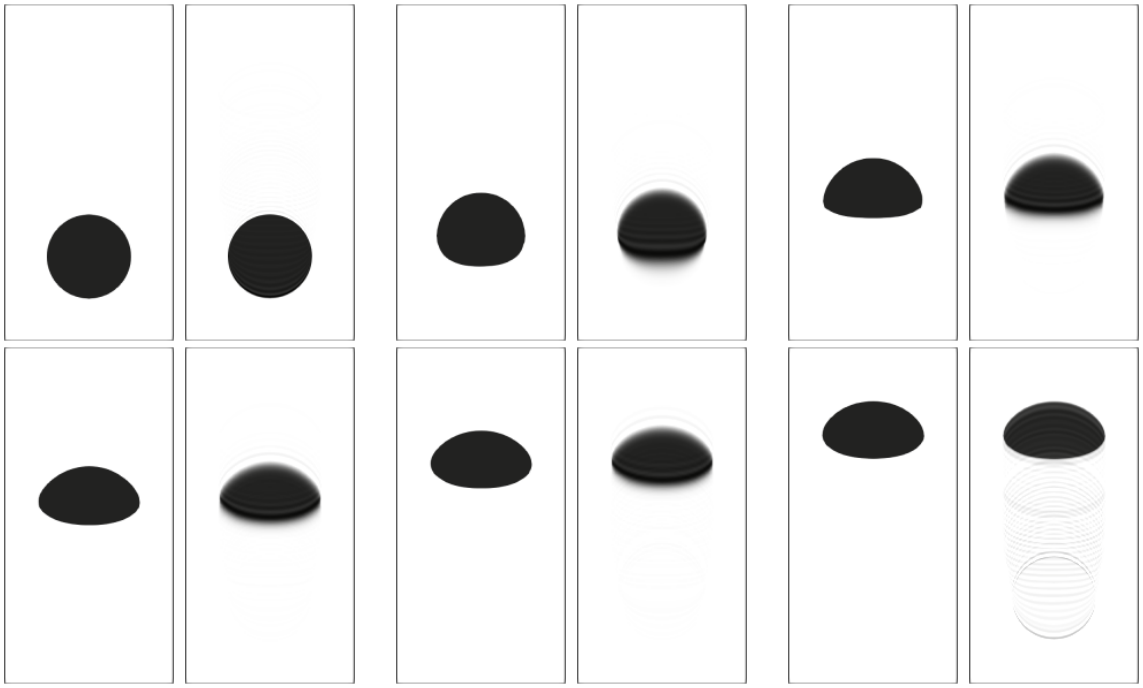


Figure 4.28: *In situ* visualization image file of the bubble shape (left), streaming DMD reconstructed image ($r = 60$) at the same instant (right). From top left to bottom right, $t = \{0.000, 0.625, 1.250, 1.875, 2.500, 3.000\}$s.

Other metrics also show a moderate degradation in the quality of DMD reconstructions. Figure 4.30 shows the time history of the Frobenius and $\mathcal{L}_2$ norms in time between approximation and snapshots. We can see that as the bubble rises, both the relative Frobenius norm and the relative error grow, but both are still relatively low (less than 10^{-1}).

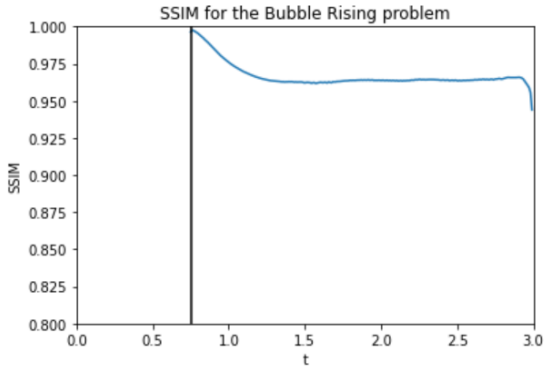


Figure 4.29: Evolution in time of the structural similarity index for the DMD reconstructed images for the bubble shape at the midplane.

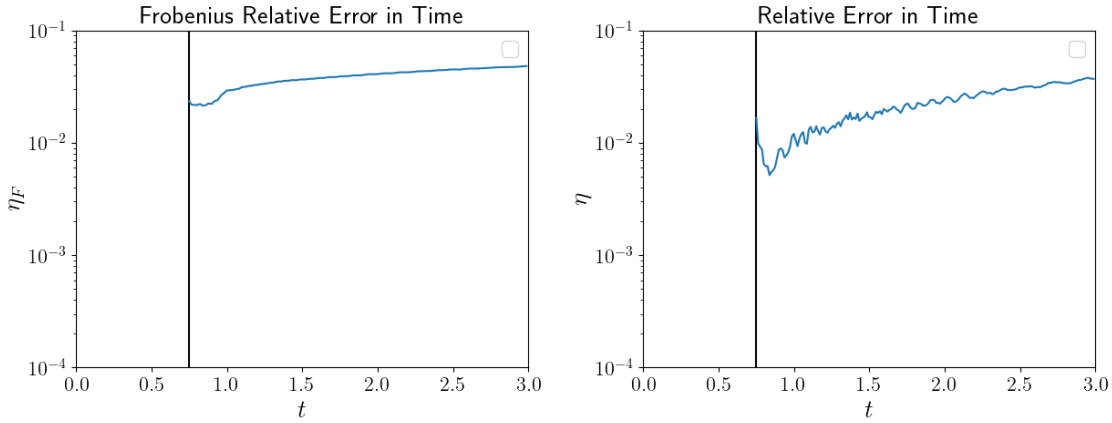


Figure 4.30: Evolution of the Frobenius relative error in time for ($r = 60$) (left), evolution of the $\mathcal{L}_2$ relative error in time for ($r = 60$) (right) for the bubble rising problem.

4.3 COVID-19 spread via the continuous SEIRD model

In this chapter, we evaluate the short-time future estimates using DMD for the obtained data from numerical simulations of a continuous SEIRD model for COVID-19. We consider the real-world example of the Lombardy region, in Italy. Results for the simulation were presented and validated on [132, 140, 149]. Modeling details are covered in Chapter 3 of this thesis.

According to DMD workflow presented on 2.4, this problem can be solved using DMD with the following inputs:

- Data Generation: Simulation is processed using `libMesh` and data is generated in the `libMesh` HDF5 format.
- Data Transformation: Solution projection into tailored reference mesh.

- Data Ingestion: Python `h5py` library.
- Snapshots Matrix Creation: s, e, i, r, d and c snapshots.
- Snapshot Embedding Definition: State vectors.
- Matrix Factorization: rSVD algorithm.
- DMD Basis Construction: $r = 20$.
- State Vector Approximation: Short-time prediction.
- Metrics Evaluation: Relative energy, Frobenius norm, relative error in time.

The use of DMD on epidemiological models can be of great help on decision making of public policies to contain the outbreak of diseases or further infections or deaths. For this problem, we consider training DMD with data relative to 46 days of simulation and predicting the next 14 days. That is, the snapshot matrix contains information regarding 46 days of simulations, leading to $m = 184$ snapshots. Figure 4.31 show the singular values decay and the relative energy lost during the approximation for different values of r. All compartments, including the cumulative infected compartment, are processed in the DMD algorithm. Based on Figure 4.31, we choose $r = 20$ for all compartments.

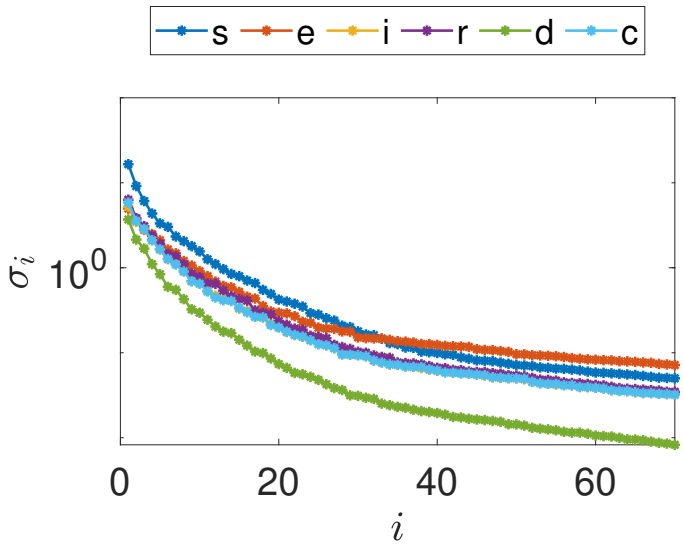


Figure 4.31: Singular values for the snapshot matrix of the continuous SEIRD model.

An initial 3 days shift in the data is considered to avoid issues with the compartments initialized with zeros. We then proceed to reconstruct the results from day 4

until day 46 and then predict 14 days in the future as well. Results for the 60th day comparing the computed numerical solutions and the DMD predictions are seen on Figures 4.32 and 4.33. From left to right, we present the numerical solutions, the DMD prediction and the relative error between the two results. We can note that the most of the compartments show results in agreement with the simulations while the exposed compartment reveals more differences.

Figure 4.34 shows the relative error in time between the DMD results and the snapshots. We observe that as soon as the prediction stage starts (dashed line), the errors tend to grow. We also note that the exposed compartment yields the larger relative error for the 60th day, which agrees to the results observed on Figure 4.32. Table 4.5 shows the speedup between the DMD algorithm and the numerical simulations as well as the overall relative error between the snapshot matrix and the matrix containing the DMD solutions. Comparing the results presented in Figures 4.32, 4.33 and 4.34, we can conclude that the predictions are reasonably accurate in comparison with the numerical solutions, specially when considering the time required for calculation.

Table 4.5: Relative error between reconstructed (and predicted) data and the computed snapshots and speedup between DMD and the numerical simulation.

Compartments	Relative Error (η_F)	Speedup
s	1.345×10^{-3}	822.61
e	5.187×10^{-2}	938.09
i	1.304×10^{-2}	755.97
r	7.316×10^{-3}	1036.93
d	1.097×10^{-2}	977.91
c	1.251×10^{-2}	977.03

We also consider the population conservation property of the continuous SEIRD model. Figure 4.35 shows the total population during the simulation, normalized by the total population modeled in the initial conditions. The total population is computed as the sum of the integral of the compartments divided by the sum of the integral of the elements of the mesh. Since the SEIRD model does not consider any population growth, the value must be theoretically constant for all the simulation. We observe good agreement among the numerical simulation results and the DMD reconstruction. A small gap revealing a 0.1% difference between the predicted total population with the computed total population close to $t = 60$ days can be related to the larger relative errors observed on Figure 4.32 (specially for the exposed compartment) for the 14 days prediction.

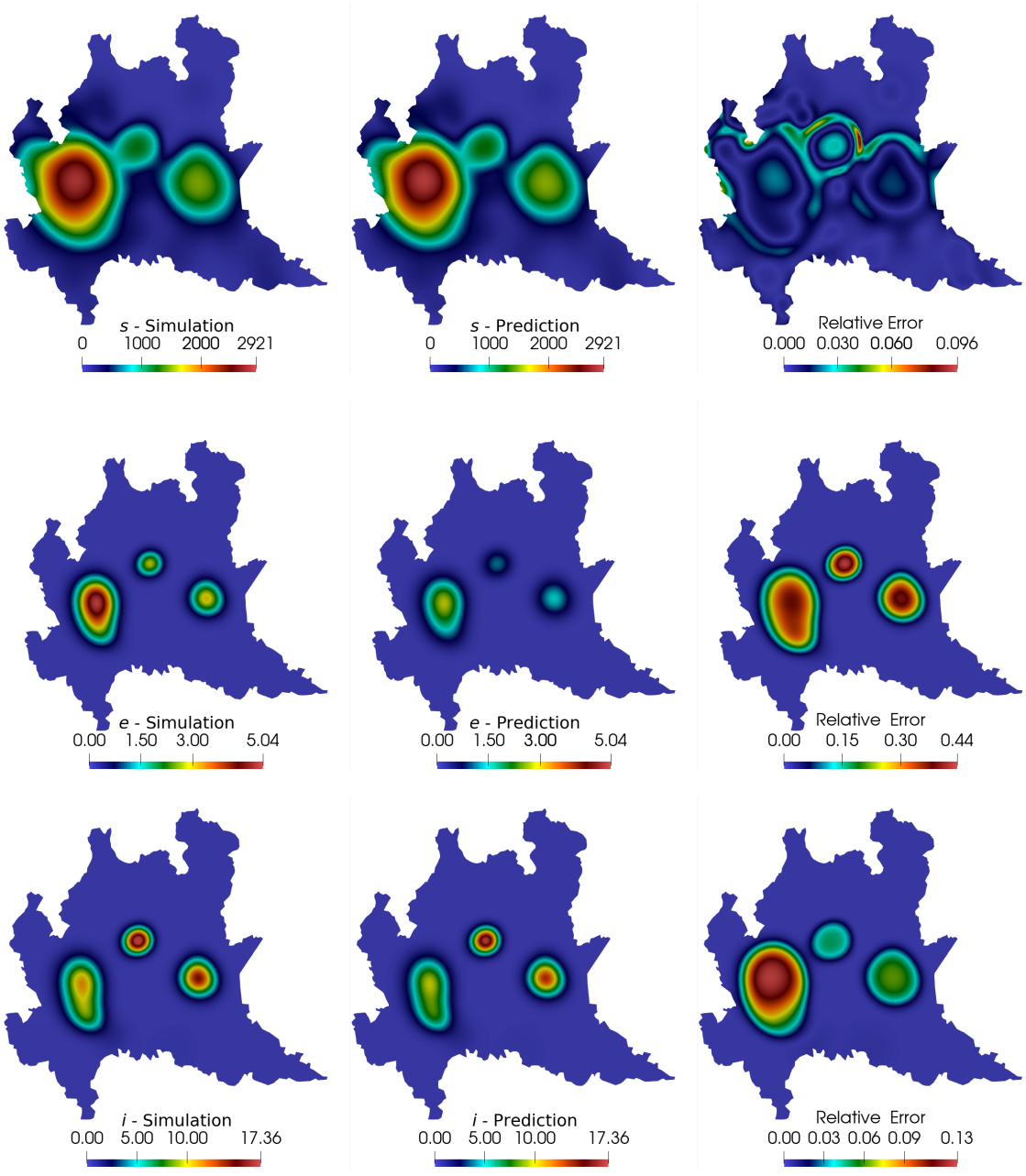


Figure 4.32: Comparison between computed and predicted solutions for the susceptible, exposed and infected compartments at $t = 60$ days.

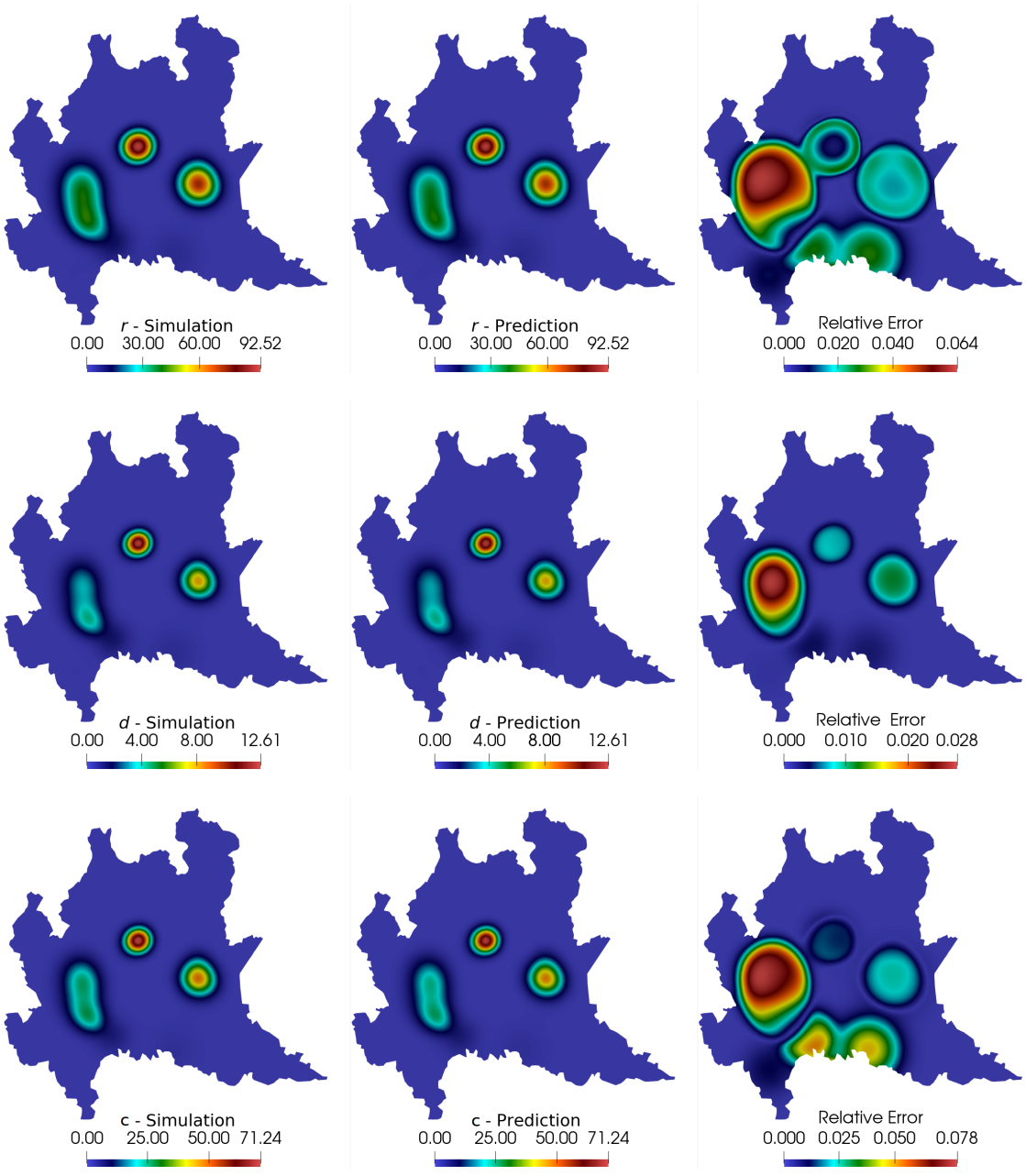


Figure 4.33: Comparison between computed and predicted solutions for the recovered, deceased and cumulative infected compartments at $t = 60$ days.

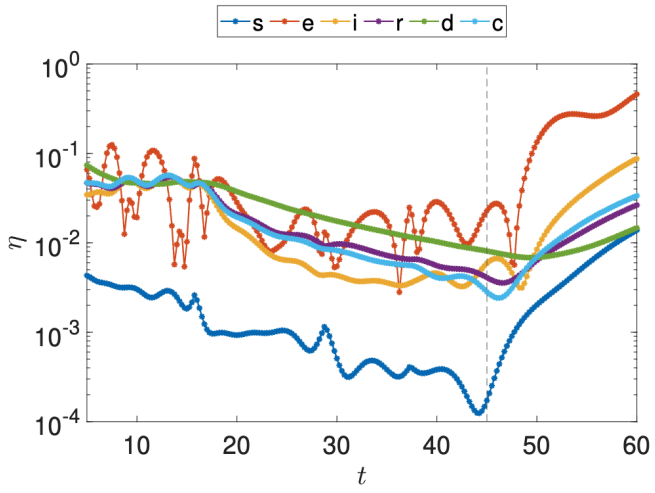


Figure 4.34: Relative error between numerical simulation snapshots and DMD reconstruction and prediction. The dashed line represents the beginning of the DMD prediction stage.

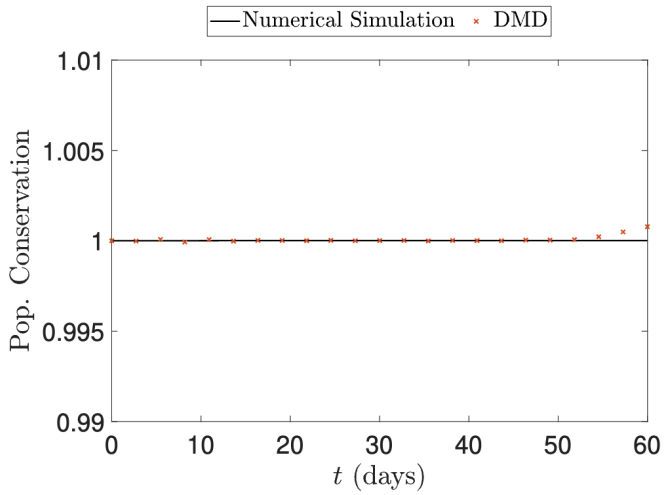


Figure 4.35: Population conservation for the simulation and DMD results.

Chapter 5

Conclusions

Scientific Machine Learning models are revolutionizing modern science and engineering. Among the different SciML models available, snapshot-based methods are increasingly being employed in the industry and academia. In this thesis, we investigate the use of Dynamic Mode Decomposition to multiple applications, contexts, and data types. Some questions triggered the development of this study:

- Regarding applications, our concern starts with using DMD on the lock-exchange configuration of a turbulent gravity flow. It is known that DMD can approximate nonlinear dynamics. Still, we exploited its use on a convection-dominated flow with multiple spatio-temporal scales in 2D and 3D, in order to verify if DMD is able to properly capture the dynamics involved in complex phenomena. Also, we assess DMD on a rising bubble problem to verify its capability of preserving the bubble's structure and geometry. Lastly, we also look forward to applying DMD - and its short-time prediction ability - to infer the spread of a contagious disease by applying the method to a continuous epidemiological model. If successful, the application of DMD in a public health example is extremely important for understanding and predicting epidemics.

- Regarding contexts, we question if the signal reconstruction from DMD is accurate not only in terms of the usual error metrics (such as Frobenius or $\mathcal{L}^2$) but if the approximation is translated into interpretable quantities of interest during DMD postprocessing. We propose investigating this issue in all applications considered in this thesis.

- Lastly, we deal with different data types. One of the main concerns when using DMD occurs when data is not readily available for application. For instance, we propose investigating using mesh projection techniques when snapshots are provided in different vector spaces. When snapshots exist in different spaces, they cannot correctly be assembled into the snapshots matrix. Another question arises from this problem: if the mesh projection works for the DMD

approximation, does any reference tailored mesh suffices for generating good approximation results? Aside from mesh projection techniques, we also consider how to properly handle snapshots from domains partitioned for parallel computing and then written in different output files for the same time step. In closing, we also investigate what would happen to singular values - and consequently, to DMD modes and eigenvalues - when data is submitted to different degrees of data compression. Furthermore, regarding data types, we also realized that in-situ visualization, common in current large-scale computations, produces images at runtime, and we explore how DMD can interact with this stream of image files. In this case, the quantities measured are pixel values.

To answer our questions, we begin exploring the results obtained by assessing DMD on density-driven gravity currents. Our numerical simulation consists of a lock-exchange configuration between two fluids with different densities. We start by modeling the problem in two dimensions and considering the hypothesis that fluid motion is dictated by the different densities between the two fluids. The `FEniCS` library is considered for generating the data. DMD extracts the modes and eigenvalues and reconstructs the solutions in a matter of seconds. We analyze the approximation by considering different numbers of basis vectors on our DMD approximation and exploring the problem's quantities of interest. We notice that the results have good accuracy in the reconstruction of the system. Moreover, the front position and the mass conservation are accurately computed for all the tested approximations. This answers our questions regarding the use of DMD on turbulent gravity currents. DMD can properly reconstruct the solutions - and quantities of interest - of a numerical simulation containing 3001 snapshots using 50 DMD modes and eigenvalues only, although it yields more accurate results for the approximations when considering 250 basis vectors. When considering $r = 250$, the relative errors approached 10^{-3} for all snapshot approximations. When approximating by $r = 50$, although the $\mathcal{L}^2$ relative error are of $\eta \approx 10^{-1}$ for all simulation time, the quantities of interest are almost identical when compared to the results obtained by the DMD approximation using a more enriched basis.

Then, we expand this problem to a situation where the numerical simulation is equipped with AMR/C strategies, generating snapshots with different dimensions and preventing the utilization of DMD. To circumvent this issue, we employ mesh projection so that all snapshots can be accurately recovered for the same mesh, enabling the construction of the snapshots matrix. We observed that this approach suffices to bring all snapshots to the same space, given that this space is such that all spatial scales from the adaptive snapshots can be accurately described. This approach can take some time when a completely non-intrusive strategy is considered, but it is negligible compared to the total computational time required for the

simulations. We validate the results by comparing the numerical simulation using a fixed and adaptive mesh, yielding accurate results. Then, we project the adaptive solutions into a reference mesh - the same grid considered for the fixed mesh simulation - and approximate the solution using DMD. Both projected solutions and approximation are compared to the fixed mesh solution, considered the "ground truth" for this problem. This numerical example is the numerical proof required to notice that the mesh projection into a tailored reference mesh is a good solution for AMR/C snapshots, given that the results for the DMD approximations constructed from the projected snapshots yield identical results when compared to the adaptive and fixed mesh solutions.

The next step is considering a more complex model for the same problem. We model a three-dimensional gravity current with more complex boundary conditions. This problem is solved using the `libMesh` library. The author did not conceive the numerical simulation code for this problem, but the co-authors made it available for this work in [68]. Given that this is a larger and more complex simulation, parallel computing is required to compute the simulation in a feasible time. Therefore, snapshots are evaluated after domain decomposition, and each domain partition is computed and stored in a separate output file. We assemble the snapshots using snapshot output files and mesh information, removing nodal duplicates (communication nodes) existing in the partitions to avoid multiple measurements for the same spatial coordinates and creating a global nodal numbering to avoid miscalculation of the DMD algorithm. As for the solutions obtained on the nonlinear systems, data is also written to disk as compressed HDF5 files to reduce disk pressure. Data compression is done by the ZFP algorithm using different strategies. We aim to compare DMD approximations from data with different compression strategies to evaluate how data compression affects the DMD approximation. We consider four datasets: an uncompressed dataset, a dataset compressed using a lossless strategy, that is when information can be completely retrieved after decompression. The other two datasets are obtained through lossy data compression, implying information loss, with different degrees of compression. Standard DMD is considered using the batch of decompressed snapshots for each compression strategy, and results are evaluated in comparison with a batch of non-compressed snapshots. We start by analyzing the quality of the data compression using the PSNR metric and evaluating the CR for each quantity. We first notice that the more aggressive the snapshots compression strategy, the less storage is required to store data. We observe that the compression strongly affects some quantities, reducing the required storage up to two orders of magnitude for the more aggressive lossy approach. By decompressing the snapshots, we observe that the latter singular values of $\mathbf{Y}_1$ differ depending on the compression strategy. As we make compression techniques more aggressive, more singular values

- starting from the last and reaching toward the first - are affected. However, this does not affect DMD significantly, given its nature of using the first r singular values. We have observed that the dominant singular values are identical for all compression strategies applied in this work, yielding similar results for all tested cases with a significant reduction in storage. We also notice a similar behavior for the DMD eigenvalues, where the more agressive compression affected a small portion of the DMD eigenvalues. Reconstructions and quantities of interest also are accurately described. By observing the metrics, we notice that the $\mathcal{L}^2$ relative error is such that $\eta \leq 10^{-2}$ for all cases in most of the simulation and $\eta \leq 2 \times 10^{-2}$ for all simulation. That is, using compressed snapshots yields accurate results compared to the results obtained from raw data with the advantage of reducing the store required in orders of magnitude and little to no increase in I/O time when decompressing snapshots for DMD approximation.

For the same model, we also observe that the numerical simulation becomes large enough to turn the Offline and Online phases diffuse, urging the necessity of analyzing results during the simulation execution instead of waiting until its completion. For this problem, Paraview Catalyst [16], an *in situ* visualization tool, is employed to generate PNG files containing images from a predefined midplane on the 3D simulation. The PNG files require significantly less storage - allowing a much faster data transfer for the DMD procedure - than the usual output files. In this case, we consider a different DMD algorithm: the streaming DMD. This strategy uses the iSVD algorithm to update the basis for each new image snapshot generated on the fly and not store past snapshots. We compute the DMD approximation of the pixel values for the images. We test the numerical approximation of DMD for image snapshots by considering the usual numerical metrics (the Frobenius and $\mathcal{L}^2$ relative error) as well as image quality metrics (the $SSIM$). We observe that DMD can accurately reconstruct pixel values, yielding a $SSIM \geq 0.998$ for the whole simulation. Considering other metrics, in terms of relative errors, the approximations did not reach the 10^{-2} threshold during the simulations.

The following example is a bubble-rising problem, where a spherical portion of a lighter fluid is immersed in a heavier fluid. Although data was kindly provided by the co-authors of [123], this problem is also equipped with AMR/C strategies, requiring the same data treatment as the 2D density-driven gravity current problem. This problem is also equipped with data compression and submitted to parallel computing, requiring all three data transformation strategies proposed in this thesis. This problem also counts with four datasets with the same data compression strategies used in the 3D lock-exchange problem. We start by decompressing the snapshot partitions and assembling the snapshots such that all snapshots are correctly mapped and decompressed. However, the snapshots are not ready for assembly on the snap-

shots matrix, requiring solution projection into the same mesh for all cases. In this example, we start by testing three different meshes with different spatial resolutions to evaluate the quality of the DMD approximation when the mesh projection is made such that some of the spatial information is lost. To project the solutions into the three mesh candidates, we consider the $\mathcal{L}^2$ projection, a standard, fast and reliable approach for projecting solutions among different finite element meshes. Using the uncompressed data, we first evaluate the simulation results for all proposed meshes after projection. We observe that the coarser mesh produces worse solutions than the other mesh candidates, affecting some quantities of interest regarding the bubble geometry, such as volume and sphericity. Relative to the bubble dynamics, such as rise velocity and center of mass, given that the original data was obtained from the same simulation with the same dynamical parameters, the quantities are preserved for all meshes. We then proceed to compute the DMD approximation using the three meshes. We notice that the relative error is kept below $\eta = 2 \times 10^{-3}$ for all meshes, with a slightly worse accuracy for the coarse mesh. The quantities of interest yield the same results observed from the values computed from the projected solutions, meaning that the DMD approximation is accurate enough not to reveal any different behavior. From this point, we consider projecting the four datasets submitted to different compression strategies to the finer candidate mesh. We have observed that, for this problem, data compression was not sufficient to affect the singular values and eigenvalues in a significant manner. After applying DMD to the decompressed snapshots projected to the tailored reference mesh, we observe similar results as seen in the 3D lock-exchange problem: slightly different latter singular values, accurate reconstructions according to the $\mathcal{L}^2$ relative error in time (for all cases, $\eta \leq 10^{-2}$ and quantities of interest such as mass conservation.

This simulation is also considered complex enough, so the Offline and Online phases should be executed simultaneously. That is, the generation of PNG files during simulation execution and the snapshot compression is employed. Results are assessed for both cases, and quantities of interest regarding bubble rising problems are also analyzed.

Moreover, we employ DMD on an epidemiological model that quantifies the spread of the COVID-19 disease in the Lombardy region of Italy. The simulation was idealized and coded by the authors of [139]. In this case, AMR/C strategies are also employed to discretize more densely populated regions further. In this example, we apply DMD for short-time predictions given the current disease-spreading scenario. Results and quantities of interest are compared with the fixed mesh simulations.

For all problems and strategies, a portion of the proposed workflow can be reproduced into a Python library developed by the author called PADMe[1]. The idea

[1] `https://github.com/gf-barros/padme`

behind the library is to create the whole online phase for the DMD algorithm, from ingesting raw simulation data to post-processing DMD approximation data. It contains the modules needed for treating parallel snapshots, projecting solutions into tailored reference meshes, and decompressing compressed snapshots. This library was not meant to compete against the existing and already established DMD libraries such as PyDMD [75] or libROM [76], although PADMe is equipped with robust and efficient DMD code and post-processing, being completely self-contained if one wants to hold the whole DMD pipeline. Actually, PADMe can be coupled easily with these libraries since the preprocessed snapshots matrix is a NumPy [141] object that could be used as input for DMD in both libraries. Appendix C describes in detail a Jupyter notebook with examples showing the library features and how to arrive at some results obtained in this paper.

That said, the questions proposed at the beginning of this chapter could be answered in this thesis. That is,

- In terms of applications:

    1. DMD can reproduce turbulent gravity current dynamics in the most important scales for both 2D and 3D cases given sufficient approximation basis vectors. The results of the approximations are accurate when compared to the numerical results.
    2. DMD approximations preserved the bubble structure and dynamics.
    3. DMD was able to approximate the solution from a continuous compartmental model used to assess the spread of an infectious disease. In fact, DMD was also used to generate temporal predictions on the continuous SEIRD model used to model the spread of COVID-19. For a training set of 46 days, DMD can predict the next 14 days with some degree of accuracy.

- In terms of context:

    1. Quantities of interest are excellent metrics regarding the problems approximated via DMD, allowing us to assess the information obtained from the approximations regarding physical properties. In all situations, the quantities of interest for DMD approximation yield accurate solutions. For the case of the 2D lock-exchange simulation, approximations with relative errors up to almost 10^{-1} provided quantities of interest practically similar to approximations with $\eta \approx 10^{-3}$.

- In terms of data types:

1. We observed that solution projection enables the use of DMD when snapshots exist in different vector spaces with no significant increase in computational time and resources compared to the nonlinear solvers in the numerical simulations. The results obtained after projection are similar to the fixed mesh approaches, suggesting that mesh projection does not affect the dynamics of the problem. However, when testing different reference meshes for projection, we observed that the quality of the approximations is affected by the geometry of the solution after projection.
2. Numerical simulations submitted to parallel computing and generating partitioned snapshots as output files can also be assembled. We need one snapshot (and all of its partitions) to create a map that will indicate the node number on each partition to a global numbering that will position that node into the snapshot vector and determine if that node requires further computation (communication node).
3. When submitted to data compression, we have observed that snapshots are affected on the last singular values. Given that DMD approximates the solution for the first r basis vectors, our results had little to no none interference from the data compression strategies.

All of these answers generated new questions that can be listed as future work for this thesis:

- For solution projection, other strategies can be used. We have seen that, for coarser meshes, the quantities of interest regarding bubble geometry are affected. More robust strategies for mesh projection [84, 150, 151] can be applied on the `Data Transformation` step of the DMD workflow to reduce the complexity of the mesh topology required for the tailored reference mesh.

- Regarding data compression strategies, we apply ZFP for compressing the snapshots in this thesis. More robust and contemporary methods regarding data compression are available in the literature and could lead to better results in terms of required storage for the snapshots.

- Combining the previous two items, we have observed that one of the possible reasons why the bubble rising problem did not yield similar results in compression rates when compared to the 3D lock-exchange case is that nodes were not properly reordered after mesh projection (step not considered on the 3D lock-exchange, that was solved using a fixed mesh). Further investigation can be done by invoking a node reordering routine to optimize the snapshot compression.

- One of the data pre-processing strategies approached in this thesis is the assembly of parallel snapshots. We assemble the solution computed (and written) in parallel into snapshots, stack the snapshots matrix and then apply the SVD factorization to the $\mathbf{Y}_1$ matrix. For larger and more complex problems, applying SVD (or even the rSVD or more efficient approaches) may be infeasible for serial computations, requiring the parallel computing of the singular values and vectors. Nonetheless, assembling parallel snapshots into "serial" snapshots for parallel computation of the SVD may be inefficient. It is essential to mention that there is no direct relation between parallel snapshots and the parallel partitions of the $\mathbf{Y}_1$ matrix for parallel computing of the singular values. The parallelism of snapshots computation is dictated by the parallelization of the nonlinear systems - which is usually governed by sparse matrices - while the parallel computation of the SVD consists of partitioning a dense matrix. An investigation into how to map the parallel snapshots into the parallel computing of the SVD algorithm could lead to a future expansion of this work.

- In this thesis, we applied the DMD approximation applying the SVD factorization on $\mathbf{Y}_1$ and the r first vectors truncation afterward. More robust (and nonlinear) factorization algorithms are presented in the literature [146, 147], or more robust SVD algorithms [152] can be investigated in the `Matrix Factorization` phase.

- We have considered the standard and the streaming DMD in this work. Another future work could be to test the ability of DMD to approximate nonlinear manifolds such as the eDMD[53], the kDMD [55], and LANDO [43].

- It is common to mix DMD with other machine learning algorithms to improve a specific part of the problem (such as noise filtering or temporal prediction). We also list as future work to assess the use of machine learning or deep learning techniques to enable parameter interpolation without intrusive behavior in finite element code, as seen, for example, in [153, 154].

- A portion of the results obtained in this thesis are obtained by coupling *in situ* visualization to generate data during simulation runtime. One using this strategy can check multiple points of view during the *in situ* visualization step to map different quantities and angles. In this case, DMD can be used for short-time prediction purposes with the available data. Checking the solution stability for the next few time steps can be helpful when considering computational steering [14] using image data [68].

- Lastly, still regarding the streaming DMD used in this thesis, more studies regarding online/streaming DMD are available for streaming data. Aside from the streaming DMD, the Online DMD [45] can be tested using PADMe. Another interesting approach is seen on [155], where the authors provided a windowed approach for DMD by splitting the snapshots matrix into smaller submatrices and evaluating DMD in each partition to be later assembled into the original snapshots matrix.

References

[1] BAKER, N., ALEXANDER, F., BREMER, T., et al. "Workshop Report on Basic Research Needs for Scientific Machine Learning: Core Technologies for Artificial Intelligence", 2 2019. doi: 10.2172/1478744. Disponível em: <https://www.osti.gov/biblio/1478744>.

[2] BRUNTON, S. L., KUTZ, J. N. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press, 2019.

[3] BRUNTON, S. L., NOACK, B. R., KOUMOUTSAKOS, P. "Machine learning for fluid mechanics", *Annual Review of Fluid Mechanics*, v. 52, pp. 477–508, 2020.

[4] KUTZ, J. N., FU, X., BRUNTON, S. L. "Multiresolution dynamic mode decomposition", *SIAM Journal on Applied Dynamical Systems*, v. 15, n. 2, pp. 713–735, 2016. ISSN: 15360040. doi: 10.1137/15M1023543.

[5] RÜDE, U., WILLCOX, K., MCINNES, L. C., et al. "Research and education in computational science and engineering", *SIAM Review*, v. 60, n. 3, pp. 707–754, 2018. ISSN: 00361445. doi: 10.1137/16M1096840.

[6] PEHERSTORFER, B., WILLCOX, K., GUNZBURGER, M. "Survey of multifidelity methods in uncertainty propagation, inference, and optimization", *SIAM Review*, v. 60, n. 3, pp. 550–591, 2018. ISSN: 00361445. doi: 10.1137/16M1082469.

[7] RAO, S. S. *The finite element method in engineering*. 2017. ISBN: 9780128117682. doi: 10.1115/1.3167179.

[8] ZIENKIEWICZ, O., TAYLOR, R., ZHU, J. Z. *The Finite Element Method: its Basis and Fundamentals: Seventh Edition*. 2013. ISBN: 9781856176330. doi: 10.1016/C2009-0-24909-9.

[9] STRIKWERDA, J. C. “Finite Difference Schemes and Partial Differential Equations.” *Mathematics of Computation*, v. 55, n. 192, pp. 869, 1990. ISSN: 00255718. doi: 10.2307/2008454.

[10] AHRENS, J., GEVECI, B., LAW, C. “ParaView: An End-User Tool for Large-Data Visualization”. In: Hansen, C. D., Johnson, C. R. (Eds.), *Visualization Handbook*, Butterworth-Heinemann, pp. 717–731, Burlington, jan. 2005. ISBN: 978-0-12-387582-2. doi: 10.1016/B978-012387582-2/50038-1. Disponível em: <https://www.sciencedirect.com/science/article/pii/B9780123875822500381>.

[11] CHILDS, H., BRUGGER, E., WHITLOCK, B., et al. *VisIt: An End-User Tool For Visualizing and Analyzing Very Large Data*. United States. Department of Energy. Office of Science, Oct 2012.

[12] MA, K. L. “In situ visualization at extreme scale: Challenges and opportunities”, *IEEE Computer Graphics and Applications*, v. 29, n. 6, pp. 14–19, 2009. ISSN: 02721716. doi: 10.1109/MCG.2009.120.

[13] FOSTER, I., AINSWORTH, M., BESSAC, J., et al. “Online data analysis and reduction: An important co-design motif for extreme-scale computers”, *The International Journal of High Performance Computing Applications*, v. 35, n. 6, pp. 617–635, 2021.

[14] SILVA, V., CAMPOS, V., GUEDES, T., et al. “DfAnalyzer: Runtime dataflow analysis tool for Computational Science and Engineering applications”, *SoftwareX*, v. 12, 2020. ISSN: 23527110. doi: 10.1016/j.softx.2020.100592.

[15] NEWBERRY, F., WETTERER-NELSON, C., EVANS, J. A., et al. “Software Tools to Enable Immersive Simulation”, *Engineering Archive*, 2021.

[16] AYACHIT, U., BAUER, A. C., BOECKEL, B., et al. “Catalyst Revised: Rethinking the ParaView in Situ Analysis and Visualization API”. In: *International Conference on High Performance Computing*, pp. 484–494. Springer, 2021.

[17] STIGLER, S. M. “The History of Statistics: The Measurement of Uncertainty before 1900”, *Technology and Culture*, v. 29, n. 2, pp. 299, apr 1988. ISSN: 0040165X. doi: 10.2307/3105539. Disponível em: <https://www.jstor.org/stable/3105539?origin=crossref>.

[18] BRADSHAW, N. A. “Florence Nightingale (1820–1910): An Unexpected Master of Data”. 2020. ISSN: 26663899.

[19] O'CONNOR, S., WAITE, M., DUCE, D., et al. "Data visualization in health care: The Florence effect". 2020. ISSN: 13652648.

[20] TUKEY, J. W. "The future of data analysis", *The annals of mathematical statistics*, v. 33, n. 1, pp. 1–67, 1962.

[21] TUKEY, J. W., OTHERS. *Exploratory data analysis*, v. 2. Reading, MA, 1977.

[22] COOLEY, J. W., TUKEY, J. W. "An Algorithm for the Machine Calculation of Complex Fourier Series", *Mathematics of Computation*, v. 19, n. 90, pp. 297, 1965. ISSN: 00255718. doi: 10.2307/2003354.

[23] SALMOIRAGHI, F., BALLARIN, F., CORSI, G., et al. "Advances in geometrical parametrization and reduced order models and methods for computational fluid dynamics problems in applied sciences and engineering: Overview and perspectives". In: *ECCOMAS Congress 2016 - Proceedings of the 7th European Congress on Computational Methods in Applied Sciences and Engineering*, v. 1, pp. 1013–1031, 2016. ISBN: 9786188284401. doi: 10.7712/100016.1867.8680.

[24] KUTZ, J. N., BRUNTON, S. L., BRUNTON, B. W., et al. *Dynamic mode decomposition: data-driven modeling of complex systems*. SIAM, 2016.

[25] TAIRA, K., HEMATI, M. S., BRUNTON, S. L., et al. "Modal Analysis of Fluid Flows: Applications and Outlook", *AIAA Journal*, v. 58, n. 3, pp. 998–1022, 2020.

[26] BADDOO, P. J., HERRMANN, B., MCKEON, B. J., et al. "Physics-informed dynamic mode decomposition", *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, v. 479, n. 2271, 2023. ISSN: 14712946. doi: 10.1098/rspa.2022.0576.

[27] SCHMID, P., SESTERHENN, J. "Dynamic Mode Decomposition of Numerical and Experimental Data", *Bulletin of the American Physical Society*, v. 53, n. Number 15, 2008.

[28] SCHMID, P. J. "Dynamic mode decomposition of numerical and experimental data", *Journal of fluid mechanics*, v. 656, pp. 5–28, 2010.

[29] TU, J. H., ROWLEY, C. W., LUCHTENBURG, D. M., et al. "On dynamic mode decomposition: Theory and applications", *Journal of Computational Dynamics*, v. 1, n. 2, pp. 391–421, 2014. ISSN: 21582505. doi: 10.3934/jcd.2014.1.391.

[30] ROWLEY, C. W., MEZI, I., BAGHERI, S., et al. "Spectral analysis of nonlinear flows", *Journal of Fluid Mechanics*, v. 641, pp. 115–127, 2009. ISSN: 14697645. doi: 10.1017/S0022112009992059.

[31] CALMET, H., PASTRANA, D., LEHMKUHL, O., et al. "Dynamic Mode Decomposition Analysis of High-Fidelity CFD Simulations of the Sinus Ventilation", *Flow, Turbulence and Combustion*, v. 105, n. 3, pp. 699–713, 2020. ISSN: 15731987. doi: 10.1007/s10494-020-00156-8.

[32] ALLA, A., BALZOTTI, C., BRIANI, M., et al. "Understanding mass transfer directions via data-driven models with application to mobile phone data", *SIAM Journal on Applied Dynamical Systems*, v. 19, n. 2, pp. 1372–1391, 2020.

[33] PROCTOR, J. L., ECKHOFF, P. A. "Discovering dynamic patterns from infectious disease data using dynamic mode decomposition", *International Health*, v. 7, n. 2, pp. 139–145, 2015. ISSN: 18763405. doi: 10.1093/inthealth/ihv009.

[34] FONZI, N., BRUNTON, S. L., FASEL, U. "Data-driven nonlinear aeroelastic models of morphing wings for control: Data-driven nonlinear aeroelastic models", *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, v. 476, n. 2239, 2020. ISSN: 14712946. doi: 10.1098/rspa.2020.0079rspa20200079.

[35] BRUNTON, B. W., JOHNSON, L. A., OJEMANN, J. G., et al. "Extracting Spatial-Temporal Coherent Patterns in Large-Scale Neural Recordings Using Dynamic Mode Decomposition", *Journal of Neuroscience Methods*, v. 258, pp. 1–15, 2016.

[36] MANN, J., KUTZ, J. N. "Dynamic Mode Decomposition for Financial Trading Strategies", *Quantitative Finance*, v. 16, n. 11, pp. 1643–1655, 2016.

[37] SASHIDHAR, D., KUTZ, J. N. "Bagging, optimized dynamic mode decomposition (BOP-DMD) for robust, stable forecasting with spatial and temporal uncertainty-quantification", *arXiv preprint arXiv:2107.10878*, 2021.

[38] PENROSE, R. "A generalized inverse for matrices", *Mathematical Proceedings of the Cambridge Philosophical Society*, v. 51, n. 3, pp. 406–413, 1955. doi: 10.1017/S0305004100030401.

[39] MEZIĆ, I. "Analysis of Fluid Flows via Spectral Properties of the Koopman Operator", *Annual Review of Fluid Mechanics*, v. 45, pp. 357–378, 2013.

[40] KOOPMAN, B. O. "Hamiltonian Systems and Transformation in Hilbert Space", *Proceedings of the National Academy of Sciences*, v. 17, n. 5, pp. 315–318, 1931.

[41] BRUNTON, S. L., KUTZ, J. N. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. 2019. ISBN: 9781108380690. doi: 10.1017/9781108380690.

[42] VINUESA, R., BRUNTON, S. L. "Emerging Trends in Machine Learning for Computational Fluid Dynamics", *Computing in Science & Engineering*, v. 24, n. 5, pp. 33–41, 2022. doi: 10.1109/MCSE.2023.3264340.

[43] BADDOO, P. J., HERRMANN, B., MCKEON, B. J., et al. "Kernel learning for robust dynamic mode decomposition: linear and nonlinear disambiguation optimization", *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, v. 478, n. 2260, 2022. ISSN: 14712946. doi: 10.1098/rspa.2021.0830.

[44] KOU, J., ZHANG, W. "An improved criterion to select dominant modes from dynamic mode decomposition", *European Journal of Mechanics, B/Fluids*, v. 62, pp. 109–129, 2017. ISSN: 09977546. doi: 10.1016/j.euromechflu.2016.11.015.

[45] ZHANG, H., ROWLEY, C. W., DEEM, E. A., et al. "Online dynamic mode decomposition for time-varying systems", *SIAM Journal on Applied Dynamical Systems*, v. 18, n. 3, pp. 1586–1609, 2019. ISSN: 15360040. doi: 10.1137/18M1192329.

[46] HÉAS, P., HERZET, C. "Low-Rank Dynamic Mode Decomposition: An Exact and Tractable Solution", *Journal of Nonlinear Science*, v. 32, n. 1, 2022. ISSN: 14321467. doi: 10.1007/s00332-021-09770-w.

[47] CHEN, K. K., TU, J. H., ROWLEY, C. W. "Variants of dynamic mode decomposition: Boundary condition, Koopman, and fourier analyses", *Journal of Nonlinear Science*, v. 22, n. 6, pp. 887–915, 2012. ISSN: 09388974. doi: 10.1007/s00332-012-9130-9.

[48] UL HAQ, I., IWATA, T., KAWAHARA, Y. "Dynamic mode decomposition via convolutional autoencoders for dynamics modeling in videos", *Computer Vision and Image Understanding*, v. 216, 2022. ISSN: 1090235X. doi: 10.1016/j.cviu.2021.103355.

[49] "Search for "Dynamic Mode Decomposition" using Web of Science. Clarivate Analytics engine. Access on 17/03/2021". 2021. Disponível em: <https://www.webofknowledge.com>.

[50] SPURIN, C., ARMSTRONG, R. T., MCCLURE, J., et al. "Dynamic mode decomposition for analysing multi-phase flow in porous media", *Advances in Water Resources*, v. 175, pp. 104423, 2023. ISSN: 0309-1708. doi: https://doi.org/10.1016/j.advwatres.2023.104423. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0309170823000581>.

[51] LIEW, J., GÖÇMEN, T., LIO, W. H., et al. "Streaming dynamic mode decomposition for short-term forecasting in wind farms", *Wind Energy*, v. 25, n. 4, pp. 719–734, 2022. ISSN: 10991824. doi: 10.1002/we.2694.

[52] PROCTOR, J. L., BRUNTON, S. L., KUTZ, J. N. "Dynamic mode decomposition with control", *SIAM Journal on Applied Dynamical Systems*, v. 15, n. 1, pp. 142–161, 2016. ISSN: 15360040. doi: 10.1137/15M1013857.

[53] WILLIAMS, M. O., KEVREKIDIS, I. G., ROWLEY, C. W. "A Data–Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition", *Journal of Nonlinear Science*, v. 25, n. 6, pp. 1307–1346, 2015. ISSN: 14321467. doi: 10.1007/s00332-015-9258-5.

[54] LI, Q., DIETRICH, F., BOLLT, E. M., et al. "Extended Dynamic Mode Decomposition with Dictionary Learning: A Data-Driven Adaptive Spectral Decomposition of the Koopman Operator", *Chaos*, v. 27, n. 10, 2017.

[55] WILLIAMS, M. O., ROWLEY, C. W., KEVREKIDIS, I. G. "A kernel-based method for data-driven Koopman spectral analysis", *Journal of Computational Dynamics*, v. 2, n. 2, pp. 247–265, 2015. ISSN: 21582505. doi: 10.3934/jcd.2015005.

[56] NOÉ, F., NÜSKE, F. "A Variational Approach to Modeling Slow Processes in Stochastic Dynamical Systems", *Multiscale Modeling and Simulation*, v. 11, n. 2, pp. 635–655, 2013.

[57] NÜSKE, F., SCHNEIDER, R., VITALINI, F., et al. "Variational Tensor Approach for Approximating the Rare-Event Kinetics of Macromolecular Systems", *Journal of Chemical Physics*, v. 144, n. 5, 2016.

[58] WEHMEYER, C., NOÉ, F. "Time-lagged Autoencoders: Deep Learning of Slow Collective Variables for Molecular Kinetics", *Journal of Chemical Physics*, v. 148, n. 24, 2018.

[59] ASKHAM, T., KUTZ, J. N. "Variable projection methods for an optimized dynamic mode decomposition", *SIAM Journal on Applied Dynamical Systems*, v. 17, n. 1, pp. 380–416, 2018. ISSN: 15360040. doi: 10.1137/M1124176.

[60] HEMATI, M. S., ROWLEY, C. W., DEEM, E. A., et al. "De-biasing the dynamic mode decomposition for applied Koopman spectral analysis of noisy datasets", *Theoretical and Computational Fluid Dynamics*, v. 31, n. 4, pp. 349–368, 2017. ISSN: 14322250. doi: 10.1007/s00162-017-0432-2.

[61] DAWSON, S. T., HEMATI, M. S., WILLIAMS, M. O., et al. "Characterizing and correcting for the effect of sensor noise in the dynamic mode decomposition", *Experiments in Fluids*, v. 57, n. 3, 2016. ISSN: 07234864. doi: 10.1007/s00348-016-2127-7.

[62] HEATH, M. T. *Scientific Computing: An Introductory Survey, Revised Second Edition*. Philadelphia, PA, USA, SIAM-Society for Industrial and Applied Mathematics, 2018. ISBN: 1611975573.

[63] ECKART, C., YOUNG, G. "The Approximation of One Matrix by Another of Lower Rank", *Psychometrika*, v. 1, n. 3, pp. 211–218, 1936.

[64] TAIRA, K., BRUNTON, S. L., DAWSON, S. T., et al. "Modal analysis of fluid flows: An overview", *AIAA Journal*, v. 55, n. 12, pp. 4013–4041, 2017.

[65] SIROVICH, L. "Turbulence and the Dynamics of Coherent Structures I, II and III." *Q. Appl. Math.*, v. 45, n. 3, pp. 561–590, 1987.

[66] HALKO, N., MARTINSSON, P.-G., TROPP, J. A. "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions", *SIAM Review*, v. 53, n. 2, pp. 217–288, 2011. ISSN: 00361445. doi: 10.1137/090771806.

[67] HEMATI, M. S., WILLIAMS, M. O., ROWLEY, C. W. "Dynamic mode decomposition for large and streaming datasets", *Physics of Fluids*, v. 26, n. 11, 2014. ISSN: 10897666. doi: 10.1063/1.4901016.

[68] BARROS, G. F., GRAVE, M., CAMATA, J. J., et al. "Enhancing dynamic mode decomposition workflow with in situ visualization and data compression", *Engineering with Computers*, 2023. ISSN: 14355663. doi: 10.1007/s00366-023-01805-y.

[69] MATSUMOTO, D., INDINGER, T. "On-the-fly algorithm for Dynamic Mode Decomposition using Incremental Singular Value Decomposition and Total Least Squares". 2017.

[70] BRAND, M. "Incremental singular value decomposition of uncertain data with missing values". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, v. 2350, pp. 707–720, 2002. ISBN: 3540437452. doi: 10.1007/3-540-47969-4_47.

[71] OXBERRY, G. M., KOSTOVA-VASSILEVSKA, T., ARRIGHI, W., et al. "Limited-memory adaptive snapshot selection for proper orthogonal decomposition", *International Journal for Numerical Methods in Engineering*, v. 109, n. 2, pp. 198–217, 2017. ISSN: 10970207. doi: 10.1002/nme.5283.

[72] CHOI, Y., BROWN, P., ARRIGHI, W., et al. "Space–time reduced order model for large-scale linear dynamical systems with application to boltzmann transport problems", *Journal of Computational Physics*, v. 424, pp. 109845, 2021.

[73] DAVIDSON, S. B., BOULAKIA, S. C., EYAL, A., et al. "Provenance in Scientific Workflow Systems." *IEEE Data Eng. Bull.*, v. 30, 2007.

[74] DEELMAN, E., MANDAL, A., JIANG, M., et al. "The role of machine learning in scientific workflows", *International Journal of High Performance Computing Applications*, v. 33, n. 6, pp. 1128–1139, 2019. ISSN: 17412846. doi: 10.1177/1094342019852127.

[75] DEMO, N., TEZZELE, M., ROZZA, G. "PyDMD: Python Dynamic Mode Decomposition", *The Journal of Open Source Software*, v. 3, n. 22, pp. 530, 2018. doi: https://doi.org/10.21105/joss.00530.

[76] CHOI, Y., ARRIGHI, W. J., COPELAND, D. M., et al. "libROM". out. 2019. Disponível em: <`https://github.com/LLNL/libROM`>.

[77] KIM, Y., KANG, S. H., CHO, H., et al. "Improved Nonlinear Analysis of a Propeller Blade Based on Hyper-Reduction", *AIAA Journal*, v. 60, n. 3, pp. 1909–1922, 2022. ISSN: 1533385X. doi: 10.2514/1.J060742.

[78] PERKEL, J. M. "Workflow systems turn raw data into scientific knowledge", *Nature*, v. 573, n. 7772, pp. 149–150, 2019. ISSN: 14764687. doi: 10.1038/d41586-019-02619-z.

[79] OLSON, R. S., BARTLEY, N., URBANOWICZ, R. J., et al. "Evaluation of a tree-based pipeline optimization tool for automating data science". In: *GECCO 2016 - Proceedings of the 2016 Genetic and Evolutionary Computation Conference*, pp. 485–492, 2016. ISBN: 9781450342063. doi: 10.1145/2908812.2908918.

[80] THE HDF GROUP. "Hierarchical data format version 5". 2020. Disponível em: <`https://www.hdfgroup.org/solutions/hdf5/`>.

[81] AYACHIT, U., BAUER, A., GEVECI, B., et al. "Paraview catalyst: Enabling in situ data analysis and visualization". In: *Proceedings of the First Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*, pp. 25–29, 2015.

[82] ALNÆS, M. S., BLECHTA, J., HAKE, J., et al. "The FEniCS Project Version 1.5", *Archive of Numerical Software*, v. 3, n. 100, 2015. doi: 10.11588/ans.2015.100.20553.

[83] KIRK, B. S., PETERSON, J. W., STOGNER, R. H., et al. "libMesh: a C++ library for parallel adaptive mesh refinement/coarsening simulations", *Journal Engineering with Computers*, v. 22, n. 3, pp. 237–254, 2006.

[84] PONT, A., CODINA, R., BAIGES, J. "Interpolation with restrictions between finite element meshes for flow problems in an ALE setting", *International Journal for Numerical Methods in Engineering*, v. 110, n. 13, pp. 1203–1226, 2017. doi: https://doi.org/10.1002/nme.5444. Disponível em: <`https://onlinelibrary.wiley.com/doi/abs/10.1002/nme.5444`>.

[85] GEUZAINE, C., REMACLE, J.-F. "Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities", *International Journal for Numerical Methods in Engineering*, v. 79, n. 11, pp. 1309–1331, 2009. doi: https://doi.org/10.1002/nme.2579. Disponível em: <`https://onlinelibrary.wiley.com/doi/abs/10.1002/nme.2579`>.

[86] BARROS, G. F., GRAVE, M., VIGUERIE, A., et al. "Dynamic mode decomposition in adaptive mesh refinement and coarsening simulations", *Engineering with Computers*, v. 38, n. 5, pp. 4241–4268, 2022. ISSN: 14355663. doi: 10.1007/s00366-021-01485-6.

[87] ULLMANN, S., ROTKVIC, M., LANG, J. "POD-Galerkin reduced-order modeling with adaptive finite element snapshots", *Journal of Computational Physics*, v. 325, pp. 244–258, 2016.

[88] DIFFENDERFER, J., FOX, A. L., HITTINGER, J. A., et al. "Error analysis of ZFP compression for floating-point data", *SIAM Journal on Scientific Computing*, v. 41, n. 3, pp. A1867–A1898, 2019. ISSN: 10957197. doi: 10.1137/18M1168832.

[89] AINSWORTH, M., TUGLUK, O., WHITNEY, B., et al. "Multilevel techniques for compression and reduction of scientific data—the univariate case", *Computing and Visualization in Science*, v. 19, n. 5, pp. 65–76, 2018.

[90] AINSWORTH, M., TUGLUK, O., WHITNEY, B., et al. "Multilevel techniques for compression and reduction of scientific data—The multivariate case", *SIAM Journal on Scientific Computing*, v. 41, n. 2, pp. A1278–A1303, 2019.

[91] AINSWORTH, M., TUGLUK, O., WHITNEY, B., et al. "Multilevel techniques for compression and reduction of scientific data-quantitative control of accuracy in derived quantities", *SIAM Journal on Scientific Computing*, v. 41, n. 4, pp. A2146–A2171, 2019.

[92] AINSWORTH, M., TUGLUK, O., WHITNEY, B., et al. "Multilevel techniques for compression and reduction of scientific data—The unstructured case", *SIAM Journal on Scientific Computing*, v. 42, n. 2, pp. A1402–A1427, 2020.

[93] JIN, S., TAO, D., TANG, H., et al. "Accelerating Parallel Write via Deeply Integrating Predictive Lossy Compression with HDF5", *arXiv preprint arXiv:2206.14761*, 2022.

[94] LIANG, X., WHITNEY, B., CHEN, J., et al. "MGARD+: Optimizing Multilevel Methods for Error-Bounded Scientific Data Reduction", *IEEE Transactions on Computers*, v. 71, n. 7, pp. 1522–1536, 2022. doi: 10.1109/TC.2021.3092201.

[95] BRATKO, A., FILIPIČ, B., CORMACK, G. V., et al. "Spam filtering using statistical data compression models", *Journal of Machine Learning Research*, v. 7, pp. 2673–2698, 2006. ISSN: 15324435.

[96] LITTLESTONE, N., WARMUTH, M. "Relating data compression and learnability", *Technical report, Department of Computer and Information Sciences, Santa Cruz, CA*, 1986.

[97] SCULLEY, D., BRODLEY, C. E. "Compression and machine learning: A new perspective on feature space vectors". In: *Data Compression Conference Proceedings*, pp. 332–341, 2006. doi: 10.1109/DCC.2006.13.

[98] GHAHRAMANI, Z. "Probabilistic machine learning and artificial intelligence". 2015. ISSN: 14764687.

[99] GAVISH, M., DONOHO, D. L. "The Optimal Hard Threshold for Singular Values is $4/\sqrt{3}$", *IEEE Transactions on Information Theory*, v. 60, n. 8, pp. 5040–5053, 2014.

[100] DONOHO, D. L. "De-Noising by Soft-Thresholding", *IEEE Transactions on Information Theory*, v. 41, n. 3, pp. 613–627, 1995.

[101] WANG, Z., BOVIK, A. C. "Mean Squared Error : Love It or Leave It ?" *IEEE Signal Processing Magazine*, v. 26, n. 1, pp. 98–117, 2009.

[102] WANG, Z., BOVIK, A. C., SHEIKH, H. R., et al. "Image quality assessment: From error visibility to structural similarity", *IEEE Transactions on Image Processing*, v. 13, n. 4, pp. 600–612, 2004. ISSN: 10577149. doi: 10.1109/TIP.2003.819861.

[103] AVANAKI, A. N. "Exact global histogram specification optimized for structural similarity", *Optical Review*, v. 16, n. 6, pp. 613–621, 2009. ISSN: 13406000. doi: 10.1007/s10043-009-0119-z.

[104] VAN DER WALT, S., SCHÖNBERGER, J. L., NUNEZ-IGLESIAS, J., et al. "scikit-image: image processing in Python", *PeerJ*, v. 2, pp. e453, 6 2014. ISSN: 2167-8359. doi: 10.7717/peerj.453. Disponível em: <`https://doi.org/10.7717/peerj.453`>.

[105] SIMPSON, J. E. "Gravity currents in the laboratory, atmosphere, and ocean." *in: Annual Review of Fluid Mechanics*, v. 14 , M. Van Dyke; J.V. Wehausen; J.L. Lumley, 1982. ISSN: 00664189.

[106] CANO-LOZANO, J. C., BOLAÑOS-JIMÉNEZ, R., GUTIÉRREZ-MONTES, C., et al. "The use of Volume of Fluid technique to analyze multiphase flows: Specific case of bubble rising in still liquids", *Applied Mathematical Modelling*, v. 39, n. 12, pp. 3290–3305, 2015. ISSN: 0307904X. doi: 10.1016/j.apm.2014.11.034.

[107] NECKER, F., HÄRTEL, C., KLEISER, L., et al. "High-resolution simulations of particle-driven gravity currents", *International Journal of Multiphase*

Flow, v. 28, n. 2, pp. 279–300, 2002. ISSN: 03019322. doi: 10.1016/S0301-9322(01)00065-9.

[108] HUPPERT, H. E. "Gravity currents: A personal perspective", *Journal of Fluid Mechanics*, v. 554, pp. 299–322, 2006. ISSN: 14697645. doi: 10.1017/S002211200600930X.

[109] DAI, A., HUANG, Y. L. "Boussinesq and non-Boussinesq gravity currents propagating on unbounded uniform slopes in the deceleration phase", *Journal of Fluid Mechanics*, v. 917, 2021. ISSN: 14697645. doi: 10.1017/jfm.2021.300.

[110] BIRMAN, V. K., MEIBURG, E. "High-resolution simulations of gravity currents", *Journal of the Brazilian Society of Mechanical Sciences and Engineering*, v. 28, n. 2, pp. 169–173, 2006. ISSN: 18063691. doi: 10.1590/S1678-58782006000200006.

[111] HUGHES, T. J. R., SCOVAZZI, G., FRANCA, L. P. "Multiscale and Stabilized Methods". In: *Encyclopedia of Computational Mechanics Second Edition*, pp. 1–64, 2017. doi: 10.1002/9781119176817.ecm2051.

[112] RASTHOFER, U., GRAVEMEIER, V. "Recent Developments in Variational Multiscale Methods for Large-Eddy Simulation of Turbulent Flow", *Archives of Computational Methods in Engineering*, pp. 1–44, 2017.

[113] AHMED, N., REBOLLO, T. C., JOHN, V., et al. "A review of variational multiscale methods for the simulation of turbulent incompressible flows", *Archives of Computational Methods in Engineering*, v. 24, n. 1, pp. 115–164, 2017.

[114] CODINA, R., BADIA, S., BAIGES, J., et al. "Variational multiscale methods in computational fluid dynamics", *Encyclopedia of Computational Mechanics Second Edition*, pp. 1–28, 2018.

[115] BARROS, G. F., CÔRTES, A. M. A., COUTINHO, A. L. "Dynamic mode decomposition for density-driven gravity current simulations", *CILAMCE 2020 - Proceedings of the XLI Ibero-Latin-American Congress on Computational Methods in Engineering*, 2020.

[116] TAO, D., DI, S., LIANG, X., et al. "Fixed-PSNR Lossy Compression for Scientific Data". In: *Proceedings - IEEE International Conference on Cluster Computing, ICCC*, v. 2018-September, pp. 314–318, 2018. ISBN: 9781538683194. doi: 10.1109/CLUSTER.2018.00048.

[117] KORHONEN, J., YOU, J. "Peak signal-to-noise ratio revisited: Is simple beautiful?" In: *2012 4th International Workshop on Quality of Multimedia Experience, QoMEX 2012*, pp. 37–38, 2012. ISBN: 9781467307253. doi: 10.1109/QoMEX.2012.6263880.

[118] BULL, D. R., ZHANG, F. "Chapter 4 - Digital picture formats and representations". In: Bull, D. R., Zhang, F. (Eds.), *Intelligent Image and Video Compression (Second Edition)*, second edition ed., Academic Press, pp. 107–142, Oxford, 2021. ISBN: 978-0-12-820353-8. doi: https://doi.org/10.1016/B978-0-12-820353-8.00013-X. Disponível em: <`https://www.sciencedirect.com/science/article/pii/B978012820353800013X`>.

[119] ZHAO, K., DI, S., LIAN, X., et al. "SDRBench: Scientific Data Reduction Benchmark for Lossy Compressors". In: *2020 IEEE International Conference on Big Data (Big Data)*, pp. 2716–2724, 2020. doi: 10.1109/BigData50022.2020.9378449.

[120] HUA, J., LOU, J. "Numerical simulation of bubble rising in viscous liquid", *Journal of Computational Physics*, v. 222, n. 2, pp. 769–795, 2007. ISSN: 10902716. doi: 10.1016/j.jcp.2006.08.008.

[121] COUPEZ, T. "Convection of local level set function for moving surfaces and interfaces in forming flow". In: *AIP Conference Proceedings*, pp. 61–66. AIP, 2007.

[122] VILLE, L., SILVA, L., COUPEZ, T. "Convected level set method for the numerical simulation of fluid buckling", *International Journal for numerical methods in fluids*, v. 66, n. 3, pp. 324–344, 2011.

[123] GRAVE, M., CAMATA, J. J., COUTINHO, A. L. "A new convected level-set method for gas bubble dynamics", *Computers & Fluids*, v. 209, pp. 104667, 2020.

[124] BRACKBILL, J. U., KOTHE, D. B., ZEMACH, C. "A continuum method for modeling surface tension", *Journal of Computational Physics*, v. 100, n. 2, pp. 335–354, 1992.

[125] YANG, H. M. *Epidemiologia matemática: Estudos dos efeitos da vacinação em doenças de transmissão direta*. Editora da UNICAMP, 2001.

[126] ERICKSON, R. A., PRESLEY, S. M., ALLEN, L. J., et al. "A dengue model with a dynamic Aedes albopictus vector population", *Ecological Modelling*, v. 221, n. 24, pp. 2899–2908, 2010.

[127] DANTAS, E., TOSIN, M., CUNHA JR, A. "Calibration of a SEIR–SEI epidemic model to describe the Zika virus outbreak in Brazil", *Applied Mathematics and Computation*, v. 338, pp. 249–259, 2018.

[128] MUKANDAVIRE, Z., DAS, P., CHIYAKA, C., et al. "Global analysis of an HIV/AIDS epidemic model", *World Journal of Modelling and Simulation*, v. 6, n. 3, pp. 231–240, 2010.

[129] DYE, C., GAY, N. "Modeling the SARS epidemic", *Science*, v. 300, n. 5627, pp. 1884–1885, 2003.

[130] MIDEKISA, A., SENAY, G., HENEBRY, G. M., et al. "Remote sensing-based time series models for malaria early warning in the highlands of Ethiopia", *Malaria Journal*, v. 11, n. 1, pp. 165, 2012.

[131] LEKONE, P. E., FINKENSTÄDT, B. F. "Statistical inference in a stochastic epidemic SEIR model with control intervention: Ebola as a case study", *Biometrics*, v. 62, n. 4, pp. 1170–1177, 2006.

[132] VIGUERIE, A., VENEZIANI, A., LORENZO, G., et al. "Diffusion–reaction compartmental models formulated in a continuum mechanics framework: application to COVID-19, mathematical analysis, and numerical study", *Computational Mechanics*, v. 66, pp. 1131–1152, 2020.

[133] VIGUERIE, A., LORENZO, G., AURICCHIO, F., et al. "Simulating the spread of COVID-19 via spatially-resolved susceptible-exposed-infected-recovered-deceased (SEIRD) model with heterogeneous diffusion", *Applied Mathematics Letters*, v. 111, pp. 106617, 2021.

[134] ARINO, J., PORTET, S. "A simple model for COVID-19", *Infectious Disease Modelling*, v. 5, pp. 309–315, 2020.

[135] GIORDANO, G., BLANCHINI, F., BRUNO, R., et al. "Modelling the COVID-19 epidemic and implementation of population-wide interventions in Italy", *Nature Medicine*, pp. 1–6, 2020.

[136] CARCIONE, J. M., SANTOS, J. E., BAGAINI, C., et al. "A simulation of a COVID-19 epidemic based on a deterministic SEIR model", *Frontiers in Public Health*, v. 8, pp. 230, 2020.

[137] VOLPATTO, D. T., RESENDE, A. C. M., ANJOS, L., et al. "Spreading of COVID-19 in Brazil: Impacts and uncertainties in social distancing strategies", *medRxiv*, 2020.

[138] BRAUER, F., CASTILLO-CHAVEZ, C., FENG, Z. *Mathematical models in epidemiology*. Springer, 2019.

[139] GRAVE, M., COUTINHO, A. L. G. A. "Adaptive Mesh Refinement and Coarsening for Diffusion-Reaction Epidemiological Models", *Computational Mechanics*, v. 67, pp. 1177–1199, 2021.

[140] VIGUERIE, A., LORENZO, G., AURICCHIO, F., et al. "Simulating the spread of COVID-19 via a spatially-resolved susceptible–exposed–infected–recovered–deceased (SEIRD) model with heterogeneous diffusion", *Applied Mathematics Letters*, v. 111, 2021. ISSN: 18735452. doi: 10.1016/j.aml.2020.106617.

[141] HARRIS, C. R., MILLMAN, K. J., VAN DER WALT, S. J., et al. "Array programming with NumPy", *Nature*, v. 585, n. 7825, pp. 357–362, set. 2020. doi: 10.1038/s41586-020-2649-2. Disponível em: <https://doi.org/10.1038/s41586-020-2649-2>.

[142] ANDERSON, E., BAI, Z., BISCHOF, C., et al. *LAPACK Users' Guide*. Third ed. , Society for Industrial and Applied Mathematics, 1999.

[143] KRAKE, T., REINHARDT, S., HLAWATSCH, M., et al. "Visualization and selection of Dynamic Mode Decomposition components for unsteady flow", *Visual Informatics*, v. 5, n. 3, pp. 15–27, 2021. ISSN: 2468-502X. doi: https://doi.org/10.1016/j.visinf.2021.06.003. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2468502X21000309>.

[144] FRIES, W. D., HE, X., CHOI, Y. "LaSDI: Parametric Latent Space Dynamics Identification", *Computer Methods in Applied Mechanics and Engineering*, v. 399, pp. 115436, 2022. ISSN: 0045-7825. doi: https://doi.org/10.1016/j.cma.2022.115436. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0045782522004807>.

[145] HE, X., CHOI, Y., FRIES, W. D., et al. "gLaSDI: Parametric Physics-informed Greedy Latent Space Dynamics Identification", *arXiv preprint arXiv:2204.12005*, 2022. doi: 10.48550/ARXIV.2204.12005.

[146] KADEETHUM, T., BALLARIN, F., CHOI, Y., et al. "Non-intrusive reduced order modeling of natural convection in porous media using convolutional autoencoders: Comparison with linear subspace techniques", *Advances in Water Resources*, v. 160, pp. 104098, 2022. ISSN: 0309-1708. doi: https://doi.org/10.1016/j.advwatres.2021.104098.

Disponível em: <https://www.sciencedirect.com/science/article/pii/S0309170821002499>.

[147] KADEETHUM, T., BALLARIN, F., O'MALLEY, D., et al. "Reduced order modeling for flow and transport problems with Barlow Twins self-supervised learning", *Scientific Reports*, v. 12, n. 1, pp. 20654, 2022.

[148] NILSSON, J., AKENINE-MÖLLER, T. "Understanding SSIM". 2020. Disponível em: <https://arxiv.org/abs/2006.13846>.

[149] GRAVE, M., VIGUERIE, A., BARROS, G. F., et al. "Assessing the Spatio-temporal Spread of COVID-19 via Compartmental Models with Diffusion in Italy, USA, and Brazil", *Archives of Computational Methods in Engineering*, v. 28, n. 6, pp. 4205–4223, 2021. ISSN: 18861784. doi: 10.1007/s11831-021-09627-1.

[150] CAREY, G. F., BICKEN, G., CAREY, V., et al. "Locally constrained projections on grids", *International Journal for Numerical Methods in Engineering*, v. 50, n. 3, pp. 549–577, 2001. doi: https://doi.org/10.1002/1097-0207(20010130)50:3<549::AID-NME35>3.0.CO;2-S. Disponível em: <https://onlinelibrary.wiley.com/doi/abs/10.1002/1097-0207%2820010130%2950%3A3%3C549%3A%3AAID-NME35%3E3.0.CO%3B2-S>.

[151] FARRELL, P. E., MADDISON, J. R. "Conservative interpolation between volume meshes by local Galerkin projection", *Computer Methods in Applied Mechanics and Engineering*, v. 200, n. 1-4, pp. 89–100, jan 2011. ISSN: 00457825. doi: 10.1016/j.cma.2010.07.015.

[152] LIU, L., HAWKINS, D. M., GHOSH, S., et al. "Robust singular value decomposition analysis of microarray data", *Proceedings of the National Academy of Sciences*, v. 100, n. 23, pp. 13167–13172, 2003.

[153] HUHN, Q. A., TANO, M. E., RAGUSA, J. C., et al. "Parametric dynamic mode decomposition for reduced order modeling", *Journal of Computational Physics*, v. 475, pp. 111852, 2023. ISSN: 0021-9991. doi: https://doi.org/10.1016/j.jcp.2022.111852. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0021999122009159>.

[154] HESS, M. W., QUAINI, A., ROZZA, G. "A data-driven surrogate modeling approach for time-dependent incompressible Navier-Stokes equations with dynamic mode decomposition and manifold interpolation", *Advances in Computational Mathematics*, v. 49, n. 2, pp. 22, 2023. doi:

10.1007/s10444-023-10016-4. Disponível em: <https://doi.org/10.1007/s10444-023-10016-4>.

[155] ALLA, A., MONTI, A., SGURA, I. "Piecewise DMD for oscillatory and Turing spatio-temporal dynamics", *ArXiv*, v. abs/2303.06512, 2023.

[156] ALLA, A., KUTZ, J. N. "Nonlinear model order reduction via dynamic mode decomposition", *SIAM Journal on Scientific Computing*, v. 39, n. 5, pp. B778–B796, 2017.

[157] CARLBERG, K., BOU-MOSLEH, C., FARHAT, C. "Efficient non-linear model reduction via a least-squares Petrov-Galerkin projection and compressive tensor approximations", *International Journal for Numerical Methods in Engineering*, v. 86, n. 2, pp. 155–181, 2011. ISSN: 00295981. doi: 10.1002/nme.3050.

[158] BROOKS, A. N., HUGHES, T. J. "Streamline upwind/Petrov-Galerkin formulations for convection dominated flows with particular emphasis on the incompressible Navier-Stokes equations", *Computer methods in applied mechanics and engineering*, v. 32, n. 1-3, pp. 199–259, 1982.

[159] DONEA, J., HUERTA, A. *Finite Element Methods for Flow Problems*. John Wiley & Sons, Ltd., 2003.

[160] FORTI, D., DEDÉ, L. "Semi-implicit BDF time discretization of the Navier–Stokes equations with VMS-LES modeling in a High Performance Computing framework", *Computer and Fluids*, v. 15, n. 115, pp. 168–182, 2015. ISSN: 0045-7930. doi: https://doi.org/10.1016/j.compfluid.2015.05.011.

[161] ZIENKIEWICZ, O. C., TAYLOR, R. L., ZHU, J. Z. *The finite element method: its basis and fundamentals*. Elsevier, 2005.

[162] TEZDUYAR, T. "Stabilized Finite Element Formulations for Incompressible Flow Computations††This research was sponsored by NASA-Johnson Space Center (under grant NAG 9-449), NSF (under grant MSM-8796352), U.S. Army (under contract DAAL03-89-C-0038), and the University of Paris VI." v. 28, *Advances in Applied Mechanics*, Elsevier, pp. 1–44, 1991. doi: https://doi.org/10.1016/S0065-2156(08)70153-4. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0065215608701534>.

[163] JOSHI, V., JAIMAN, R. K. "A positivity preserving variational method for multi-dimensional convection–diffusion–reaction equation", *Journal of Computational Physics*, v. 339, pp. 247–284, 2017.

[164] BROOKS, A. N., HUGHES, T. J. "Streamline upwind/Petrov-Galerkin formulations for convection dominated flows with particular emphasis on the incompressible Navier-Stokes equations", *Computer Methods in Applied Mechanics and Engineering*, v. 32, n. 1-3, pp. 199–259, 1982.

[165] HUGHES, T. J. R., FRANCA, L. P., HULBERT, G. M. "A new finite element formulation for computational fluid dynamics: VIII. The galerkin/least-squares method for advective-diffusive equations", *Computer Methods in Applied Mechanics and Engineering*, v. 73, n. 2, pp. 173–189, 1989.

[166] HUGHES, T. J. R. "Multiscale phenomena: Green's functions, the Dirichlet-to-Neumann formulation, subgrid scale models, bubbles and the origins of stabilized methods", *Computer Methods in Applied Mechanics and Engineering*, v. 127, n. 1-4, pp. 387–401, 1995.

[167] BAZILEVS, Y., TAKIZAWA, K., TEZDUYAR, T. E. *Computational fluid-structure interaction: methods and applications*. John Wiley & Sons, 2013.

[168] CODINA, R., PRINCIPE, J., ÁVILA, M. "Finite element approximation of turbulent thermally coupled incompressible flows with numerical sub-grid scale modelling", *International Journal of Numerical Methods for Heat and Fluid Flow*, v. 20, n. 5, pp. 492–516, 2010. ISSN: 09615539. doi: 10.1108/09615531011048213.

[169] BAZILEVS, Y., TAKIZAWA, K., TEZDUYAR, T. E. *Computational Fluid-Structure Interaction: Methods and Applications*. 2012.

[170] CAREY, G. F. *Computational Grids: Generation, Adaptation, and Solution Strategies*. Taylor & Francis, 1997.

[171] BURSTEDDE, C., GHATTAS, O., GURNIS, M., et al. "Extreme-Scale AMR". In: *Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '10, p. 1–12, USA, 2010. IEEE Computer Society. ISBN: 9781424475599. doi: 10.1109/SC.2010.25. Disponível em: <https://doi.org/10.1109/SC.2010.25>.

[172] AINSWORTH, M., ODEN, J. T. *A posteriori error estimation in finite element analysis*, v. 37. John Wiley & Sons, 2011.

[173] CAREY, G. *Computational Grids: Generations, Adaptation & Solution Strategies*. Series in Computational and Physical Processes in Mechanics. Taylor & Francis, 1997.

[174] LÖHNER, R. *Applied computational fluid dynamics techniques: an introduction based on finite element methods*. John Wiley & Sons, 2008.

[175] RIVARA, M. C. "Mesh refinement processes based on the generalized bisection of simplices", *SIAM Journal on Numerical Analysis*, v. 21, n. 3, pp. 604–613, 1984.

[176] THOMPSON, R. A. *Galerkin Projections Between Finite Element Spaces*. Relatório técnico, Virginia Polytechnic Institute and State University, 2015. Disponível em: <`https://vtechworks.lib.vt.edu/handle/10919/52968`>.

[177] HUNTER, J. D. "Matplotlib: A 2D graphics environment", *Computing in Science & Engineering*, v. 9, n. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55.

[178] "Anaconda Software Distribution". 2020. Disponível em: <`https://docs.anaconda.com/`>.

[179] HECHT, F. "New development in FreeFem++", *J. Numer. Math.*, v. 20, n. 3-4, pp. 251–265, 2012. ISSN: 1570-2820. Disponível em: <`https://freefem.org/`>.

[180] WASKOM, M. L. "seaborn: statistical data visualization", *Journal of Open Source Software*, v. 6, n. 60, pp. 3021, 2021. doi: 10.21105/joss.03021. Disponível em: <`https://doi.org/10.21105/joss.03021`>.

[181] INC., P. T. "Collaborative data science". 2015. Disponível em: <`https://plot.ly`>. [Online; accessed 17-June-2023].

[182] LINDSTROM, P. "Fixed-Rate Compressed Floating-Point Arrays", *IEEE Transactions on Visualization and Computer Graphics*, v. 20, n. 12, pp. 2674–2683, 2014. doi: 10.1109/TVCG.2014.2346458.

[183] LINDSTROM, P., ISENBURG, M. "Fast and Efficient Compression of Floating-Point Data", *IEEE Transactions on Visualization and Computer Graphics*, v. 12, n. 5, pp. 1245–1250, set. 2006. ISSN: 1077-2626. doi: 10.1109/TVCG.2006.143. Disponível em: <`http://ieeexplore.ieee.org/document/4015488/`>.

[184] BURTSCHER, M., RATANAWORABHAN, P. "FPC: A High-Speed Compressor for Double-Precision Floating-Point Data", *IEEE Transactions on Computers*, v. 58, n. 1, pp. 18–31, jan. 2009. ISSN: 1557-9956. doi: 10.1109/TC.2008.131. Conference Name: IEEE Transactions on Computers.

[185] CLAGGETT, S., AZIMI, S., BURTSCHER, M. "SPDP: An Automatically Synthesized Lossless Compression Algorithm for Floating-Point Data". In: *2018 Data Compression Conference*, pp. 335–344, mar. 2018. doi: 10.1109/DCC.2018.00042. ISSN: 2375-0359.

[186] DI, S., CAPPELLO, F. "Fast Error-Bounded Lossy HPC Data Compression with SZ". In: *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 730–739, maio 2016. doi: 10.1109/IPDPS.2016.11. ISSN: 1530-2075.

[187] DIFFENDERFER, J., FOX, A. L., HITTINGER, J. A., et al. "Error Analysis of ZFP Compression for Floating-Point Data", *SIAM Journal on Scientific Computing*, v. 41, n. 3, pp. A1867–A1898, 2019. doi: 10.1137/18M1168832. Disponível em: <`https://doi.org/10.1137/18M1168832`>.

Appendix A

Projection-based SciML models and preliminary simulations

In this appendix, we describe Proper Orthogonal Decomposition (POD)[6], used to generate reduced order models (ROM) that can be used to extrapolate physical parameters or temporal evolution. POD and DMD are usually employed in different situations, but they can also be used together [153, 156]. After describing POD, we present a simple linear case where both methods can be compared in terms of accuracy.

A.1 Proper Orthogonal Decomposition (POD)

In this appendix, we explore the Proper Orthogonal Decomposition (POD) method, the most traditional method regarding projection-based model order reduction. The projection-based model order reduction approach is a family of methods that consists on deriving a low-dimensional subspace that is constructed in order to retain the essential character of the high-dimensional system input-output map, reducing the computational costs regarding to the solution of large dimensional systems while preserving the overall accuracy of the high-dimensional problem. Projecting the governing equations onto the low-dimensional subspace yields the reduced model. The projection step is often intrusive and requires knowledge of the high-fidelity model structure, that is, it is not an equation-free method as the DMD method described in more detail in this thesis. Given the following dynamical system

$$\begin{aligned} \mathbf{M}\dot{\mathbf{y}}(\mathbf{t}) &= \mathbf{E}\mathbf{y}(t) + \mathbf{f}(t, \mathbf{y}(t)), \\ \mathbf{y}(0) &= \mathbf{y}_0, \end{aligned} \tag{A.1}$$

where $\mathbf{M}$ is a mass matrix, $\mathbf{E}$ is a linear term and $\mathbf{f}$ is a nonlinear term. The POD method is also a snapshot-based method, that is, the starting point is the generation of a dataset $\mathbf{Y} \in \mathbb{R}^{n \times m}$ for all computed solutions on Equation A.1. The solutions are stacked on a snapshots matrix. The POD method then consists on applying the SVD factorization on $\mathbf{Y}$ similar to Equation 2.9 and truncating into the first r components, similar to Equation 2.10. Despite the significant computational gain regarding the reduced basis being of rank $r \ll m \ll n$, the POD method also present two important features:

- First, the POD method naturally separates the space and time correlations of $\mathbf{Y}$ into the orthonormal matrices $\mathbf{U}$ and $\mathbf{V}$, respectively. The matrix $\mathbf{U}$ only contains the spatial correlations in the data, whereas all temporal information is found in $\mathbf{V}$. For example, the most common projection method in the POD context, the Galerkin projection, does not require the information of $\mathbf{\Sigma}$ and $\mathbf{V}$ in its reduced order model formulation, since the projection only occurs on the matrices regarding spatial discretization. In this appendix we consider the Galerkin projection, meaning that the reduced basis $\mathbf{\Psi}_{POD} = \mathbf{U}_r \in \mathbb{R}^{n \times r}$, where $\mathbf{U}_r$ is the matrix containing the first r collumns of $\mathbf{U}$. Figure A.1 shows the dimensionality reduction when the matrices are projected onto $\mathbf{\Psi}_{POD}$.
- Second, due to its formulation, the POD provides an efficient low-dimensional representation of the original dataset. Among all orthonormal bases of size r, the POD basis naturally minimizes the least squares error of snapshot reconstruction. This is guaranteed by the Eckhart-Young Theorem [63].

Similar to DMD, POD also minimizes a loss function in the context of SciML models. The optimization problem for POD can be posed as

$$\arg \min_{\mathbf{\Psi}_{POD} \in \mathcal{R}^{n \times r}} ||\mathbf{Y} - \mathbf{\Psi}_{POD} \mathbf{\Psi}_{POD}^T \mathbf{Y}||_F \tag{A.2}$$

where the optimal POD basis $\mathbf{\Psi}_{POD}$ minimizes the Frobenius norm of the difference of the snapshots matrix and its re-projection into the n-dimensional space after reduction to the r-dimensional space. In this case, considering the Galerkin projection, the basis can be defined as $\mathbf{\Psi}_{POD} = \mathbf{U}_r$.

After the basis generation, the projection into the subspace must be done. First, we approximate our system solution $y(t)$ as

$$y(t) \approx \mathbf{\Psi}_{POD} \mathbf{y}_r(t). \tag{A.3}$$

The substitution of (A.3) into the full order model (A.1) leads to a residual on

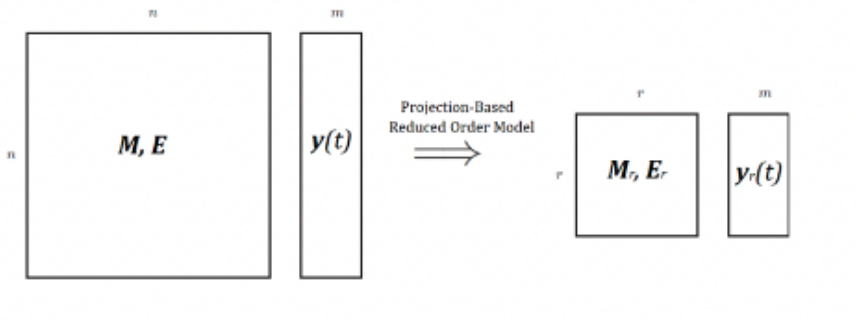


Figure A.1: Projection based ROM.

the full order model such that

$$\mathbf{M}\boldsymbol{\Psi}_{POD}\dot{\mathbf{y}}_r(t) - \mathbf{E}\boldsymbol{\Psi}_{POD}\mathbf{y}_r(t) - \mathbf{f}(t, \boldsymbol{\Psi}_{POD}\mathbf{y}_r(t)) = \mathcal{R}_{POD}. \tag{A.4}$$

The Galerkin projection is often the standard choice of projection method. It enforces the Galerkin orthogonality to the reduced order model, that is, the residual $\mathcal{R}_{POD}$ of the full order model on Equation (A.1) must be orthogonal to the range of the POD basis, that is

$$\boldsymbol{\Psi}_{POD}^T\mathcal{R}_{POD} = 0, \tag{A.5}$$

$$\boldsymbol{\Psi}_{POD}^T[\mathbf{M}\boldsymbol{\Psi}_{POD}\dot{\mathbf{y}}_r(t) - \mathbf{E}\boldsymbol{\Psi}_{POD}\mathbf{y}_r(t) - \mathbf{f}(t, \boldsymbol{\Psi}_{POD}\mathbf{y}_r(t))] = 0, \tag{A.6}$$

leading to

$$\mathbf{M}_r\dot{\mathbf{y}}(t) = \mathbf{E}_r\mathbf{y}(t) + \mathbf{f}_r(t, \mathbf{y}(t)) \tag{A.7}$$

where

- $\mathbf{M}_r = \boldsymbol{\Psi}_{POD}^T\mathbf{M}\boldsymbol{\Psi}_{POD} \in \mathbb{R}^{r\times r}$,
- $\mathbf{E}_r = \boldsymbol{\Psi}_{POD}^T\mathbf{E}\boldsymbol{\Psi}_{POD} \in \mathbb{R}^{r\times r}$,
- $\mathbf{f}_r = \boldsymbol{\Psi}_{POD}^T\mathbf{f}(t, \mathbf{y}(t)) \in \mathbb{R}^r$,

POD is also often split in two phases: the Offline and the Online phases. The Offline phase is characterized by the operations regarding large up-front such as generating the high dimensional dataset, SVD decompositions and projections, while the Online phase is responsible for evaluating the cheaper reduced order model. On linear problems and on problems that present affine parameter dependence, the described procedure of generating the reduced basis and projecting the high-dimensional system into the reduced subspace can be done only once. However, nonlinear problems and situations where the matrices have nonaffine parameter

dependence are more challenging and often require additional strategies. For instance, the dynamical system described on Equation (A.1) contains a nonlinear term. After the Galerkin projection on Eq. (A.7), we observe that the nonlinear term $\mathbf{f}_r(t, \mathbf{y}(t)) = \mathbf{\Psi}_{POD}^T \mathbf{f}(t, \mathbf{y}(t))$ requires the evaluation of the nonlinear term $\mathbf{f}(t, \mathbf{y}(t))$ before the projection into the range of the POD basis. Computing the nonlinear term in the high dimensional space for the complete evolution of the dynamical system can be expensive - requiring more resources and time - and largely affects the efficiency of the reduced order model. The ideal separation between offline and online phases becomes compromised in this case. To address this issue, one must introduce an additional approximation that removes the direct dependence of $\mathbf{f}_r$ on $\mathbf{y}(t)$. These approximation techniques are often described as hyper-reduction in the literature and are not covered in this appendix. The reduced order model dependence on the high-dimensional problem also occurs in a similar way when the problem is modeled by equations containing terms with nonaffine parameter dependence. Details on hyper-reduction techniques can be seen on [6, 157].

A.2 Numerical example: POD and DMD

In this section, we illustrate the difference between projection-based and equation-free reduced order models in a linear example with the purpose of reconstructing a function. We solve the transient advection equation in a 1D spatial domain, in which we compare the DMD method with the POD-Galerkin method in terms of accuracy and performance. The problem is governed by

$$\frac{d\mathbf{y}}{dt} + \beta \frac{d\mathbf{y}}{dt} = 0, \tag{A.8}$$

$$\mathbf{y}(x, 0) = \mathbf{y}_0, \tag{A.9}$$

$$\mathbf{y}(a, t) = \mathbf{y}(b, t) = 0, \tag{A.10}$$

being solved at spatial domain $x \in [0, 4]$ and time $[0, 3]$. Initial conditions are $\mathbf{y}_0 = sin(\pi x)$ if $0 \leq x \leq 1$ and 0 elsewhere. In order to lead Equation A.8 to our general formulation described on Equation A.1, we utilize a finite element discretization with a spatial step $\Delta x = 0.01$. We use the SUPG stabilization to avoid instabilities regarding the use of the Galerkin approximation in advection-dominated problems [158]. The dimension of the problem is $n = 399$. We note that in this case the mass matrix $\mathbf{M}$ is the identity matrix. In order to apply POD and DMD, we need to compute the snapshot set which is given by the temporal discretization of Equation A.8 with an implicit Euler scheme and a temporal step size $\Delta t = 0.01$. The initial conditions and the solution for all time steps computed can be seen on Figure A.2.

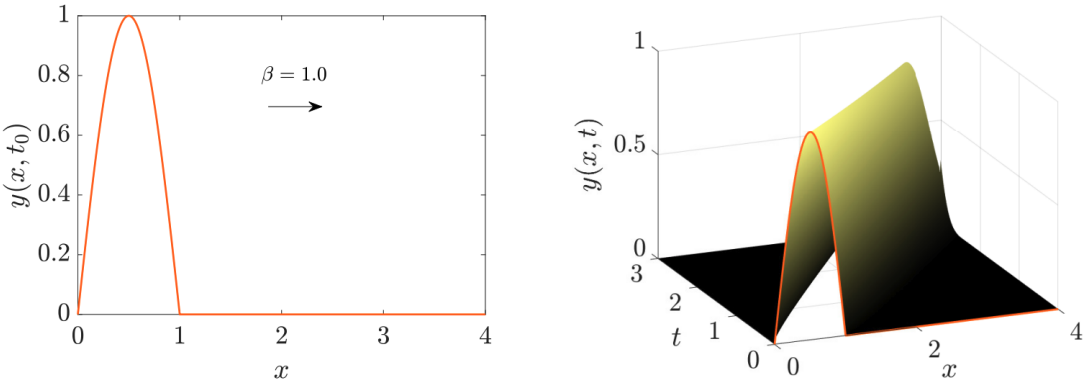


Figure A.2: Initial conditions (left) and solution of the convection problem (right).

Considering this is a linear problem, the nonlinear term from Equation A.1 vanishes and the general system now becomes:

$$\mathbf{M}\dot{\mathbf{y}}(\mathbf{t}) = \mathbf{I}\dot{\mathbf{y}}(\mathbf{t}) = \mathbf{E}\mathbf{y}(t), \mathbf{y}(0) = \mathbf{y}_0, \tag{A.11}$$

After the creation of the dataset $\mathbf{Y}$, we compare both POD-Galerkin and DMD in terms of efficiency and accuracy. In this example we consider three cases: $r = 5, 10$ and 15. Figure A.3 shows how the singular values decay for the snapshots matrix Y and the so-called relative energy κ is also evaluated for the three cases.

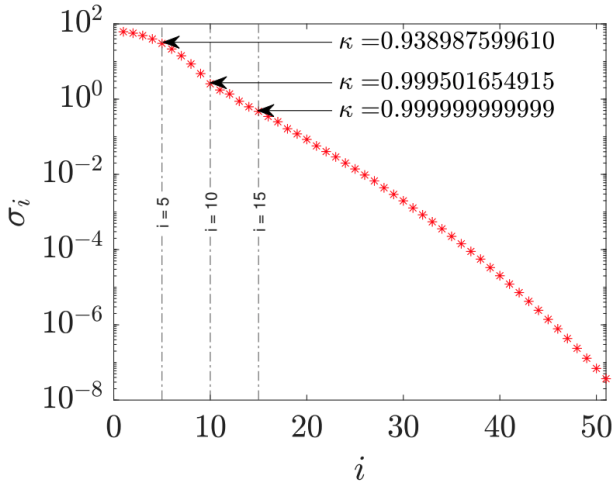


Figure A.3: Singular values of $\mathbf{Y}$ and relative energy for $r = 5, 10$ and 15.

We can see that for $r = 15$, the relative energy much closer to 1 than the values observed for $r = 5$ and 10. Figure A.4 shows the results of model order reduction with POD-Galerkin and DMD methods. From left to right, the reconstruction of $\mathbf{Y}$ using the DMD method and the POD-Galerkin method, and from top to bottom, results for basis truncation on $r = 5, 10$ and 15, respectively. As expected, increasing the rank of the basis functions leads to visually better solutions. Also, as

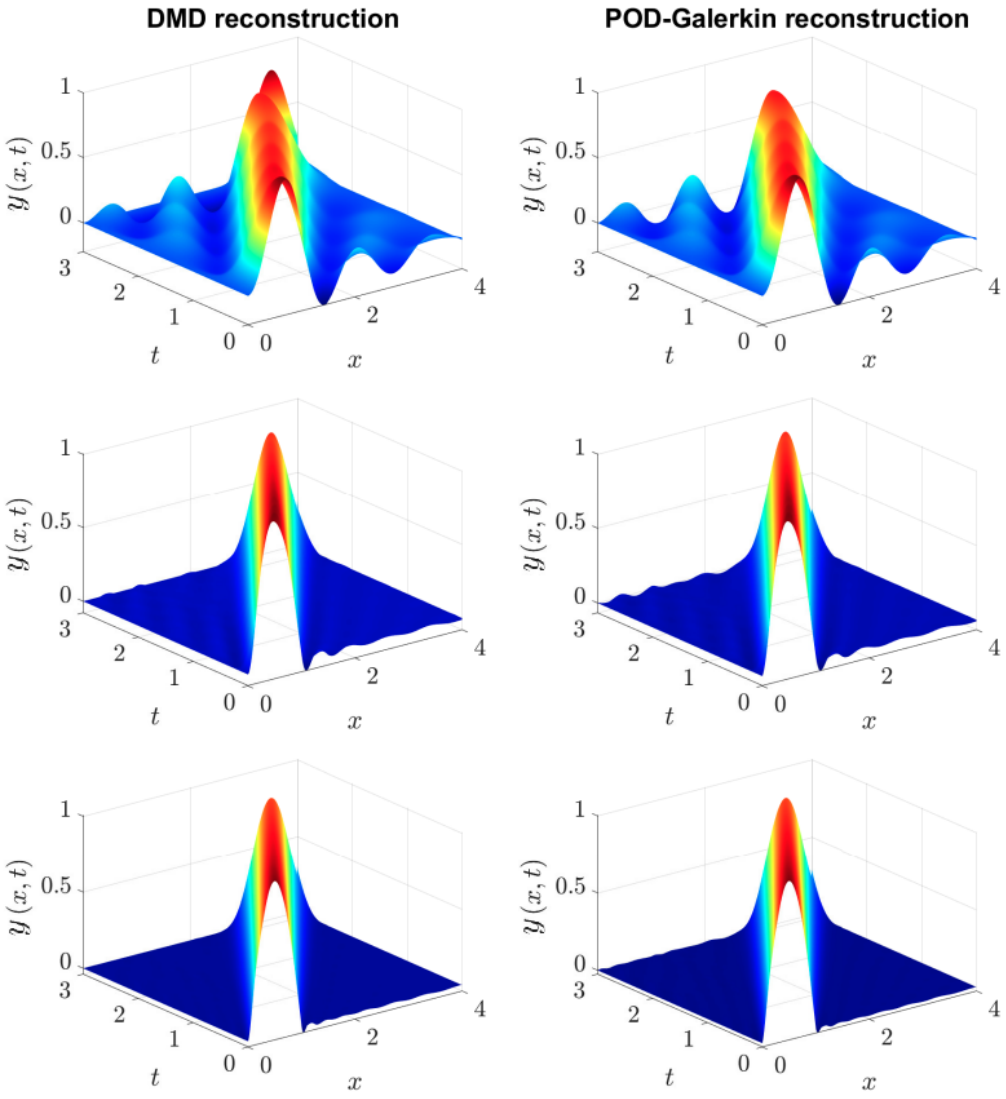


Figure A.4: Singular values of $\mathbf{Y}$ and relative energy for $r = 5, 10$ and 15.

an illustration for this case, we compare the eigenvalues of matrix $\mathbf{A}$, obtained by computing the full pseudoinverse on Equation 2.8, and its low-rank approximation $\tilde{\mathbf{A}}$. Figure A.5 shows these eigenvalues in the complex plane for the case of r = 15. We observe that the eigenvalues become close to the expected values thus the

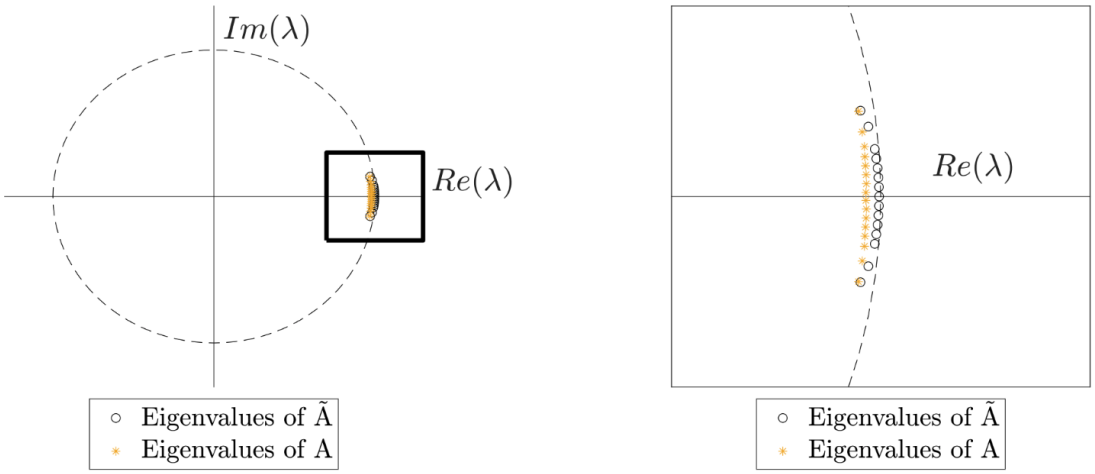


Figure A.5: Singular values of $\mathbf{Y}$ and relative energy for $r = 5, 10$ and 15.

approximation of $\mathbf{A}$ indeed conserves the dynamics of the system. It is well known that advection problems have a high variability during time evolution and it is difficult to capture the dynamics with only a few basis functions [156]. Apart from visual analysis, we compute the relative error with respect to the Frobenius norm as an accuracy analysis. We compare the accuracy for $r = 1, 2, \ldots, 100$ and the results for each case are illustrated on Figure A.6. We analyze the relative error of the Frobenius norm of both the reconstructed and original datasets in three regions. In the region for $r \leq 25$ both methods behave in a similar way where the relative error decays exponentially with the increase of r. In the second region described by $25 < r < 69$, the POD method presents better results in comparison with the DMD method, leading to a larger relative error decay with the increase of r. The third region is represented by $r \geq 69$ and a very different behavior between the two methods is observed. The POD method presents a very small and constant relative error while the DMD shows unstable behavior, returning unphysical solutions. This occurs because the mapping $\tilde{\mathbf{A}}$ approaches $\mathbf{A}$ with the increasing of r. Matrix $\mathbf{A}$ is a generally highly ill-conditioned and when the state dimension n is large, the aforementioned matrix may be even intractable to be analyzed directly [24]. That is, there is a critical r value r_c such that the DMD cannot be used, and it seems that it is proportional to the size of the dataset $\mathbf{Y}$. The increasing of r means that the truncated pseudoinverse becomes closer to the actual pseudoinverse, a matrix whose condition number is well known for having large influence on the solution of the linear least squares problem. In terms of matrix conditioning, in the best scenario,

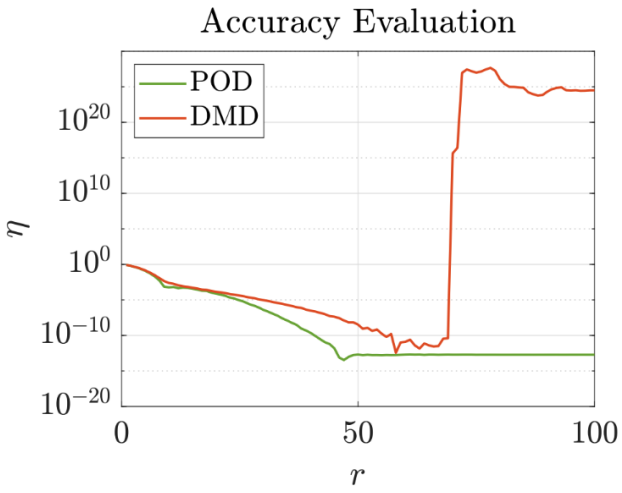


Figure A.6: Frobenius relative error for DMD and POD approximations with $1 \leq r \leq 100$.

the condition number for the solution depends on $cond(Y_1^{\dagger})$ and is proportional to $cond(Y_1^{\dagger})$ for moderate and large residuals [62].

Appendix B

Partial Differential Equations Approximation

In this thesis, we apply the Dynamic Mode Decomposition method on numerical data related to problems such as the spread of the COVID-19 disease in Italy, the evolution of a density-driven current and a bubble rising. Given those are real world situations, the PDEs related to these problems cannot be solved using analytical strategies and require some mathematical approximation strategies. These procedures are often called numerical methods, which comprise a significant number of different approaches for bringing the equations that exist on an infinite dimensions space to a finite dimension where we can quantify information using computers. In this thesis, we describe two numerical methods: the Finite Difference Method (FDM) and the Finite Element Method (FEM). Given that the PDEs of our applications are time and space dependent, our strategy is to approximate the temporal derivative using FDM while discretizing spatially using FEM.

B.1 The Finite Difference Method

In this study, the finite difference method is used to discretize the PDEs in time. In this case, the partial derivative with respect to time is approximated to a finite difference. The θ family of methods is a common strategy to integrate first-order differential equations, that is

$$\frac{\partial u}{\partial t} \approx \frac{u(t^{n+1}) - u(t^n)}{\Delta t} = \theta u(t^{n+1}) + (1-\theta)u(t^n) + \mathcal{O}((1/2-\theta)\Delta t, \Delta t^2)) \quad \text{(B.1)}$$

generates a family of methods that have various properties depending on θ [159]. For values of $\theta < 1/2$ the schemes are conditionally stable. For $\theta = 0$, the Euler method is the most common conditionally stable method. For values of $\theta \geq 1/2$ the methods are unconditionally stable. Common methods are the Backward Euler

($\theta = 1$), Galerkin ($\theta = 2/3$), and Crank-Nicolson ($\theta = 1/2$). It is important to notice that, among these methods, the Crank-Nicolson scheme is the only method that yields second-order accurate solutions in comparison with the other first-order θ methods.

Another family of methods for numerical integration of temporal derivatives is the backward differentiation formula (BDF). They are linear multistep methods that, for a given function and time, approximate the derivative of that function using information from already computed time points, thereby increasing the accuracy of the approximation [62]. These methods are especially used for the solution of stiff differential equations. The BDF2 method is considered in this thesis for being second-order accurate and being unconditionally stable. Considering that time t is split in intervals Δt, the temporal evolution of a given field ψ is

$$\frac{\partial \psi}{\partial t} = \frac{\alpha_\beta \psi_{n+1} - \psi_{BDF,\beta}}{\Delta t}, \tag{B.2}$$

where $\psi_{n+1} - \psi_{BDF,\beta} = 2\psi_n - 0.5\psi_{n-1}$ and $\alpha_\beta = 1.5$. For cases where other BDF methods are considered (i.e. BDF1 or BDF3), these parameters are changed accordingly [160].

B.2 The Finite Element Method

In this study, the finite element method is the method of choice to spatially approximate the PDEs. In this section, we initially describe the standard finite element method, that will be used in the simulations of the SEIRD model. The incompressible Navier-Stokes and the transport equations show unphysical behaviour when approximated by the standard finite element method, requiring additional stabilization terms. We address this issue further on Section B.2.1.

Finite element approximations are based on a variational form (or weak form) of the PDE. The weak form of a PDE is obtained by multiplicating the PDE by a test function $w(\mathbf{x})$ from a proper space function and integrating all over its domain. The Galerkin method is used to approximate the solutions from a infinite dimension space to a finite one. Equation B.3 shows this approximation, where $u(\mathbf{x}, t)$ is the exact solution of the PDE in the weak form, $\hat{u}(\mathbf{x}, t)$ is the approximate solution, $N_i(\mathbf{x})$ are the basis functions and $\tilde{u}_i(t)$ are the unknown variables, such that:

$$u(\mathbf{x}, t) \approx \hat{u}(\mathbf{x}, t) = \sum_{i=1}^{n} N_i(\mathbf{x}) \tilde{u}_i(t), \tag{B.3}$$

$$w(\mathbf{x}) = \sum_{i=1}^{n} N_i(\mathbf{x}) w_i. \tag{B.4}$$

The standard finite element method consists on discretizing a spatial domain $\Omega \subset \mathbb{R}^{nsd}$, with boundaries $\Gamma \subset \mathbb{R}^{nsd-1}$ into a mesh composed by nodes and elements. In this case, the unknown variables $\tilde{u}_i(t)$ are also defined as nodal values. Each element is composed by its domain $\Omega^e \subset \mathbb{R}^{nsd}$ and boundary $\Gamma^e \subset \mathbb{R}^{nsd-1}$. Figure B.1 illustrates a finite element method domain. Mathematically speaking,

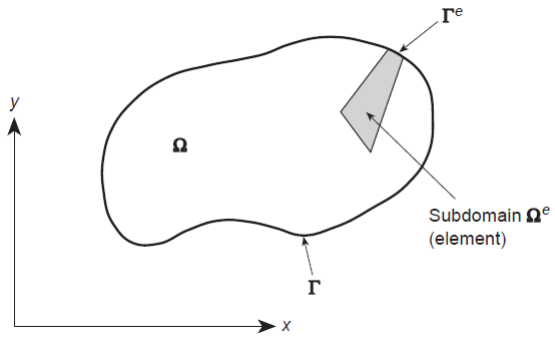


Figure B.1: Graphical representation of a two-dimensional domain divided into subdomains (elements). Adapted from ZIENKIEWICZ [161]

the discretization of the spatial domain Ω into *nel* number of elements is such that:

$$\begin{cases} \Omega = \cup_{e=1}^{nel} \Omega^e, \\ \emptyset = \cap_{e=1}^{nel} \Omega^e. \end{cases} \tag{B.5}$$

The Galerkin isoparametric formulation is used to approximate the SEIRD model in this study, so that all functions are interpolated in the domain using the same basis functions N_i. The basis function N_i are illustrated on Fig. B.2 and must respect the interpolation condition presented as:

$$N_i(x_j) = \delta_{ij}, \tag{B.6}$$

where δ_{ij} is the Kronecker delta. The compact support property added to the domain subdivision seen on equation (B.5) allows us to simplify the representation of the variational formulation integrals over the complete domain as a sum of the integral inside each element. Since the integration of each element is required, a reference space ξ is used where the definition of the basis functions in the element are well defined, enabling the use of well known quadrature rules, assisting on the automation of the process.

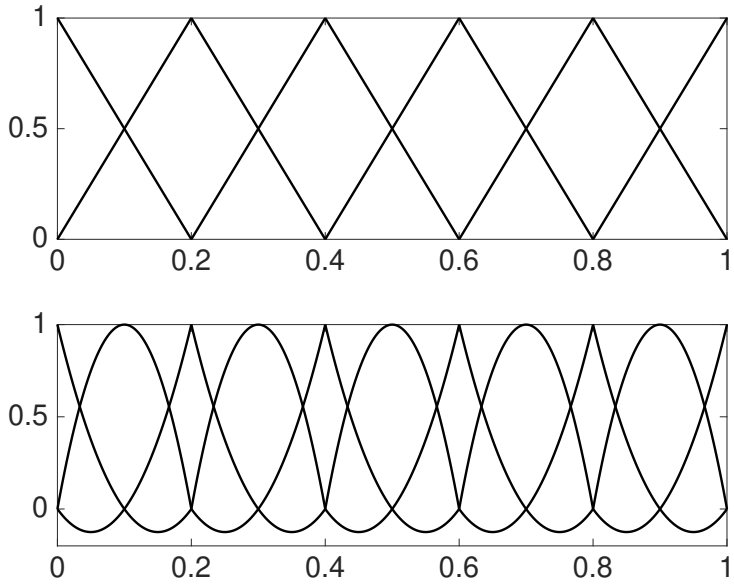


Figure B.2: Illustration of a unit one-dimentional domain split into five finite elements with polynomial basis functions (linear and quadractic, respectively).

B.2.1 The Finite Element Method in Flow Problems

Velocity and pressure coupling

In this thesis, we approximate the incompressible Navier-Stokes equation to model the lock-exchange and the bubble rising problems. The Navier-Stokes equations for an incompressible flow can be defined as:

$$\rho_f \left[\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} \right] - \mu_f \Delta \mathbf{u} + \nabla p = \mathbf{b} \quad \text{in } \Omega \times [0, T], \tag{B.7}$$

$$\nabla \cdot \mathbf{u} = 0 \quad \text{in } \Omega \times [0, T], \tag{B.8}$$

$$\mathbf{u} = \mathbf{u}_D \text{ in } \Gamma_D \times [0, T], \tag{B.9}$$

$$-p\mathbf{n} + \mu_f (\mathbf{n} \cdot \nabla)\mathbf{u} = \mathbf{u}_T \text{ in } \Gamma_N \times [0, T], \tag{B.10}$$

$$\mathbf{u}(\mathbf{x}, 0) = \mathbf{u_0}(\mathbf{x}) \qquad \text{in } \Omega, \tag{B.11}$$

where ρ_f is the fluid density, $\mathbf{u}$ is the fluid convective velocity, μ_f is the kinematic viscosity, p is the pressure and $\mathbf{b}$ is the volume force per unit mass of fluid. $\mathbf{u}_D$ and $\mathbf{u}_T$ are the boundary conditions prescribed on the boundaries Γ_D and Γ_N, respectively, and $\mathbf{u}_0$ are the initial conditions. Boundaries are such that $\Gamma_D \cup \Gamma_N = \Gamma$ and $\Gamma_D \cap \Gamma_N = \emptyset$.

Equation (B.8) is the incompressibility constraint of the fluid, being the consequence of the fact that in an incompressible continuum the rate of change of the mass density following the motion is zero. This incompressibility constraint causes

unstable behaviour in the finite element method when an inappropriate combination of element interpolation functions for the velocity and pressure is employed. As a consequence, instabilities in the pressure field may appear, and this is independent of the Reynolds number. The Ladyzhenskaya-Babuška-Brezzi condition - also known as LBB condition or inf-sup condition - is stated as: The existence of a stable finite element approximate solution $(\mathbf{u}^h, p^h)$ to the steady Stokes problem or the incompressible Navier-Stokes equation depends on choosing a pair of spaces V^h and Q^h, such that the following discrete inf-sup condition holds:

$$\inf_{q^h \in Q^h} \sup_{w^h \in W^h} \frac{(q^h, \nabla \cdot w^h)}{||q||_0 ||w^h||_1} \geq \theta_{LBB} > 0, \tag{B.12}$$

where θ_{LBB} is independent of the mesh size. That is, not all pair of elements used to approximate the velocity and pressure fields are suitable for modelling the incompressible Navier-Stokes equation (or even the Stokes equation). However, it is known that stabilized methods can circumvent the necessity of having LBB-compatible element pairs. In this study we consider the Residual Based Variational Multiscale (RBVMS) formulation, which generate multiple stabilization terms such as the SUPG [158] and PSPG that suffice to avoid problems with the LBB condition [162].

Convection dominated flows

It is known that applying the traditional Galerkin formulation to fluid dynamics problems can lead to global spurious oscillations on high Reynolds number flows while solving the Navier-Stokes equation [159]. Diffusion-reaction equations with reaction dominated flows and convection-diffusion equations with advection dominated flows reveal the same behaviour [163]. Since this study uses the finite element method for numerical approximation of the described PDEs, additional stabilization techniques will be required.

Many stabilization techniques have been developed in the past decades, such as the Streamline Upwind Petrov-Galerkin method [164], the Galerkin Least Squares method [165] and the Subgrid Scale method [166]. All methods rely on a different way to model the weight functions. In this thesis, we focus on the RBVMS formulation [111–114] for the incompressible Navier-Stokes equation as well as the transport equation. The RBVMS formulation aims to split the velocity and pressure fields into a coarse component and a fine component. The fine-scale components are modeled in terms of the residual of the large ones, and then we replace their effect into the largescale equation. The well-known stabilization formulations SUPG, PSPG, and LSIC, appear naturally within this framework [112, 167]. The other terms that arise from this formulation might be interpreted as numerical representations of the

cross and Reynolds stresses arising in Large Eddy Simulation (LES) formulations [168]. To approximate the incompressible Navier-Stokes equation using the RBVMS formulation, the primitive variables can be decomposed such that

$$\mathbf{u} = \mathbf{u}^h + \mathbf{u}' \tag{B.13}$$

$$p = p^h + p', \tag{B.14}$$

where the subscript h denotes the coarse-scale component of the solution, while the superscript $'$ refers to the fine one. We insert the previous scale splitting in a standard variational Galerkin formulation built upon Equations B.8 - B.11. We assume the following weight spaces ${\mathbf{S}_\mathbf{u}}^h$ and S_p^h for velocity and pressure respectively, along with their trial spaces counterparts $\mathbf{V}_u^h$ and V_p^h and the weight functions $\mathbf{w}$ and q such that,

$${\mathbf{S}_\mathbf{u}}^h = \{\mathbf{u}^h(\cdot,t) \in \mathcal{H}^1(\Omega) \mid \mathbf{u}^h = \mathbf{u}_D \text{ on } \Gamma_D\}, \tag{B.15}$$

$${\mathbf{V}_\mathbf{u}}^h = \{\mathbf{w}^h(\cdot,t) \in \mathcal{H}^1(\Omega) \mid \mathbf{w}^h = 0 \text{ on } \Gamma_D\}, \tag{B.16}$$

$$S_p^h = V_p^h = \{q^h(\cdot,t) \mid q^h(\cdot,t) \in \mathcal{L}^2(\Omega)\}. \tag{B.17}$$

The weak formulation now reads: find $\mathbf{u} \in {\mathbf{S}_\mathbf{u}}^h$ and $p \in S_p^h$ such that $\forall \mathbf{w} \in {\mathbf{V}_\mathbf{u}}^h$ and $\forall q \in V_p^h$,

$$\begin{aligned}\left(\mathbf{w}, \rho\frac{\partial \mathbf{u}}{\partial t}\right)_\Omega + (\mathbf{w}, \rho\mathbf{u}\cdot\nabla\mathbf{u})_\Omega + (\nabla\mathbf{w}, \mu\nabla\mathbf{u})_\Omega - (\nabla\cdot\mathbf{w}, p)_\Omega \\ -(\mathbf{w}, \mathbf{b})_\Omega - (\mathbf{w}, \mu\nabla\mathbf{u}\cdot\mathbf{n})_\Gamma \\ +(\mathbf{w}, p\mathbf{n})_\Gamma + (q, \nabla\cdot\mathbf{u})_\Omega = 0\end{aligned} \tag{B.18}$$

Some simplifications are introduced to come up with a feasible numerical scheme. We admit the static hypothesis, such that the fine scales are algebraically related to the residuals of the governing equations. The time derivatives of the fine scales are supposed to vanish (rendering the denomination static fine scales), and the spatial derivatives in the fine scales equations are approximated through algebraic operators (much inspired on stabilized FEM) leading to the following set of fine scales equations

$$\mathbf{u}' = -\tau_m \mathcal{R}_m \tag{B.19}$$

$$p' = -\tau_c \mathcal{R}_c \tag{B.20}$$

$$\tag{B.21}$$

where τ_m and τ_c are stabilization parameters given by the standard expressions of stabilized methods,

$$\tau_m = \left(\frac{4}{\Delta t^2} + \mathbf{u}^h \cdot \mathbf{G}\mathbf{u}^h + \nu^2 \mathbf{G} : \mathbf{G} \right)^{-1/2} \tag{B.22}$$

$$\tau_c = (\mathbf{g}^* \cdot \tau_m \mathbf{g}^*)^{-1} \tag{B.23}$$

begin $\mathbf{G}$ a second rank metric tensor

$$\mathbf{G} = \frac{\partial \xi^T}{\partial \mathbf{x}} \frac{\partial \xi}{\partial \mathbf{x}} \tag{B.24}$$

and $\mathbf{g}^*$ a vector obtained from the column sums of $\dfrac{\partial \xi}{\partial \mathbf{x}}$, the inverse Jacobian of the element mapping between the parent and the physical domain

$$\mathbf{g}^* = \{g_i^*\}, \text{ where} \tag{B.25}$$

$$g_i^* = \sum_{j=1}^{d} \left(\frac{\partial \mathbf{xi}}{\partial \mathbf{x}} \right)_{ji}. \tag{B.26}$$

The residuals for the continuity and moment equations are given in function of the coarse-scales, that is,

$$\mathcal{R}_m = \rho_f \left[\frac{\partial \mathbf{u}^h}{\partial t} + (\mathbf{u}^h \cdot \nabla)\mathbf{u}^h \right] - \mu_f \Delta \mathbf{u}^h + \nabla p - \mathbf{b}, \tag{B.27}$$

$$\mathcal{R}_c = \nabla \cdot \mathbf{u}^h. \tag{B.28}$$

It is important to remember that the weighting functions also are decomposed into coarse and fine scales, resulting in two different equations. However, since we are considering an approximation of the fine scales based on the residuals, we only have to solve the coarse-scale equations. Finally, after substituting and rearranging terms, we can obtain the RBVMS formulation for the Navier-Stokes equations as follows: find $\mathbf{u} \in {\mathbf{S_u}}^h$ and $p \in S_p^h$ such that $\forall \mathbf{w} \in {\mathbf{V_u}}^h$ and $\forall q \in V_p^h$,

$$\begin{aligned}
\left(\mathbf{w}, \rho \frac{\partial \mathbf{u}^h}{\partial t} \right)_\Omega + (\mathbf{w}, \rho \mathbf{u}^h \cdot \nabla \mathbf{u}^h)_\Omega + (\nabla \mathbf{w}, \mu \nabla \mathbf{u}^h)_\Omega - (\nabla \cdot \mathbf{w}, p^h)_\Omega \\
- (\mathbf{w}, \mathbf{b})_\Omega - (\mathbf{w}, \mu \nabla \mathbf{u} \cdot \mathbf{n})_\Gamma \\
+ (\mathbf{w}, p^h \mathbf{n})_\Gamma + (q, \nabla \cdot \mathbf{u}^h)_\Omega \\
+ \sum_e (\nabla \mathbf{w}^h, \tau_m \mathcal{R}_m \otimes \mathbf{u}^h)_{\Omega^e} - \sum_e \frac{1}{\rho} (\nabla \mathbf{w}^h, \tau_m \mathcal{R}_m \otimes \tau_m \mathcal{R}_m)_{\Omega^e} \\
+ \sum_e \rho (\nabla \mathbf{w}^h, \tau_c \mathcal{R}_c)_{\Omega^e} + \sum_e \frac{1}{\rho} (\nabla \mathbf{q}^h, \tau_m \mathcal{R}_m)_{\Omega^e} = 0.
\end{aligned} \tag{B.29}$$

Similar to the incompressible Navier-Stokes equation, the transport equation also requires stabilization for convection or reaction dominated problems [163]. Considering a scalar field ϕ, the transport equation reads

$$\frac{\partial \phi}{\partial t} + \beta \cdot \nabla \phi - \mathbf{k} \Delta \phi + s\phi = f \tag{B.30}$$

where β is a velocity field responsible for the convection term, $\mathbf{k}$ is the diffusivity tensor, s is the reaction term and f is an external force term. The convection-diffusion-reaction problems can be parametrized by non-dimensional numbers. The Peclet number (Pe) represents the significance of convection relative to diffusion. For large Pe, convection dominates, while for small Pe diffusion dominates. The convection-dominant case gives rise to interior and boundary layers in ϕ, which causes difficulties in the numerical approximation of the convection-diffusion-reaction equation. Diffusion-dominant cases reveal good approximation within the Galerkin framework [159]. Figure B.3 shows different Pe solutions for the transport equation: a one dimension domain is assumed, advective velocity is constant and points from left to right, and ϕ is set to unity on the left side of the interval and zero on the right side. For $Pe = 0$ the analytic solution is a straight line connecting the prescribed boundary values. As the Pe number increases, the solution forms a thin boundary layer on the right side of the domain. Solutions containing large gradients present a source of difficulty for numerical approximation of the convection–diffusion-reaction equation.

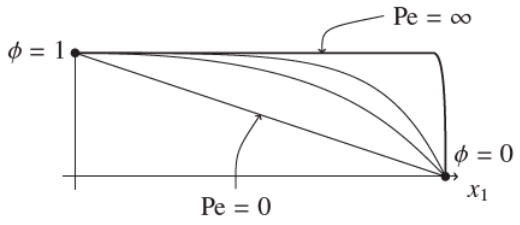


Figure B.3: Illustration of the solution behavior for the Convection–diffusion equation at Pe number ranging from zero to nearly infinity. From [169].

Another important non-dimensional number is the Damköhler number (Da), which represents the significance of reaction relative to convection. The Damköhler number is widely used in chemical engineering to relate the chemical reaction timescale (reaction rate) to the transport phenomena rate occurring in a system. For large Da reaction dominates, while for small Da convection dominates. The case $Da = \infty$ corresponds to pure reaction, meaning that the whole substance was converted during transport. It is known that the Galerkin formulation for the transport equations does not yield stable solutions for large Dahmkoler and Peclet numbers - that is, reaction dominated flows and convective dominated flows [163]. In this thesis, we consider the SUPG formulation in the finite element discretization. The weak form of the problem can be obtained by integrating the Equation (B.30) in

its strong form against weighting function $w \in \mathcal{H}^1(\Omega)$, where $\mathcal{H}^1(\Omega)$ is the Sobolev space of the square integrable functions with an integrable first weak derivative, and applying the divergence theorem. Being $P^k(\Omega^e)$ the space of polynomials of degree equal or less than k over Ω^e, the function spaces are defined as:

$$S_\phi^h = \{\phi^h(\cdot,t) \in \mathcal{H}^1(\Omega) \mid \phi^h(\cdot,t)|_{\Omega_e}|_{\Omega_e} \in P^k(\Omega^e), \forall e\}, \tag{B.31}$$

$$W^h = \{w^h \in \mathcal{H}^1(\Omega) \mid w^h|_{\Omega_e} \in P^k(\Omega^e), \forall e\}. \tag{B.32}$$

The variational formulation for this problem considering the SUPG method reads: find $\phi^h \in \mathcal{S}_\phi^h$ such that $\forall \mathbf{w}^h \in \mathcal{W}^h$

$$\left(w, \frac{\partial \phi}{\partial t}\right)_\Omega - (w, \beta \cdot \nabla\phi)_\Omega + (\nabla w, \mathbf{k}\nabla\phi)_\Omega$$
$$+ \sum_{e=1}^{nel} (\beta \cdot \nabla w^h \tau \mathcal{R}_\phi)_{\Omega^e} - (w^h, f)_\Omega - (w^h, g_q)_\Gamma = 0 \tag{B.33}$$

being $\mathcal{R}_\phi = \beta \cdot \nabla\phi^h - \nabla \cdot (\mathbf{k}\nabla\phi^h) + s\phi^h - f$ the residual of the transport equation after discretization.

In this section we presented the weak form of all governing equations used in this thesis. The incompressible Navier-Stokes equation, used in the 2D and 3D gravity current and bubble rising problems are discretized according to Equation B.29. It is important to mention that each problem has its variations in terms of weak form equations (i.e. the bubble rising problem has two external forces terms, one regarding the body forces and one related to the surface tension responsible to preserve the bubble structure). The temporal derivative is approximated using the Backward Euler ($\theta = 1$ on the theta temporal integration method) for all problems in this thesis. The transport equation is used for the level set equation on the bubble rising problem, the sediment concentration on both lock-exchange problems and the continuous SEIRD model. For the fluid dynamics problems, the convection is dominant requiring the SUPG formulation on Equation B.33 and time discretization is done by the BDF2 method. For the SEIRD model, the diffusive term dominates the dynamics, meaning that the Galerkin method suffices for spatial discretization. Also, temporal integration is also done by the BDF2 method. For the Galerkin method, the term regarding the residual of the transport equation on Equation B.33 is not considered.

B.3 Adaptive Mesh Refinement/Coarsening (AMR/C)

In this section, we describe the Adaptive Mesh Refinement/Coarsening (AMR/C) approach in finite element approximations. Finite element meshes can have different topologies. It is common for finite element meshes to have more elements on regions of interest for the dynamics of the problem and less elements in regions that are not important for the solution. The general structure of the AMR/C scheme is given in Algorithm 6. Three criteria are fundamental in an AMR/C algorithm: *remeshing*, *flagging*, and *stopping*. The remeshing criterion defines whether the computed solution at a given time-step requires remeshing driven by global *a posteriori* error estimators or by calling the refinement/coarsening procedure at every j time-steps. Error estimators and indicators of different degrees of complexity and computational cost can be used to drive adaptive simulations. We can highlight residual, interpolation-based, flux-jump, patch recovery and adjoint-based dual estimators [170]. Next, all mesh elements are visited and flagged for refinement or coarsening. The element flagging criterion is often represented by local *a posteriori* error estimators or indicators, i.e. flux-jumps of the solution gradient. In possession of the flagged elements, the remeshing algorithm is invoked. Finally, the previous mesh solution must be projected or interpolated into the new mesh. This is done by the projection/interpolation algorithm. The whole process is repeated until the stopping criterion is achieved. This criterion could be error equidistribution (until a certain threshold), a given element size, the maximum number of elements in the mesh, or how many refinement levels are desired. Note that the meshes generated in Algorithm 6, guided by the error estimation procedure, have different dimensions (number of degrees of freedom) and different topologies, which poses difficulties for DMD (in fact, for any snapshots-based method).

Reducing the number of equations n_{eq} on the nonlinear system is crucial for efficiency, specially considering that the optimal computational complexity of a single physics transient finite element simulation is of $\mathcal{O}(n_{eq}^{\frac{4}{3}})$ [171], being n_{eq} the number of equations of the system. Milestones addressing this subject are finite element *a posteriori* error indicators [172], techniques such as adaptation, interpolation [173, 174] and projection [84, 150, 151], as well as libraries and frameworks containing automated versions of AMR/C techniques [82, 83]. To setup the nomenclature of our refinement procedure, we consider some definitions. Our non-degenerate family of simplices meshes $\{\mathcal{T}_z\}_{z\in\ddagger}$ are obtained by local refinements of a reference coarse mesh, denoted hereafter by $\mathcal{T}^{\text{coarse}}$ at every iteration z. At the other end, we consider a reference fine mesh, denoted hereafter by $\mathcal{T}^{\text{fine}}$, assumed to resolve all the scales of the solution, meaning that its characteristic mesh length is capable of cap-

Algorithm 6 Adaptive Mesh Refinement/Coarsening Algorithm

INPUT: Finite element solution computed at a given time instant t_i.
OUTPUT: Finite element solution computed at a given time instant t_i projected onto an adaptive mesh.

1. Definition of a remeshing criterion: global *a posteriori* error estimator, remeshing at every j time-steps, etc.
for each computed solution **do**
  if one or more remeshing criteria are met **then**
    while one or more stopping criteria are not met **do**
      2. Compute local error estimators (or another strategy) to identify the elements requiring refinement or coarsening and flag them.
      3. Call the refinement/coarsening procedure to generate a new mesh for the flagged elements. Different strategies for mesh refinement/coarsening lead to distinct meshes.
      4. Project or interpolate the solution computed on instant t_i onto the new mesh
    end while
  end if
end for

turing all spatial scales. So each member of the family of meshes $\{\mathcal{T}_z\}_{z\in\ddagger}$ is a mesh with characteristic length between $\mathcal{T}^{\text{coarse}}$ and $\mathcal{T}^{\text{fine}}$. The specification of $\mathcal{T}^{\text{coarse}}$ and $\mathcal{T}^{\text{fine}}$ are problem dependent and are described in each application in the numerical results chapter. We consider a spatial error vector $\eta_s^2 = \{\eta_e^2\}, e = 1, 2, ..., nel$, which is the number of elements in the current mesh.

B.3.1 AMR/C algorithm: `FEniCS` simulations

In this thesis, we consider the `FEniCS` framework for the 2D lock-exchange simulation, which is equipped with AMR/C. For this problem, we consider a flux-jump approach. The flux-jump is an interesting choice for this problem given that the spatial adaptivity for this problem is mostly required on the interface between $c = 1$ and $c = 0$. For the *remeshing* phase, we do not compute error estimators to track the necessity of remeshing. Yet we invoke the AMR/C routine at every time step. It is important to mention that `FEniCS` is not equipped with coarsening strategies. Therefore, if we keep refining regions of interest in the domain, we may end up with refined regions that were of interest previously and will add no relevant accuracy to the solution. Therefore, for all time steps, we project the previous solution into a coarse mesh $\mathcal{T}^{\text{coarse}}$ where all elements contain the largest element size desired for that simulation. The *flagging* is invoked and the mesh is refined accordingly. For the *flagging* phase, we consider the following error indicator

$$\eta_e^2 = \sum_{s=1}^{3} ||h_s^{1/2}[\nabla\{c\}]_s||_{2(s)}^2, \tag{B.34}$$

where h_{el} is the element size, s is the triangle edge, h_s is the edge length, and $[\nabla\{c\}]_s = (\nabla\{c\})^+ \cdot \mathbf{n}^+ + (\nabla\{c\})^- \cdot \mathbf{n}^-$ is the jump of the gradient of the average phase field across the inter element boundary s, and $\mathbf{n}$ is the unit normal vector to s. The superscripts + and − indicate that the vectors are evaluated in the positive and negative directions of the facet s, respectively. Given the spatial error vector η_s we flag elements for refinement according to the maximum norm $||\eta_s||_\infty$. The maximum norm is defined such that a given mesh should present local errors within a prescribed value and presents direct influence on the mesh generated. After flagging the elements that require refinement, the standard refinement routine in `FEniCS` is invoked. By default, the bisection method [175] is used to generate new elements. In this case, we establish two levels of refinement for the *stopping* criterion. That is, $\mathcal{T}^{\text{fine}}$ is achieved by refining all elements of the $\mathcal{T}^{\text{coarse}}$ mesh. The refinement procedure is described in Algorithm 7 and illustrated on Figure B.4.

Algorithm 7 2D lock-exchange AMR/C strategy

INPUT: Finite element solution computed at a given time instant t_i.
OUTPUT: Finite element solution computed at a given time instant t_i projected onto an adaptive mesh.

1. Solve the nonlinear system for the previous mesh (in case of initial step, the fine mesh $\mathcal{T}^{\text{fine}}$ is used)
2. Local error vector η_s^2 is calculated using Equation B.34 for every element in the mesh.
for all time steps of the simulation: **do**
  while the target element size is not achieved or the maximum number of refinement iterations is reached: **do**
    3. Project the solution onto the updated mesh $\mathcal{T}_z$ by interpolation ($\mathcal{T}^{\text{coarse}}$ if it is the first iteration). `FEniCS` standard projection technique is the $\mathcal{L}^2$ projection.
    4. Loop over the cells in the current mesh and compare the local error of each element η_e with the maximum norm allowed for the error vector $||\eta_s||_\infty$.
    5. Mark the cells that surpassed the maximum norm allowed for the mesh $||\eta_s||_\infty$.
    6. Refine and update the mesh.
  end while
  7. Proceed to the nonlinear system solver and solve the present time step for the generated mesh.
end for

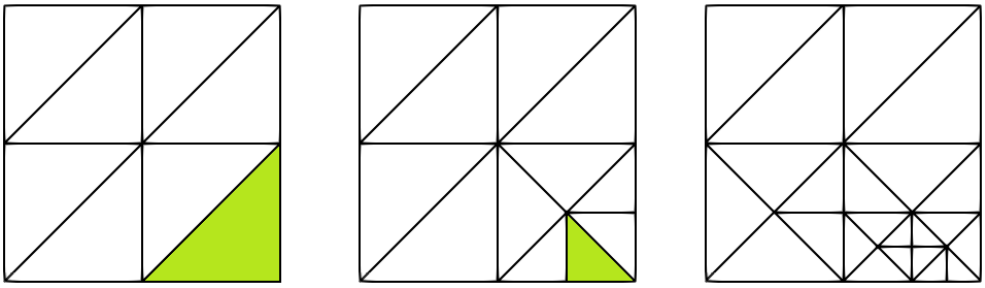


Figure B.4: Triangular mesh refinement procedure using the bisection method. The green element is marked for refinement since its value η_e is greater than a prescribed tolerance at each iteration.

B.3.2 AMR/C algorithm: `libMesh` simulations

In this thesis, we consider the `libMesh` for most of the applications. The applications that were simulated using `libMesh` and required AMR/C strategies are the 3D bubble rising problem and the SEIRD model. `libMesh` supports AMR/C by h-refinement (element subdivision), p-refinement (increasing the polynomial approximation order), and hp-refinement, which is a combination of both. Different from `FEniCS`, `libMesh` supports coarsening in all the previous AMR/C options. For this thesis, the h-refinement with hanging nodes is considered. Figure B.5 shows two levels of h-refinement and the presence of haging nodes on a quadrilateral mesh.

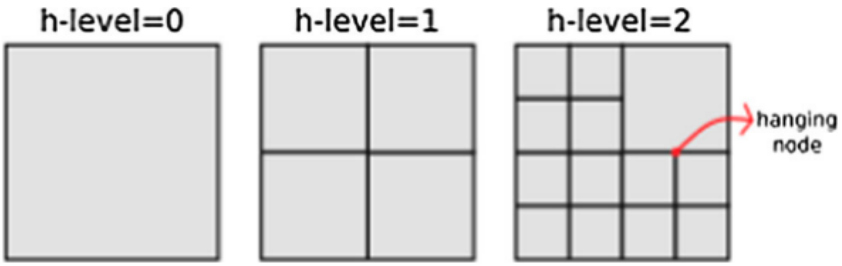


Figure B.5: Adaptive mesh refinement after two levels of refinement on three of the four original elements of $\mathcal{T}^{\text{coarse}}$.

For `libMesh` AMR/C strategy, the flux-jump described in Equation B.34 is also considered and the error vector η_s^2 is computed. The *flagging* strategy, however, differs from the `FEniCS` strategy. This is done by a statistical element flagging strategy. It is assumed that the element error $||\eta_e||$ is distributed approximately in a normal probability function. Here, the statistical mean μ_s and standard deviation σ_s of all errors are calculated. Whether an element is flagged is depending on a refining (r_f) and a coarsening (c_f) fraction. For all errors $||\eta_e|| < \mu_s - \sigma_s c_f$ the elements are flagged for coarsening and for all $||\eta_e|| > \mu_s - \sigma_s r_f$ the elements are marked for refinement. This procedure is described in Figure B.6. The refinement level is repeated until the desired element size is achieved.

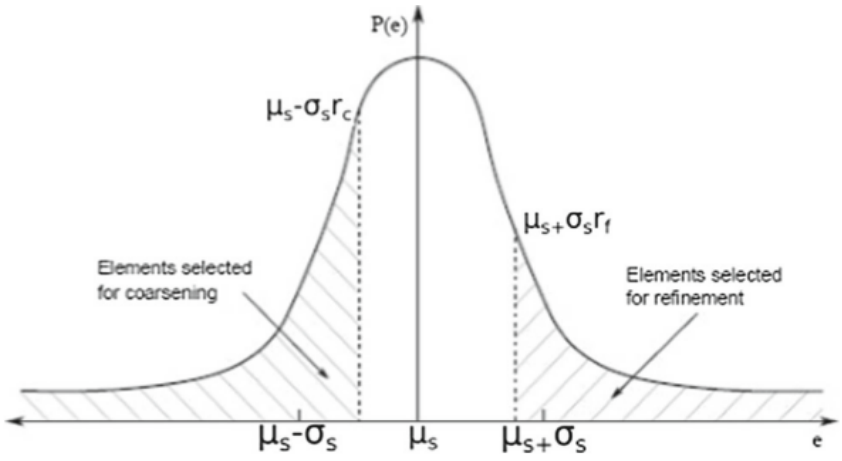


Figure B.6: Elements in hatched areas are flagged to statistical AMR/C process used by `libMesh`.

B.3.3 Solution projection

The projection or interpolation of numerical solutions between finite element meshes is a well-known computational mechanics subject. Solution projection is a crucial piece in the AMR/C strategy, since the solution computed on a previous mesh must be projected into the new generated mesh, as described in the previous sections. Many issues regarding boundary conditions, data visualization, or coupling arise from this kind of problem. The choice of a proper method to successfully project functions in different finite element spaces is not a trivial task since conservation may not be satisfied [84, 150, 151]. This issue is not addressed in the present study since the mesh projection occurs as a post-processing phase after computing the AMR/C solutions. Therefore it cannot lead to cumulative errors. Furthermore, we are also careful to choose reference target meshes with characteristic length equivalent with the existent in the AMR/C meshes to avoid any major accuracy losses. For the sake of generality, we consider the $\mathcal{L}^2-$projection as our method of choice, and it can be defined as follows [176]. Assuming that the solution on the donor mesh $\mathbf{u}^h(\mathbf{x}) = \sum_{j=1}^{n} \mathbf{u}_j N_j(\mathbf{x})$ to be projected onto the target mesh must satisfy the orthogonality condition,

$$(\mathbf{u}^h - \mathbf{u}_{proj}, \mathbf{v})_{\mathcal{L}^2} = 0 \qquad \forall \mathbf{v} \in V^{\psi} \tag{B.35}$$

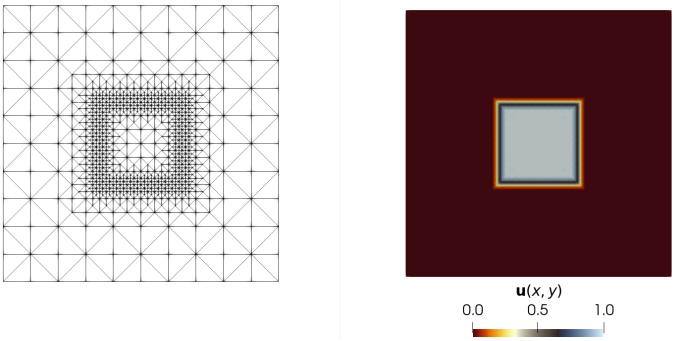
where V^{ψ} is a finite-dimensional subspace of $\mathcal{L}^2(\Omega)$ defined by the target mesh and the interpolant $\mathbf{u}_{proj}$ is the optimal interpolant in the $\mathcal{L}^2-$norm for V^{ψ}. The orthogonal projection can be defined in terms of the following linear system

$$\mathbf{M}\mathbf{u}^{proj} = \mathbf{P}\mathbf{u} \tag{B.36}$$

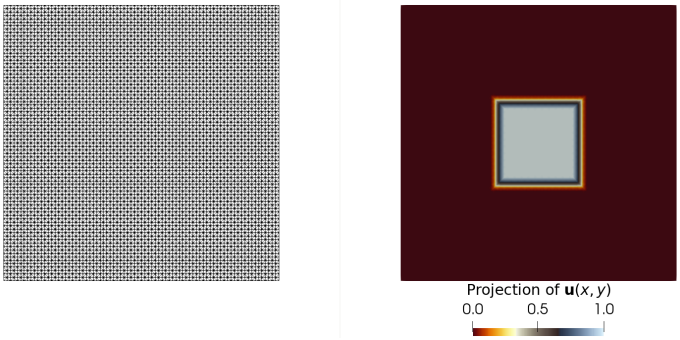
where $\mathbf{M}$ is the mass matrix and $\mathbf{P}$ is the projection matrix. The mass matrix is usual in finite element computations. Theoretical details regarding the proof that the $\mathbf{P}$ matrix is full-rank when every degree of freedom of the original mesh is also a degree of freedom of the reference mesh is seen on the appendix of [86]. Figure B.7 shows an example where a solution obtained by an adaptive mesh simulation is projected onto two meshes with different topologies. For this example, we consider the square domain $\Omega = [-1, 1] \times [-1, 1]$ and the function u defined in Ω such that

$$u = \begin{cases} 1, \text{ if } (x, y) \in [-0.3, 0.3] \times [-0.3, 0.3], \\ 0, \text{ otherwise.} \end{cases} \tag{B.37}$$

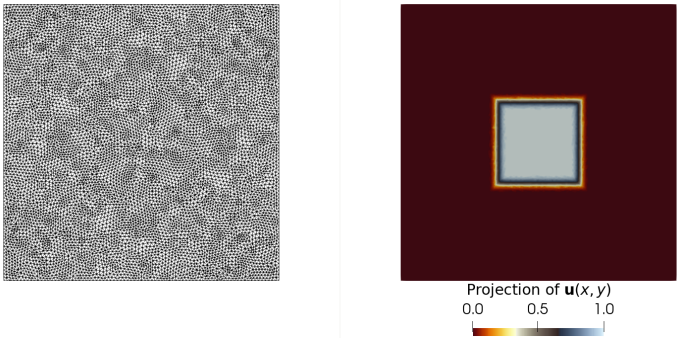
This function is approximated on a structured finite element mesh discretized into 10×10 cells where each cell is divided into two triangular elements. An AMR/C procedure is invoked to refine three times the transition between $\mathbf{u} = 1$ and $\mathbf{u} = 0$, creating a new mesh containing 1672 elements and 857 nodes. Figure B.7(a) shows the new mesh generated after the AMR/C procedure and the function approximated by the resulting function space. Two meshes - one structured and another unstructured - are considered for projection. The structured mesh contains 80×80 cells, resulting in 12800 triangular elements and 6561 nodes. The element sizes of the structured mesh are similar to the smallest elements in the adaptive mesh. The unstructured mesh presents a smaller characteristic length than the structured mesh and contains 17088 elements and 8705 nodes. Figures B.7(b) and B.7(c) show the proposed meshes and the projections of the solution onto the new finite element spaces. Note that the projected solutions are fairly accurate since their infinity norm is in good agreement with the adaptive mesh solution's infinity norm.



(a) Adaptive mesh (left) and solution (right), $||\mathbf{u}^h||_\infty = 1.000$.



(b) Structured mesh (left) and projected solution (right), $||\mathbf{u}^{proj}||_\infty = 1.000$.



(c) Unstructured mesh (left) and projected solution (right), $||\mathbf{u}^{proj}||_\infty = 0.999$.

Figure B.7: Comparison of different $\mathcal{L}^2-$projection examples on structured and unstructured meshes.

Appendix C

PADMe

Given the nature of the multiple applications presented in this thesis and the large variety of libraries used for numerical simulations and output file types, we created a centralized code for solving the problems presented in this work. The PADMe[1] library contains a collection of pre-processing, processing, and post-processing codes regarding snapshots and DMD. In this appendix, we describe in detail the usability of the library.

The library is self-contained, in the sense that all modules from the DMD workflow on Figure 2.4 can be altered in the PADMe code. It is required some usual Python libraries such as NumPy[141] for processing and Matplotlib[177] for data visualization. A `Makefile` is located in the root of the library code to create the environment (via `conda` [178] or other standard virtual environment tools such as `venv` or `virtualenv`), install the requirements for the library to work and create a Jupyter kernel. The code used in this thesis is available in Jupyter notebooks for easy visualization and replication, but PADMe is encouraged to be applied in Python scripts, given the complexity of data manipulation in DMD.

PADMe works based on two dictionaries and a list. The first dictionary, named `snapshot_ingestion_parameters` on the demonstration notebooks, dictates the data ingestion parameters. This dictionary is responsible for defining the parameters required for properly reading data from the simulation output files. It accommodates the ingestion, for instance, if the file is generated on FEniCS HDF5 files, FreeFem++[179] VTK, or libMesh ZFP-compressed HDF5 files. Each output file has its properties (i.e., legacy VTK files require line mapping from the solutions, HDF5 files require the dataset name, etc.), and this dictionary is responsible for these instructions. There are flags for the cases where snapshots are parallel, adaptive, or compressed. The list, namely the `filenames` list, is obtained through string manipulation and returns a list of absolute paths for each simulation output file.

[1] https://github.com/gf-barros/padme

- `'snapshots_matrix'` : the sliced snapshots_matrix `[starting_step : ending_step]`
- `'u'` : the $\mathbf{U}$ matrix from SVD
- `'s'` : the $\mathbf{\Sigma}$ matrix from SVD
- `'vt'` : the $\mathbf{V}^T$ matrix from SVD
- `'eigenvals_original'` : Eigenvalues from $\tilde{\mathbf{A}}$ matrix
- `'eigenvals_processed'` : Eigenvalues from $\tilde{\mathbf{A}}$ matrix after $log(\Lambda)/\Delta t$
- `'t'` : array containing time steps used for approximation
- `'dmd_matrix'` : the DMD approximation for all times existent in key `'t'`

Figure C.1: Keys of the DMD object attribute after processing.

Lastly, the second dictionary is the `dmd_parameters`, responsible for defining the SVD algorithm used, the number of basis vectors, and other hyperparameters for DMD modeling.

Given the two dictionaries and the list, we use functions to load and manipulate the snapshots. The `snapshots_assembly` function takes the output file type (such as `"vtk_freefem"` for ASCII VTK files generated from FreeFEM++ simulations or `"h5_fenics"` for HDF5 files generated on FEniCS simulations) and the `snapshot_ingestion_parameters` dictionary. The list containing the simulation output file names is passed inside the `snapshot_ingestion_parameters` dictionary. With the snapshots matrix assembled, it can be passed as an argument for instantiating the DMD class into an object along with the `dmd_parameters`. The DMD class is responsible for DMS's core computation. Two methods are invoked for that: the `.factorization()` method, used to invoke the SVD algorithm and compute the singular values, and the `.dmd_core()` method, used to compute the DMD modes, the eigenvalues, and the DMD approximation. The result is a `.dmd_approximation` attribute (a dictionary) containing singular values, vectors, eigenvalues, and the solution. Figure C.1 shows the output artifacts from DMD, extracted from the example notebook[2]. To evaluate the artifacts, the `PostProcessingDMD` class is instantiated with the artifacts obtained from the modeling. The `PostProcessingDMD` class is equipped with methods that plot the $\mathcal{L}^2$ relative errors, the Frobenius relative error, the eigenvalues, and more visualizations and post-processing tools. All plot methods can be invoked with three different backends: Matplotlib [177], Seaborn [180], and Plotly [181].

C.1 Jupyter Notebooks

This section is responsible for exploring one example Jupyter notebook containing the usage of the PADMe library. We focus on signal reconstruction of the SEIRD

[2] `https://github.com/gf-barros/padme/blob/master/notebooks/workspace/00_basic_usage.ipynb`

model example, although some other Jupyter notebooks containing the DMD processing for the 2D lock-exchange, 3D lock-exchange and bubble rising problems are also available.

We start by importing the required packages for this example.

```
%load_ext autoreload
%autoreload 2

import os
from pathlib import Path

from natsort import natsorted # Required for sorting filename correctly
from dotenv import find_dotenv # Required for locating project's root dir

from src import DMD # DMD class for ingesting and processing data
from src import snapshots_assembly # Function for assembling the Snapshots
    Matrix
from src import (
    load_notebook_params, # Function that loads predefined examples and
        paths
    generate_paths, # Function that creates the paths to datasets
    download_dataset, # Function that downloads the dataset from Kaggle
    unpack_kaggle_dataset, # Function that unzips the downloaded dataset
)
from src import PostProcessingDMD # Class used for postprocessing and
    dataviz
```

The `pathlib` package is useful for creating absolute file paths in all operational systems. The `natsort` package is used to sort the snapshots correctly, and the `dotenv` package tracks the project root correctly. We then import the PADMe modules for pre-processing, processing, and post-processing. Notice that we import two functions that download and unzip the SEIRD dataset. We use Kaggle for storing the data online through a private dataset that can be accessed through this link[3]. Figure C.2 shows the header of the Kaggle dataset. The function `download_dataset` uses the Kaggle API to access the dataset link and downloads it to the `data/00_-raw` folder of the project while function `unpack_kaggle_dataset` is responsible for unpacking the dataset into the snapshots folders and files. Instructions regarding the credentials for accessing the Kaggle API are in the example notebook.

The next step is to create the snapshots matrix. The snapshots for this example do not require any kind of pre-processing strategy, meaning that the construction

[3] `https://kaggle.com/datasets/79eceaf4176927bbdbcbc2480ae15dff18fe72a25fb57c7de0bf7167c5680369`

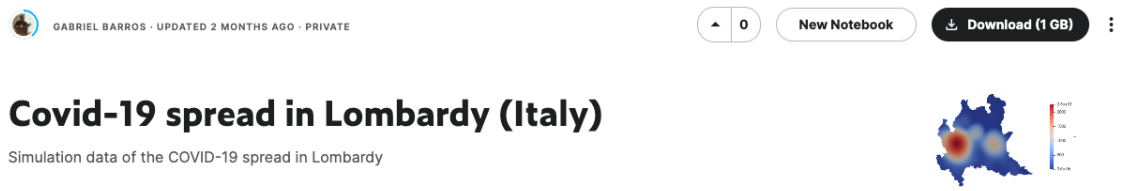


Figure C.2: The SEIRD dataset is available at Kaggle.

of the snapshots matrix is straightforward. We create the `filenames` list as:

```
os_walk_files = next(os.walk(dict_paths["snapshots_filepath"]))
folders = natsorted(os_walk_files[1])
filenames = [
    dict_paths["snapshots_filepath"]
    / Path(snapshot_folder)
    / Path(f"out_1_000_{str(i).zfill(5)}.h5")
    for i, snapshot_folder in enumerate(folders)
]
```

and consequently the `snapshot_ingestion_parameters` as:

```
snapshot_ingestion_parameters = {
    "filenames": filenames,
    "dataset": "s",
}
```

considering that the dataset to be explored is respective to the susceptible compartment. That said, the snapshots matrix can be created as:

```
dataset = snapshots_assembly(
    "h5_libmesh",
        snapshot_ingestion_parameters=snapshot_ingestion_parameters
)
dataset.shape
```

given that the simulation output files are HDF5 files generated by the `libMesh` library. The next step is to define the `dmd_parameters` dictionary.

```
dmd_parameters = {
    "factorization_algorithm": "randomized_svd",
    "basis_vectors": 25,
    "randomized_svd_parameters": {
        "power_iterations": 1,
        "oversampling": 20,
```

```
    },
    "starting_step": 0, # Can be 0 for susceptibles, but should be >0 for
        zero initialized fields.
    "dt_simulation": 0.25,
}
```

That is, our problem will be approximated with $r = 25$ basis vectors, using the rSVD algorithm with 1 power iteration and 20 random vectors for oversampling. The starting point for DMD is the initial conditions and the time step size for the problem is such $\Delta t = 0.25$. DMD can be evaluated as

```
dmd = DMD(dataset, dmd_parameters)
dmd.factorization()
dmd.dmd_core()
```

After computing the DMD approximation, we instantiate the `PostProcessingDMD` to plot the modeling artifacts using any of the three available data visualization libraries:

```
dmd_visualizer = PostProcessingDMD(dmd.dmd_approximation)
dmd_visualizer.plot_singular_values("seaborn")
dmd_visualizer.plot_eigenvalues("plotly")
dmd_visualizer.compute_temporal_l2_norm("matplotlib")
```

Appendix D

Data Compression

With the extreme amounts of data produced by large-scale scientific simulations on leadership high-performance computing (HPC) systems and scientific in- struments, data management has become a serious problem [94]. Strategies regarding data compression are often considered for large applications where the raw output data transfer and storage are prohibitive due to limited I/O bandwidth. Data compression is a widely studied topic and the search for efficient ways to store data with error bounds is an active research topic. The idea behind compression schemes for floating-point data is to take fixed-precision values and compress them to a variable-length bit stream [182].

In general, the compressors can be split into two categories, lossless and lossy data compressors. Generic lossless compression algorithms such as Gzip and BZIP have shown to be less effective for float-point data sets generated by scientific applications [183]. Although there are lossless compressors designed for numerical data [183–185], they have limited compression rates due to high precision representation. In this way, the least significant bits of these data have random characteristics, disfavoring the prediction of recurring values. However, leading bits tend to favor the development of a new class of error-bounded lossy compression algorithms [182, 186]. In [119] we find an assessment of several state-of-the-art lossy and lossless scientific data compressors.

In the present thesis we consider the ZFP library for compressed floating-point values [182]. The ZFP library is recognized as one of the most effective high-speed lossy floating-point data compressors [119]. ZFP operates on d-dimensional arrays ($d \in [1, 4]$) by partitioning them into blocks of 4^d values. Each block is compressed/decompressed entirely independently from all other blocks. The scheme uses orthogonal block transform and embedded coding ideas to achieve a high compression ratio. Therefore, ZFP users could control a block's quality and compressed size by modifying four library parameters: *minbits*, *maxbits*, *maxprec* and *minexp*. The *minbits* and *maxbits* parameters define the minimum and the maximum number

of compressed bits used to represent a block, the *maxprec* governs the number of most significant uncompressed bits encoded per transform coefficient and provides a mechanism for controlling the relative error. The *minexp* defines the smallest absolute bit plane number encoded and should be interpreted as the base-2 logarithm of an absolute error tolerance. At a high level, ZFP provides five predefined compression modes that affect these parameters: expert, fixed-rate, fixed-precision, fixed-accuracy, and reversible. The first four modes allow a higher compression ratio with controllable loss of accuracy, while the last one performs lossless compression. This work adopts the fixed-accuracy mode that, given a tolerance, sets the *minexp* parameter to `floor(log2(tolerance))`. In this thesis, we test the cases where lossy compression is performed with tolerances of 10^{-6} and 10^{-3}, being the latter the most aggressive scheme tested. In practice, this means that for an uncompressed value, f, and a reconstructed value, g, the absolute difference $|f - g|$ is at most 2^{minexp}. Details about these compression modes can be found in [187], and a rigorous error analysis for all of them is done in [88]. In general, according to [119], ZFP tends to over-preserve the compression precision.

The benefits of data compression can be obtained in HDF5 through user-provided dynamically loaded filters. HDF5 has the concept of a filter pipeline, which is just a series of operations performed on data. The library provides flexibility to use various data compression filters on individual HDF5 datasets. Compressed data is stored in chunks and automatically decompressed by the library and filter plugin when a chunk is accessed. The primary compression filters included in the library are GZIP and SZIP. However, HDF5 also allows third-party filters such as LZO, BZIP2, LZF, LZ4, FPZIP. All filters mentioned before apply lossless data compression. Some of the filters available for lossy compression are SZ, FPZIP, and ZFP. The complete list of all filters can be found at the HDF Group website [1].

[1] `https://portal.hdfgroup.org/display/support/Registered+Filter+Plugins`